

Титульник

**Тема Исследование и разработка интеллектуальных методов анализа
профиля автора русскоязычного текста**

Задание 1

Задание 2

АННОТАЦИЯ

Тема: «Исследование и разработка интеллектуальных методов анализа профиля автора русскоязычного текста».

Объем дипломной работы: 83 страницы (без приложений), содержащих 30 рисунков и 10 таблиц. При написании работы использован 31 источник.

Объектом исследования являются процессы и методы анализа текстов на естественном языке, направленные на выявление характеристик автора.

Предметом исследования являются методы обработки естественного языка и методы машинного обучения, применяемые для решения задачи автоматического профилирования.

Цель работы - разработка и экспериментальное обоснование эффективности веб-сервиса для автоматического профилирования автора текста, направленного на повышение точности и качества персонализации цифровых платформ.

Во введении обоснована актуальность задачи автоматического профилирования в контексте современных цифровых платформ, сформулированы цель, задачи и научная новизна исследования.

В первой главе проведен комплексный анализ современных подходов к автоматическому профилированию автора текста, систематизированы способы извлечения признаков и методы классификации.

Во второй главе представлено теоретическое описание системы, включая формальную модель обработки текста, методы извлечения признаков и алгоритмы классификации.

В третьей главе описаны проектирование и реализация программной системы, представлены архитектурные решения и компоненты системы.

В четвертой главе проведено тестирование методов автоматического профилирования, выполнено сравнение эффективности различных моделей машинного обучения.

В заключении подведены итоги работы, сформулированы выводы и практические рекомендации по применению системы автоматического профилирования.

Приложения содержат исходный код системы, исходный текст для контрольного примера

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	6
СПИСОК ИСПОЛЬЗОВАННЫХ СОКРАЩЕНИЙ.....	8
ВВЕДЕНИЕ.....	9
ГЛАВА 1. ОБЗОР МЕТОДОВ И ПОДХОДОВ К АВТОМАТИЧЕСКОМУ ПРОФИЛИРОВАНИЮ АВТОРА ТЕКСТА.....	12
1.1 Постановка задачи.....	12
1.2 Методы обработки естественного языка.....	13
1.3 Автоматическое профилирование.....	15
1.4 Лингвистические признаки для автоматического профилирования.....	17
1.5 Обзор моделей машинного обучения.....	19
1.6 Обзор подходов в прогнозировании составного профиля.....	21
1.7 Обзор существующих систем.....	23
1.8 Выбор подходящего метода.....	24
1.9 Выводы по главе.....	25
ГЛАВА 2. ТЕОРЕТИЧЕСКОЕ ОПИСАНИЕ СИСТЕМЫ.....	26
2.1 Формальное описание методов и процессов автоматического профилирования.....	26
2.2 Описание функций агрегации и формализация признаков.....	27
2.3 Описание исходных данных.....	28
2.4 Описание алгоритма обработки документа.....	29
2.5 Анализ признакового пространства.....	31
2.6 Обучение и оптимизация моделей классификации.....	40
2.7 Выводы по главе.....	43
ГЛАВА 3. ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ СИСТЕМЫ.....	44
3.1 Концептуальное проектирование системы.....	44
3.2 Диаграмма вариантов использования.....	47
3.3 Диаграмма классов системы.....	48

3.4 Диаграмма последовательности.....	51
3.5 Диаграмма компонентов.....	53
3.6 Описание архитектуры развертывания системы.....	54
3.7 Методы подбора гиперпараметров.....	56
3.8 Контрольный пример.....	56
3.9 Выводы по главе.....	63
ГЛАВА 4. ТЕСТИРОВАНИЕ МЕТОДОВ АВТОМАТИЧЕСКОГО ПРОФИЛИРОВАНИЯ АВТОРА ТЕКСТА.....	64
4.1 Тестирование стабильности моделей.....	64
4.2 Тестирование обобщённости моделей.....	71
4.3 Результаты экспериментов.....	76
4.4 Выводы по главе.....	77
ЗАКЛЮЧЕНИЕ.....	78
СПИСОК ЛИТЕРАТУРЫ.....	80
ПРИЛОЖЕНИЕ А. КОНТРОЛЬНЫЙ ПРИМЕР.....	84
ПРИЛОЖЕНИЕ Б. ИЗВЛЕКАЕМЫЕ ПРИЗНАКИ.....	85
ПРИЛОЖЕНИЕ В. ИСХОДНЫЙ КОД.....	89

СПИСОК ИСПОЛЬЗОВАННЫХ СОКРАЩЕНИЙ

API (Application Programming Interface) - программный интерфейс приложения, набор методов для взаимодействия между программными компонентами.

BERT (Bidirectional Encoder Representations from Transformers) - двунаправленная модель представлений энкодера на основе архитектуры Transformer. Предобученная языковая модель.

CAC (Customer Acquisition Cost) - стоимость привлечения клиента.

NLP (Natural Language Processing) - область искусственного интеллекта, занимающаяся взаимодействием компьютера и человеческого языка.

POS-tagging (Part-of-Speech tagging) - морфологическая разметка, определение частей речи в тексте.

NER (Named Entity Recognition) - распознавание именованных сущностей, задача извлечения структурированной информации из текста.

LLM (Large Language Model) - большие языковые модели.

HTML (HyperText Markup Language) - язык разметки гипертекста, используется для создания веб-страниц.

IDEF0 (Integration Definition for Function Modeling) - методология функционального моделирования, используемая для описания бизнес-процессов.

SVC (Support Vector Classifier) - метод опорных векторов для задач классификации.

RBF (Radial Basis Function) - радиальная базисная функция.

MLP (Multilayer Perceptron) - многослойный перцептрон.

NB (Naive Bayes) - наивный байесовский классификатор.

GBM (Gradient Boosting Machine) - градиентный бустинг.

RF (Random Forest) - модель случайного леса.

LR (Logistic Regression) логистическая регрессия.

KNN (k-Nearest Neighbors) - метод k ближайших соседей.

ВВЕДЕНИЕ

В современной цифровой среде сталкиваются интересы двух сторон: клиенты ожидают высокого качества сервиса и клиентоориентированности, в то время как бизнес стремится к повышению основных метрик и минимизации издержек. Ключом к удовлетворению обеих потребностей является глубокое понимание пользователя, которое может быть достигнуто в том числе и за счет составления его профиля.

Профиль пользователя представляет собой отражение информации о человеке. В традиционном подходе профиль заполняется самим пользователем вручную. Однако этот метод сталкивается с серьезной проблемой: пользователи зачастую либо оставляют поля незаполненными, либо намеренно искажают информацию о себе, что делает традиционные подходы к таргетированию неэффективными. Таким образом, возникает необходимость в автоматическом составлении профиля.

Одним из возможных источников данных о профиле является текст, который представляет собой цифровой отпечаток автора. В современном мире пользователи оставляют множество текстовых следов, в которых неявно закодированы их личностные и демографические характеристики.

За извлечение информации из неструктурированного текста отвечают методы обработки естественного языка (NLP). Современные технологии NLP способны эффективно обрабатывать огромные объемы текстовых данных, и их высокая результативность убедительно доказана исследованиями [3, 6].

Однако, несмотря на доказанную эффективность методов и острую потребность рынка, готовых программных решений для автоматического профилирования, особенно для русскоязычных текстов, практически не существует. Разработка такого решения позволит удовлетворить потребности бизнеса и клиентов. В связи с этим данная работа направлена на исследование современных методов автоматического профилирования и создание практического инструментария, реализующего данные методы для решения прикладных задач.

Объект исследования - процессы и методы анализа текстов на естественном языке, направленные на выявление характеристик автора.

Предмет исследования - алгоритмы и программные решения для автоматического профилирования автора текста на русском языке, а также оценка их практической применимости и эффективности.

Цель работы - разработка и экспериментальное обоснование эффективности веб-сервиса для автоматического профилирования автора текста, направленного на повышение точности и качества персонализации цифровых платформ за счет автоматического определения атрибутов автора.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести обзор существующих методов и подходов к автоматическому профилированию автора текста на русском языке;
2. Выполнить анализ и обоснование выбора методов, применимых для исследования;
3. Разработать процесс извлечения и подготовки признаков, используемых при моделировании;
4. Построить и обучить модели, реализующие выбранные методы профилирования;
5. Спроектировать и разработать веб-сервис, реализующий функциональность автоматического профилирования автора текста;
6. Провести тестирование и оценку качества разработанных методов.

Научная новизна работы заключается в комплексном исследовании существующих методов автоматического профилирования автора текста применительно к русскоязычному материалу, а также в разработке и реализации веб-сервиса, объединяющего и обеспечивающего практическое применение различных моделей профилирования в единой системе.

Планируемые результаты работы:

- Разработанный веб-сервис, предоставляющий функционал для автоматического определения профиля автора по произвольному русскоязычному тексту;
 - Полученные в ходе сравнительного эксперимента количественные оценки эффективности ряда современных методов машинного обучения для задачи профилирования;
 - Практические рекомендации по применению методов автоматического профилирования для русскоязычных текстов с учетом их характеристик.
- На защиту выносятся следующие результаты:
- Веб-сервис для автоматического профилирования автора русскоязычных текстов;
 - Результаты сравнительного анализа эффективности методов машинного обучения для задачи профилирования;
 - Практические рекомендации по применению методов профилирования.

ГЛАВА 1. ОБЗОР МЕТОДОВ И ПОДХОДОВ К АВТОМАТИЧЕСКОМУ ПРОФИЛИРОВАНИЮ АВТОРА ТЕКСТА

1.1 Постановка задачи

Персонализация цифровых сервисов традиционно опирается на данные профиля, которые пользователь заполняет вручную. Однако на практике такой подход сталкивается с рядом проблем. Пользователи нередко оставляют профильные поля пустыми или указывают недостоверную информацию. В результате системы рекомендаций, таргетинга и персонализации опираются на неполные или искажённые данные, что приводит к снижению эффективности бизнес-процессов. Последствием становится ухудшение ключевых коммерческих показателей: уменьшается релевантность коммуникаций, растёт стоимость привлечения клиента (CAC), а отдача от маркетинговых и рекламных кампаний, отношение числа пользовательских переходов к числу показов - снижаются.

Эту проблему можно решить через автоматическое профилирование пользователя. При помощи моделей машинного обучения, строящихся на основе признаков, извлечённых из текстовых сообщений пользователя, становится возможным определять его демографические характеристики и интересы без необходимости прямого опроса. Методы обработки естественного языка (NLP) выступают ключевым инструментом, позволяющим перейти от неструктурированного текста к цифровому представлению, характеризующему стиль и содержание авторского письма. Эти признаки, образуют основу для последующего построения прогнозных моделей.

Эффективность применения NLP в смежных задачах подтверждается исследованиями: например, внедрение чат-ботов с NLP-функционалом на платформе Tenable позволило удвоить коэффициент конверсии за счёт качественной автоматической классификации пользователей [12]. В области кибербезопасности обзор российских исследований Котенко И.В. и Абраменко Г.Т. указывает на активную разработку методов анализа текста с помощью больших языковых моделей (Large Language Model) для поиска угроз и

обнаружения аномалий [13], что косвенно подтверждает потенциал NLP для задач профилирования.

Таким образом, разрабатываемая система призвана решить задачу автоматического профилирования на основе методов обработки естественного языка, преобразуя лингвистические особенности письменной речи автора в формализованный вектор признаков и используя его для определения демографических атрибутов. Такой подход позволяет преодолеть ограничения ручного заполнения и повысить точность персонализации в условиях анонимности пользователей.

Формальная постановка задачи:

- Дано: Корпус текстовых документов $D = d_1, d_2, \dots, d_n$ и множество целевых характеристик (атрибутов профиля) $P = a_1, a_2, \dots, a_n$, где каждая характеристика профиля a_i имеет своё множество допустимых значений;
- Требуется: Построить такую модель M , которая для нового, ранее не встречавшегося текста d_{new} сможет предсказать наиболее вероятный вектор его характеристик профиля P ;
- Выходные данные: профиль автора $P = (v_1, v_2, \dots, v_m)$, где v_i - значение характеристики a_i .

Концептуально, модель реализует отображение $M: d_{new} \rightarrow P$

Таким образом, в общей форме задачу можно свести к многоцелевому прогнозированию, где каждое измерение соответствует одной характеристике профиля.

Далее будут рассмотрены методы и подходы, которые можно использовать для решения этой задачи.

1.2 Методы обработки естественного языка

Обработка естественного языка - это область искусственного интеллекта, направленная на изучение методов и алгоритмов автоматической обработки и анализа текстовых данных. К типовым задачам NLP относятся как базовые лингвистические операции: морфологический и синтаксический разбор,

разметка частей речи (POS-tagging), лемматизация, извлечение именованных сущностей (NER), так и прикладные решения: машинный перевод, автореферирование, тематическое моделирование.

Развитие NLP прошло путь от систем, основанных на правилах, к статистическим методам (скрытые марковские модели, n-граммы) и, наконец, к современной эре глубокого обучения. Переломным моментом стало появление архитектуры Transformer, которая легла в основу таких прорывных языковых моделей, как BERT. Эти модели, предобученные на огромных корпусах текстов, позволяют извлекать из слов и предложений контекстуализированные векторные представления текста. Использование предобученных моделей и их тонкая настройка под решение целевой задачи позволяет достигать высоких результатов, что подтверждается исследованиями в области атрибуции текста, где, например, мультязычная модель BERT показала наилучшую точность (83,2%) в задаче определения возраста автора русскоязычного текста [2].

Эффективность методов NLP напрямую зависит от специфики языка. Модели, обученные на одном языке (например, английском), часто демонстрируют низкое качество на других из-за фундаментальных лингвистических различий.

Русский язык обладает рядом особенностей, критически важных для применения методов NLP:

1. Богатая морфология: сложная система спряжений, склонений и падежей;
2. Свободный порядок слов;
3. Распространенная омонимия.

Дополнительную сложность представляет анализ живой цифровой коммуникации. Исследования подтверждают, что атрибуция в социальных сетях сопряжена с уникальными трудностями: низкий объем и неструктурированность текстов, обилие сленга и шумов, а также преднамеренная или непреднамеренная неграмотность пользователей, что делает этап предобработки текста критически важным [2,3,5].

При этом ключевой особенностью таких текстов является их машинописная природа, которая, как отмечает Кривошапова Н.В., напрямую влияет на эмоциональное воплощение: «Тексты, написанные собственноручно, отличаются от текстов, набранных на компьютере, эмоциональностью - в них она представлена ярче» [5].

Перед извлечением лингвистических признаков текст подлежит предварительной предобработке, которая включает несколько стандартных этапов. На начальном этапе выполняется токенизация - разбиение текста на отдельные слова и предложения. Далее проводится нормализация, включающая приведение текста к единому регистру и удаление лишних токенов. Ввиду богатой морфологии русского языка ключевым этапом является лемматизация, которая приводит слова к их словарной форме.

После завершения предобработки из текста можно извлечь статистические признаки, векторы и иные числовые значения для последующего машинного обучения. Простейшими подходами к векторизации являются частотные методы: модель «мешка слов» и TF-IDF (Term Frequency - Inverse Document Frequency), которые формируют разреженные векторные представления. Альтернативой выступают плотные дистрибутивные представления (Word2Vec, FastText и др.), кодирующие семантику слова на основе его контекстного окружения в корпусе. Современные контекстуальные модели (BERT и его аналоги) создают векторные представления, зависящие от окружения слова в предложении и дающие более выразительные признаки для сложных задач.

1.3 Автоматическое профилирование

Исторически задача установления авторства решалась с помощью экспертных методов, когда специалист-лингвист анализировал почерк, стилистические особенности и идиолект автора. Эти методы и до сих пор используют при расследовании [6]. В отличие от авторской атрибуции, где цель - однозначно отнести текст конкретному известному автору, профилирование стремится выявить обобщённые характеристики, которые могут быть общими для многих авторов. Современные подходы к автоматическому

профилированию принято разделять на два направления: контентно-зависимые и стилометрические методы.

Контентно-зависимый подход фокусируется на тематике и семантике текста. Классические исследования Schler [14] и Rao показали, что определенные темы и слова статистически значимо коррелируют с демографическими атрибутами автора (например, спорт и технологии - с мужчинами, а семья и мода - с женщинами). Преимущество подхода - интерпретируемость признаков. Однако он сильно зависит от тематики корпуса и плохо обобщается на другие области, так как контентные признаки могут «зашумлять» стилистические паттерны [16].

Стилометрический подход анализирует формальные и статистические особенности авторского стиля: лексическое разнообразие, частотность слов, синтаксис и пунктуацию, что рассматривается как «отпечаток пальца» автора. Эффективность подхода подтверждена многочисленными исследованиями. Koppel показал, что использование 250 наиболее частотных слов позволяет определить пол автора с точностью около 70-80% [16]. Исследования [8, 17] выявили специфические языковые маркеры для разных полов в английском и русском языках. Универсальность стилометрии демонстрирует её применение даже для анализа исходного кода [8].

Таким образом, стилометрия предлагает мощный и методологически обоснованный аппарат для решения задачи профилирования, фокусируясь на подсознательных и устойчивых паттернах письменной речи.

Однако стилометрия имеет ограничения. Её эффективность зависит от объема текста (минимальный порог - около 250 слов [19]), она уязвима к сознательной имитации стиля и обфускации [1]. Ключевой проблемой является жанровая зависимость: модель, обученная на одном жанре (например, личные письма), может плохо работать на другом (формальные описания). Исследования Литвиновой Т.А. подтверждают, что влияние жанра может быть сильнее, чем влияние пола или возраста автора [4, 18]. Эта проблема актуальна и для других языков, например, индонезийского [9]. В работе по автоматической атрибуции

индонезийских литературных текстов также подчеркивается, что точность атрибуции заметно снижается при работе с краткими или упрощенными текстами, где набор стилистических маркеров оказывается недостаточно репрезентативным [9].

При решении прикладных задач профиль автора может наполняться самыми разными характеристиками: от демографических (пол, возраст) и социальных (групповая принадлежность) до индивидуально-психологических (характер, образование). В работе А.С. Романова подчеркивается, что, несмотря на это разнообразие, все атрибуты автора выстраиваются в иерархическую структуру. Ключевую роль в ней играют первичные признаки (пол и возраст), которые служат основой для определения вторичных параметров и последующего построения модели уникальной языковой личности [1].

1.4 Лингвистические признаки для автоматического профилирования

Анализ современных исследований в области автоматического определения характеристик автора текста выявил устойчивый набор лингвистических признаков, демонстрирующих статистически значимую корреляцию с демографическими и социально-личностными параметрами.

Как показано в работе Кривошаповой Н.В., именно «анализ синтаксического, лексико-фразеологического и стилистического уровней текста представляет наибольший интерес и наибольшую сложность» для атрибуции [5]. Автор отмечает, что такие структурные маркеры, как характер синтаксических конструкций (склонность к сложным предложениям, использование вставных конструкций), специфика пунктуации (например, активное использование тире или многоточий) и композиционные привычки (например, трехчастная структура блог-поста с кольцевой композицией), формируют устойчивый каркас авторского стиля [5]

Систематизация этих признаков позволяет выделить несколько ключевых категорий, наиболее информативных для решения задачи профилирования.

1.4.1 Лексические признаки

На лексическом уровне прослеживаются устойчивые, хотя и не универсальные, закономерности. Исследования указывают на то, что для мужского авторского стиля может быть характерно более частое использование терминологической лексики, профессиональных жаргонизмов и ссылок на политику или технологии. В противоположность этому, женский стиль часто отличается преобладанием оценочной лексики, эмоционально окрашенных междометий и лексики, связанной с межличностными отношениями и сферой чувств [24, 26, 29].

Проявляются и возрастные различия. Молодежь активно использует неформальную лексику, сленг и англицизмы. Их тексты характеризуются более высоким процентом местоимений и глаголов, особенно в настоящем времени, что отражает ориентацию на текущий момент и личный опыт. Взрослые и пожилые авторы, напротив, демонстрируют более богатый и разнообразный словарный запас, используют больше отглагольных существительных, абстрактных понятий и профессиональной лексики [2, 11, 26].

1.4.2 Морфологические признаки

Морфологический уровень анализа, особенно для русского языка с его развитой парадигмой, является высокоинформативным. Утверждения о том, что женщины статистически значимо чаще используют местоимения первого лица единственного числа, качественные прилагательные и частицы, находят подтверждение в ряде исследований. Мужской стиль может характеризоваться более частым использованием существительных, артиклей и числительных [18, 24, 26, 27, 29].

Возрастная динамика на морфологическом уровне проявляется в использовании грамматических времен. Для молодежи характерно доминирование глаголов в настоящем времени, в то время как старшее поколение чаще обращается к прошедшему времени, что отражает тенденцию к рефлексии и описанию опыта. Кроме того, с возрастом может увеличиваться использование причастий и деепричастий, что указывает на усложнение грамматического строя речи [26].

1.4.3 Статистический и синтаксический уровень

Статистические показатели текста предоставляют количественные метрики для дифференциации авторских стилей. Наиболее информативными оказываются индекс лексического разнообразия (TTR), который имеет сложную зависимость от пола и возраста, а также средняя длина предложения и слова [2, 18, 25, 29].

Синтаксис также является важным дифференцирующим признаком. Женские тексты чаще содержат конструкции, выражающие неуверенность или смягчение (например, «мне кажется», «возможно»), в то время как мужские могут быть более прямолинейными и императивными [24].

Возрастная динамика здесь выражена ярче. Молодежь использует более короткие предложения, что отражает влияние устной речи и цифровой коммуникации. Взрослые авторы демонстрируют владение сложноподчиненными конструкциями [2, 11].

1.4.4 Пунктуационный и структурный уровень

Некоторые исследования отмечают, что женщины склонны использовать больше восклицательных знаков, многоточий и эмотиконов, что может свидетельствовать о большей эмоциональной экспрессии [24].

С возрастной точки зрения, молодые авторы активно используют нестандартную пунктуацию (например, множественные знаки или ее полное отсутствие), что является стилевой маркировкой и частью цифрового этикета. Старшее поколение придерживается более строгих, канонических правил пунктуации, используя ее в первую очередь для синтаксического структурирования текста, а не для передачи эмоций [26].

1.5 Обзор моделей машинного обучения

В современных исследованиях по автоматическому профилированию авторов текста наблюдается разнообразие применяемых алгоритмов машинного обучения, что отражает поиск оптимального баланса между точностью

классификации, интерпретируемостью результатов и вычислительной эффективностью.

Линейные методы, включая логистическую регрессию и метод опорных векторов с линейным ядром, остаются популярным выбором для начальных экспериментов благодаря своей прозрачности и низким вычислительным требованиям. Исследование Argamon демонстрирует эффективность логистической регрессии с L1-регуляризацией, достигшей точности 76.2% при определении пола автора на корпусе англоязычных блогов [17]. Сходные результаты получены Koppel, где линейный SVM показал точность 84.3% в аналогичной задаче [16]. Применение нелинейных ядер в SVM оправдано в случаях сложных взаимодействий между стилометрическими признаками.

Вероятностные подходы, представленные наивным байесовским классификатором, демонстрируют устойчивую эффективность в задачах анализа эмоциональной окраски и базового стилометрического профилирования. Согласно исследованию Frank и Bouckaert, данный метод уступает современным алгоритмам бустинга менее чем на 2% по точности классификации [20], что делает его конкурентным решением для задач с ограниченными вычислительными ресурсами.

Метод k-ближайших соседей находит применение в исследованиях демографических характеристик и эмоционального анализа коротких текстов. Ключевыми преимуществами алгоритма являются простота реализации и отсутствие этапа обучения, однако метод демонстрирует ограниченную масштабируемость и чувствительность к выбору метрики расстояния.

Ансамблевые алгоритмы занимают лидирующие позиции по точности классификации в современных исследованиях. Random Forest, благодаря устойчивости к переобучению и способности эффективно работать с разнородными признаками, успешно применяется в задачах стилометрического анализа [10]. Алгоритм обеспечивает дополнительное преимущество в виде встроенной оценки важности признаков, что ценно для интерпретации результатов. Градиентный бустинг демонстрирует особую эффективность в

сложных задачах классификации, достигая точности до 92% при анализе эмоциональной окраски текстов в социальных медиа [22].

Нейросетевые архитектуры, несмотря на свой потенциал, показывают неоднозначные результаты в задачах профилирования авторов. Сравнительные исследования указывают, что традиционные методы машинного обучения могут превосходить сложные нейросетевые модели, особенно при работе с ограниченными объемами обучающих данных [7, 20, 21].

1.6 Обзор подходов в прогнозировании составного профиля

При задаче автоматического профилирования автора «профиль» часто представляет собой совокупность нескольких атрибутов - пол, возрастная категория, уровень образования, профессиональная группа и т.п. С практической точки зрения выделяются два принципиально распространённых подхода к решению такой составной задачи: плоская и иерархичная классификация

1.6.1 Плоская классификация

Плоская (мультицелевая) классификация рассматривает задачу как одновременное прогнозирование всех атрибутов профиля с помощью единой модели. Плоская классификация реализует задачу профилирования автора как стандартную многоклассовую классификацию, где каждый целевой класс представляет собой уникальную комбинацию всех прогнозируемых атрибутов профиля.

Данный подход, часто реализуемый через единую модель с одним выходным слоем, обеспечивает целостное рассмотрение профиля автора и позволяет модели самостоятельно выявлять сложные взаимосвязи между демографическими характеристиками. Основное преимущество плоской классификации заключается в ее архитектурной простоте - требуется разработка и обслуживание только одной модели, что упрощает процесс развертывания и мониторинга системы.

Вместе с тем плоская мультиклассовая схема имеет и существенные недостатки. Число классов растёт как произведение мощностей отдельных

атрибутов, что усиливает проблему разреженности и требует существенно большего объёма размеченных примеров для каждой комбинации; классический следствие - ухудшающаяся точность на редких сочетаниях и повышенный риск переобучения. Кроме того, метрика качества в такой постановке обычно ориентируется на «совпадение всего вектора», поэтому даже небольшая ошибка по одному из атрибутов приводит к провалу по всей комбинированной метке - эффект, близкий к каскадной ошибке. Наконец, плоская модель теряет гибкость при расширении набора профилируемых атрибутов: добавление новой категории возрастной группы или ещё одного измерения резко увеличивает число целевых классов и усложняет интерпретацию предсказаний.

1.6.2 Иерархическая классификация

Иерархическая (поэтапная, каскадная) классификация отражает иную интуицию: некоторые атрибуты естественно определяются в последовательности или обладают вложенной структурой, поэтому выгоднее сначала определить «широкую» категорию, а затем, опираясь на результат, уточнить более детальные признаки. При классическом иерархическом сценарии система сначала предсказывает один атрибут (например, пол), затем на основании этого решения выбирается специализированная модель для предсказания лингвистических маркеров, характерных для данной подгруппы.

Такой подход позволяет использовать специализированные модели и признаки для отдельных ветвей, что особенно полезно при сильной неоднородности данных или когда распределение меток по листьям иерархии крайне несбалансировано. Иерархия естественно отображает бизнес-логику и облегчает интерпретацию результата: ошибки верхнего уровня дают понятный диагноз, а экспертиза может быть сфокусирована на проблемных ветках.

Анализ ошибок классификатора, проведенный Литвиновой Т.А., выявил важную закономерность: модель демонстрировала существенно более высокую точность в определении пола автора по сравнению с прогнозированием возрастной группы [4]. Данное наблюдение свидетельствует о различной

степени выраженности «гендерного сигнала» в текстах представителей разных возрастных групп.

1.7 Обзор существующих систем

Обзор существующих инструментов показывает, что на рынке отсутствуют общедоступные платформы, предоставляющие полный функционал автоматического профилирования для русскоязычных текстов. В рамках данного обзора были проанализированы несколько систем, которые реализуют отдельные смежные задачи и представляют значительный интерес с методологической точки зрения.

1.7.1 Bukvik

Bukvik - это многоязычная исследовательская платформа в экосистеме «LitTerra», ориентированная на цифровые гуманитарные исследования. Её ключевая концепция - стилистический профиль, который представляет текст как многомерное пространство количественных признаков. Это позволяет проводить объективное сравнение текстов и авторов.

Система поддерживает анализ частей речи (POS-tagging), этимологический анализ, сетевой анализ («Общество слов»), а также расчет относительной энтропии и анализ тональности.

Несмотря на работу со стилем, фокус Bukvik смещен в сторону атрибуции авторства и сравнительного литературного анализа, а не определения профиля автора. Bukvik полезен для межтекстового анализа, визуализации и проведения экспериментов по стилометрии, но это не имеет функционала по решению задач автоматического профилирования

1.7.2 RuLex

«RuLex» («RuLingva») представляет собой автоматический анализатор сложности и читабельности текстов, изначально ориентированный на оценку учебных материалов и тексты начальной школы. Её цель - помочь преподавателям подбирать тексты, соответствующие уровню подготовки учащихся.

Инструмент вычисляет множество параметров текста (лексические и структурные характеристики), определяет читательский диапазон и уровень сложности по заранее обученным моделям; платформа сопровождается публикациями и корпусной работой, выполняет подсчёт частотностей и предлагает списки терминов.

RuLex полезен там, где требуется детальный количественный разбор текста и оценка его пригодности для определённых возрастных/образовательных групп, но он специализирован на оценке читабельности и педагогических метриках, а не на построении составного профиля автора.

1.7.3 Textometr

Textometr - это специализированный онлайн-инструмент для автоматического определения уровня сложности текстов на русском языке для иностранных учащихся. Сервис рассчитывает индексы читабельности (включая адаптированные формулы Флеша), уровень CEFR (Common European Framework of Reference for Languages - общеевропейская шкала уровней владения языком), коэффициенты лексического разнообразия, ключевые слова и частотные списки, предоставляет морфологический и синтаксический разбор и ряд вспомогательных инструментов.

Textometr хорошо подходит для лингвистической и педагогической аналитики и для извлечения ряда полезных признаков (лексических и синтаксических), которые могут служить входом в систему профилирования, однако сам по себе это инструмент анализа текста, а не завершённая платформа составления профилей авторов.

1.8 Выбор подходящего метода

В качестве базовой парадигмы выбран стилометрический подход извлечения лингвистических признаков, который фокусируется на формальных характеристиках текста, инвариантных к его тематическому содержанию. Такой выбор обусловлен методологической корректностью при работе с авторским

стилем и большей устойчивостью к тематическим сдвигам по сравнению с контентно-зависимыми методами.

Для реализации выбранного подхода будет выполнено извлечение комплексного набора лингвистических признаков с применением методов обработки естественного языка, специально адаптированных для работы с особенностями русской морфологии и синтаксиса.

Экспериментальная часть работы включит сравнительное исследование широкого спектра моделей машинного обучения - от классических алгоритмов до современных ансамблевых методов и нейронных сетей. Производительность каждой модели будет оценена как для прогнозирования отдельных атрибутов, так и для одновременного определения полного профиля автора.

1.9 Выводы по главе

В результате аналитического обзора установлено, что базисом для автоматического профилирования автора служат методы NLP, позволяющие преобразовывать неструктурированные русскоязычные тексты в данные для машинного обучения с учётом языковых особенностей.

В данной работе профилирование формализовано как задача классификации пола и возраста, поскольку эти атрибуты являются первичными для иерархической структуры профиля. Ключевым инструментом выступает стилометрический анализ, оперирующий формальными признаками текста.

В ходе обзора проведено сравнение стратегий прогнозирования демографического профиля и рассмотрен спектр моделей машинного обучения. Экспериментальная часть будет направлена на верификацию этих подходов для выявления эффективного инструментария применительно к русскоязычному материалу.

Анализ подтвердил, что задача автоматического профилирования для русского языка сохраняет актуальность. Таким образом, сформирована теоретическая база для разработки системы автоматического профилирования автора на основе методов обработки естественного языка.

ГЛАВА 2. ТЕОРЕТИЧЕСКОЕ ОПИСАНИЕ СИСТЕМЫ

В рамках настоящего исследования рассматривается задача автоматического профилирования автора на основе стилометрического анализа текстов. В первой главе был проведен комплексный обзор существующих подходов к решению данной задачи, включая анализ лингвистических особенностей и методов машинного обучения.

Далее будет представлено детальное описание разрабатываемой системы автоматического профилирования, основанной на стилометрическом подходе. В качестве практического примера реализации рассматривается система для анализа русскоязычных текстов, позволяющая определять две ключевые характеристики автора - пол и возрастную группу. Основу системы составляют формализованные лингвистические признаки, алгоритмы их извлечения и классификации, адаптированные для работы с особенностями русской морфологии и синтаксической структуры.

2.1 Формальное описание методов и процессов автоматического профилирования

Для теоретического описания системы автоматического профилирования автора введём формальную модель, описывающую взаимосвязь данных, признаков и методов анализа. Пусть система определяется кортежем

$$M = \langle D, \Phi_{Pr}, E, \Phi_E, L, \Phi_L, A, \Psi, X, P, R, F \rangle, \quad (1)$$

где описанные ниже множества и отображения задают объекты и процессы предметной области.

$D = \{t_1, t_2, \dots, t_n\}, t_i \in \alpha$ - множество текстовых документов, поступающих на обработку, где α - конечное множество символов, используемое в корпусе;

Функция предобработки $\Phi_{Pr}: D \rightarrow D_{clean}$, где D_{clean} - множество предобработанных текстов. Φ_{Pr} включает замену буквы «ё» на «е», удаление HTML-разметки;

Множество извлекаемых сущностей

$$E = e_{mail}, e_{url}, \dots, e_{tag}, e_{phone}, \quad (2)$$

где каждый элемент $e \in E$ соответствует множеству текстовых фрагментов в очищенном документе, удовлетворяющих определённому шаблону;

Функция извлечения сущностей задаётся как композиция детекторов $\Phi_E(d) = (e(d))_{e \in E}$;

Множество лингвистических аннотаций $L = l_{lemma}, l_{pos}, l_{punct}$ и соответствующее отображение $\Phi_L: D_{clean} \rightarrow A$, где A - пространство аннотированных представлений (последовательностей лемм, последовательностей POS-токенов);

Пространство признаков X - векторное пространство, порождаемое агрегированием Ψ результатов Φ_E и Φ_L ;

Множество целевых профилей $P = p_1, \dots, p_m$, где каждый профиль p_i включает совокупность метки пола $a_{пол}$ и возрастной категории $a_{возраст}$.

Отношение ассоциации признаков и профилей формализуем через отображение $R: (X \times P) \rightarrow \mathbb{R}_{\geq 0}$, где $R(x, p)$ оценивает силу (веса) связи между наборами признаков и профилем;

Множество функций анализа и классификации, определяющих отображение $F = f_\theta: X \rightarrow \Delta(P)$, где $\Delta(P)$ - пространство вероятностных распределений на P , а θ - параметры выбранной модели.

Таким образом, процесс профилирования можно описать как композицию:

$$D \xrightarrow{\Phi_{pr}} D_{clean} \xrightarrow{(\Phi_E, \Phi_L)} A \xrightarrow{\Psi} X \xrightarrow{f_\theta} \Delta(P) \xrightarrow{\text{decision}} \hat{P}. \quad (3)$$

2.2 Описание функций агрегации и формализация признаков

Для преобразования аннотированного текста в векторное пространство признаков определим функцию агрегации Ψ как композицию специализированных функций вычисления метрик.

Функция Ψ формально определяется как отображение:

$$\Psi: A \times E \rightarrow \mathbb{R}^m, \quad (4)$$

где A - пространство аннотированных представлений текста, E - пространство извлеченных сущностей, а $X \subseteq \mathbb{R}^n$ - n -мерное векторное пространство признаков.

В основе функции агрегации лежит набор отдельных метрик, где каждое правило агрегации представляет собой формальное отображение, переводящее структурные и лингвистические характеристики документа в числовую величину. Полный перечень извлекаемых признаков, их формулы и краткие описания сгруппированы по категориям и вынесены в таблицу Б.1 Приложения Б.

Каждый признак формально определяется через математическое выражение, использующее введенные базовые обозначения для элементов текста и их характеристик. Особенностью предложенного подхода является нормализация большинства признаков относительно объема текста, что обеспечивает сопоставимость результатов для документов разной длины.

2.3 Описание исходных данных

В условиях недостаточной изученности темы автоматического профилирования авторов русскоязычных текстов, о чем свидетельствует отсутствие размеченных общедоступных корпусов данных для русского языка [2], выбор обучающей выборки являлся критически важным этапом исследования.

Одним из немногих специализированных ресурсов, удовлетворяющих требованиям задачи, является корпус «BlogsGenderAge», представляющий собой коллекцию блоговых текстов на русском языке с аннотациями пола и возрастных характеристик авторов. Данный корпус включен в базу данных RusIdiolect, созданную Т. А. Литвиновой и коллективом авторов как специализированный ресурс для исследований в области автоматического профилирования автора текста.

Процесс формирования корпуса включал многоэтапную процедуру очистки и валидации данных. На первоначальном этапе были исключены информационные блоки, содержащие ссылки на другие посты и социальные сети, репосты с указанием оригинального источника. Особое внимание было уделено обеспечению качества данных через проведение ручной проверки. Специалисты анализировали соответствие заявленных в профиле характеристик

фактическому содержанию текстов. Дополнительно проводилась проверка на наличие немаркированного неавторского контента, включая новостные материалы и заимствования без указания источника, с использованием автоматизированных систем антиплагиата [31].

В настоящей работе для обеспечения надежности вычислений устанавливались ограничения как на общий объем текста, так и на его структурные характеристики. Диапазон длины документов был определен от 300 до 10 000 символов: нижняя граница обеспечивает минимальный объем языкового материала для извлечения признаков [19], тогда как верхняя позволяет избежать чрезмерного влияния сверхдлинных текстов, которые могут исказить общую картину распределения метрик.

2.4 Описание алгоритма обработки документа

Алгоритм обработки документа можно описать в виде последовательных шагов, общая схема которых представлена на рисунке 1.

Обработка документа

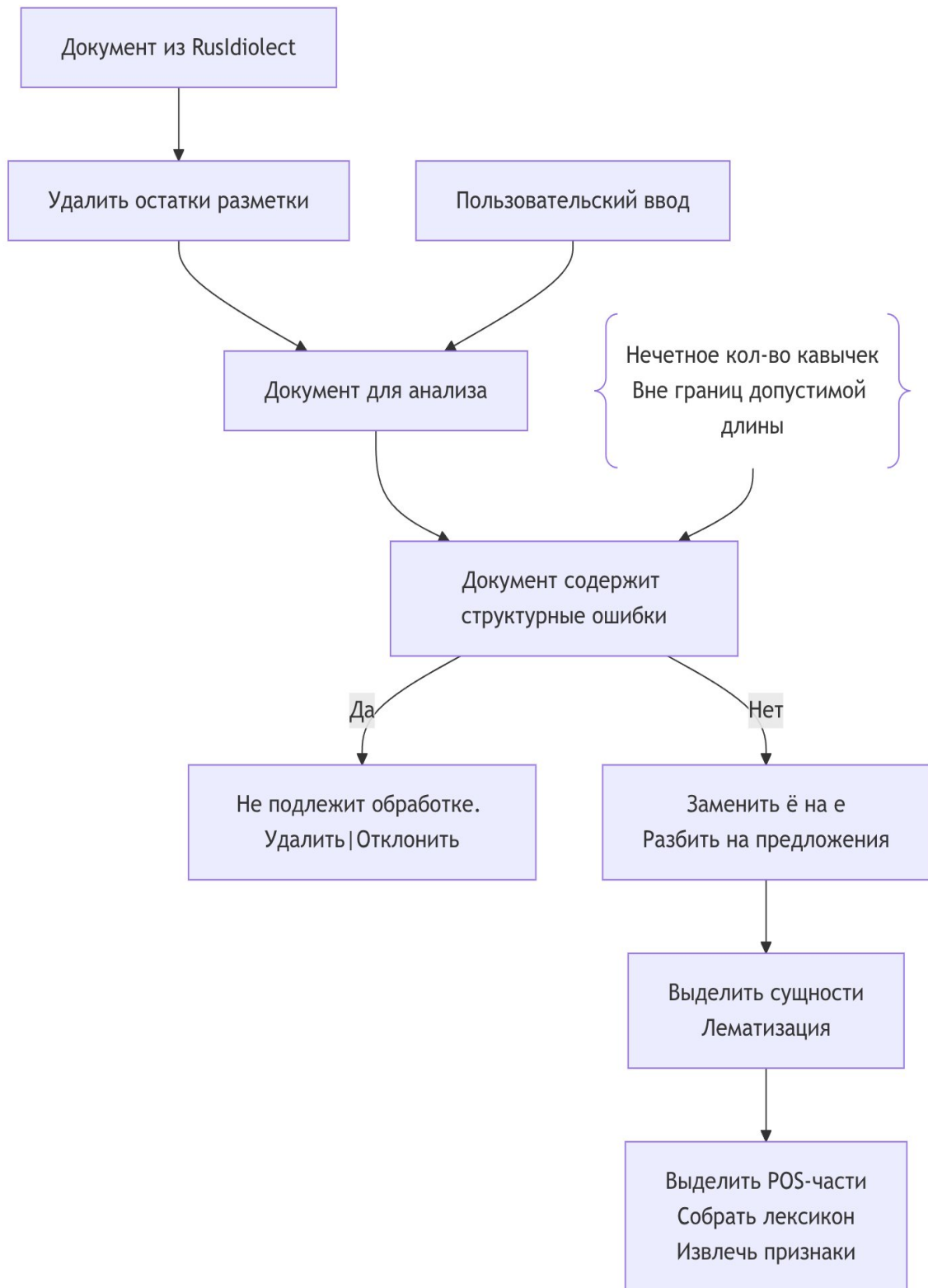


Рисунок 1 - Обработка документа

Шаг 1 - входной материал. На вход подаётся либо документ из корпуса, либо произвольный пользовательский ввод. Далее документ попадает в единую последовательность обработки;

Шаг 2 - базовая очистка от разметки. Первым действием удаляются остатки HTML-разметки;

Шаг 3 - проверка структурной корректности. После очистки система проверяет документ на простые структурные ошибки, которые делают дальнейший анализ бессмысленным или опасным. Критерии отказа могут быть простыми: пустой текст, слишком короткий или длинный текст (меньше или больше порогового количества токенов), нечётное количество кавычек. Если документ попадает под эти критерии, он удаляется из набора или возвращается как отклонённый;

Шаг 4 - нормализация русских символов и разбивка текста на предложения. Для оставшихся документов выполняется замена буквы «ё» на «е» и происходит сегментация текста;

Шаг 5 - выделение сущностей и лемматизация. На данном этапе система выполняет комплексный анализ текста для идентификации и классификации структурированных элементов. Обработка осуществляется с использованием двух взаимодополняющих подходов: правилых методов на основе регулярных выражений и статистической модели распознавания NER. Остальные слова подлежат приведению к словарной форме;

Шаг 6 - POS-аннотация и морфологический разбор. На очищенном тексте выполняется токенизация, затем морфологический разбор и POS-разметка;

Шаг 7 - построение лексикона и подсчёты частот. На основании лемм и токенов составляется локальный лексикон документа: список уникальных словоформ с их частотами;

Шаг 8 - извлечение признаков (метрик). По таблице В.1 система вычисляет требуемые показатели.

2.5 Анализ признакового пространства

2.5.1 Исходные данные и общая статистика

На основе корпуса BlogsGenderAge, описанного в предыдущем разделе, был выполнен процесс извлечения признаков в соответствии с разработанной формальной моделью. В результате каждого документа корпуса был

сформирован вектор из 80 признаков, включающих лингвистические, стилометрические и синтаксические характеристики текста. Общий объем исходной выборки составил 16091 наблюдений.

Для обеспечения качества данных и исключения влияния технических ошибок на результаты анализа была проведена фильтрация извлеченных метрик. На первом этапе удалялись наблюдения с явными признаками некорректной обработки, возникшими на этапе парсинга и токенизации. В частности, были исключены тексты, где средняя длина предложения выходила за пределы диапазона от 3 до 15 словоформ, что свидетельствует либо об ошибках разметки, либо о неинформативности или избыточной сложности материала. Дополнительно отсеивались тексты, содержащие аномально длинные предложения (более 130 символов).

Далее выборка была ограничена по ряду содержательных параметров, чтобы обеспечить релевантность материалов задачами исследования. Исключены тексты со слишком высокой плотностью служебных или нерелевантных элементов: среднее количество чисел на одно предложение не должно было превышать двух, а частота употребления перечислений и единиц измерения - 0,4 на предложение. Также был установлен порог для включения иноязычной лексики: среднее количество иностранных слов не должно было превышать одного на предложение.

В результате применения данных критериев было исключено 4077 наблюдений (25,34% от исходного объема выборки), что позволило сформировать чистую выборку, пригодную для дальнейшего анализа.

2.5.2 Анализ и первичный отбор признаков

В ходе оценки информативности признаков был проведен анализ стабильности их значений. Критерием низкой информативности стала высокая концентрация данных: признаки, у которых более 85% наблюдений попадали в узкий интервал (диапазон, составляющий $\pm 0,1$ от межквартильного размаха), были признаны константными и исключены из дальнейшего рассмотрения.

К данной категории отнесена группа признаков, отражающих преимущественно технические или служебные элементы. Среди них - доля обценной лексики, частота употребления различных знаков пунктуации (кавычки, амперсанд, скобки, точка с запятой, символы сравнения, слеш, звездочка, подчеркивание, тильда), а также ряд размеченных сущностей, таких как URL, теги, адреса электронной почты, номера телефонов и указатели. Низкая частотность и специфический характер этих элементов позволяют отнести их скорее к артефактам форматирования, нежели к значимым стилистическим особенностям текста.

Для другой группы признаков, демонстрирующих умеренный уровень концентрации (от 50% до 85% значений в узком диапазоне), был применен иной подход. С целью сохранения их потенциальной прогностической способности принято решение о преобразовании этих переменных в бинарные индикаторы. Полученный флаг будет сигнализировать о выходе значения за пределы «типичного» диапазона, что позволит зафиксировать наличие определенного сигнала. Границы указанных диапазонов будут определены на этапе работы с обучающей выборкой. В эту группу вошли признаки, связанные с употреблением единиц измерения, перечислений, иностранных слов, смайликов, дат, а также показатели широты использования частей речи и индекс формальности текста.

2.5.3 Преобразование признаков

Анализ распределений частотных признаков выявил специфическую структуру данных, связанную с наличием дефолтного нулевого значения, которое фиксируется при отсутствии соответствующей лингвистической сущности в тексте. В ряде случаев это приводит к формированию ярко выраженного модального пика в точке «0», многократно превосходящего по высоте остальные значения распределения. Подобная характеристика свойственна, например, показателям употребления адресов, вспомогательных глаголов, некоторых пунктуационных знаков и биграмм частей речи. Пример распределения признаков показан на рисунке 2.

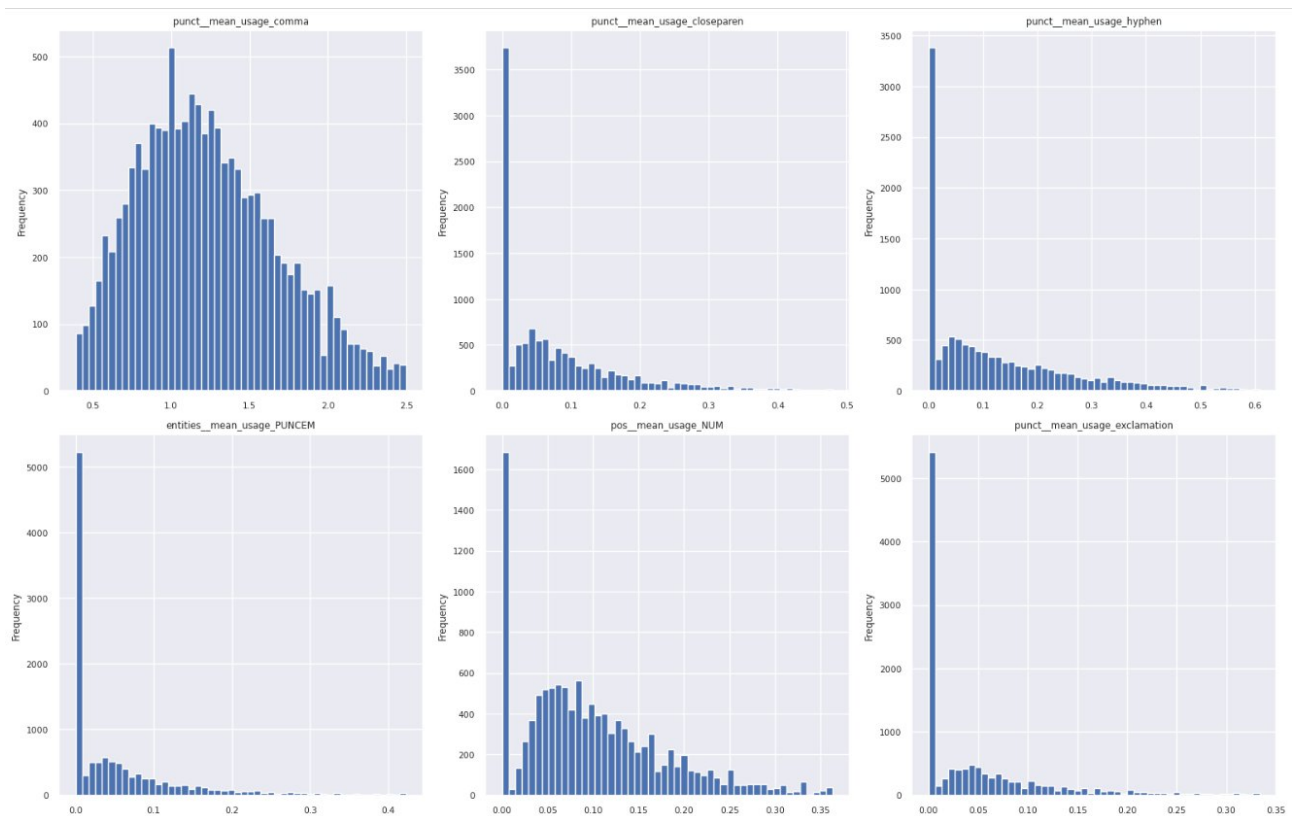


Рисунок 2 - Распределение признаков с выраженным пиком

Для признаков с подобной структурой распределения был применен двухэтапный метод преобразования, направленный на учет специфики нулевых значений и подготовку данных к дальнейшему анализу. Для каждого такого показателя создан бинарный признак-индикатор «used_X», фиксирующий сам факт наличия сущности в тексте по правилу: значение флага принимается равным единице, если исходный показатель строго больше нуля.

Предложенный подход позволяет разделить анализ на две составляющие: бинарный индикатор сигнализирует о наличии или отсутствии явления, в то время как количественные значения исследуются исключительно для тех наблюдений, где флаг истинен.

Остальные признаки сохранены в исходном частотном представлении, так как их значения демонстрируют достаточную вариативность для количественного анализа.

2.5.4 Анализ матрицы корреляции

Проведенный корреляционный анализ признаков позволил выявить ряд устойчивых групп с высокими значениями коэффициентов парной корреляции

($|r| > 0,6$), свидетельствующими о наличии мультиколлинеарности. Исключение дублирующих признаков из таких групп необходимо для упрощения модели без потери прогностической способности. Матрица корреляции с мультиколлинеарными признаками показана на рисунке 3.

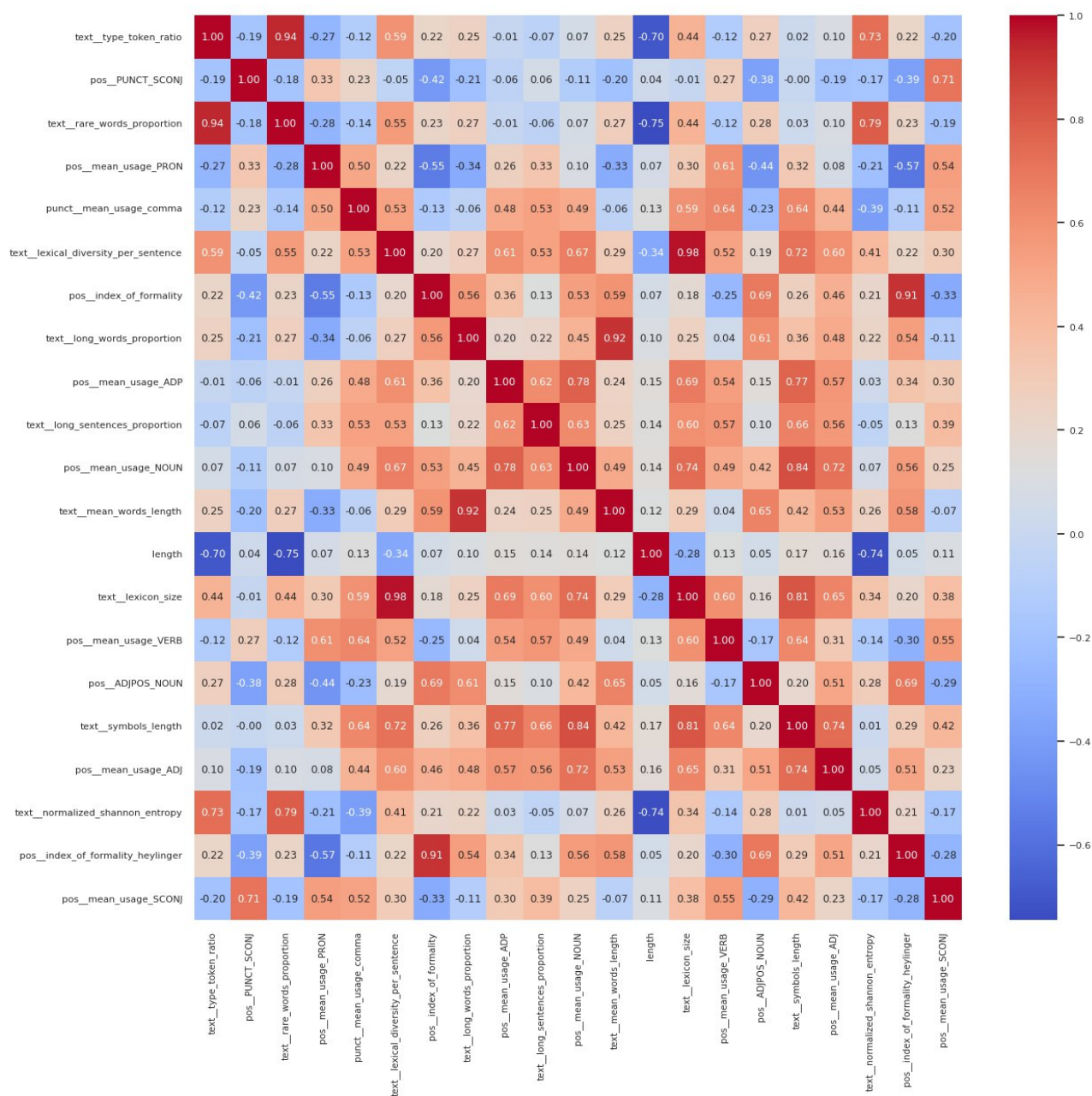


Рисунок 3 - Матрица корреляции

Наиболее тесная связь наблюдается между показателями длины текста: количество символов и доля длинных предложений практически дублируют друг друга. Из этой пары принято решение сохранить длину текста в символах как более прямую и легко интерпретируемую метрику. Близкую к функциональной зависимость демонстрируют размер лексикона и лексическое

разнообразие в расчете на предложение ($r = 0,98$), а также средняя длина слова и доля длинных слов ($r = 0,92$). В обоих случаях выбраны признаки, отражающие суть явления с меньшей избыточностью: лексическое разнообразие на предложение и средняя длина слова соответственно.

Кластер показателей лексического разнообразия, объединяющий долю редких слов, коэффициент тип-токен и нормализованную энтропию Шеннона, характеризуется корреляциями в диапазоне 0,73-0,94. Наиболее репрезентативным признаком из данной группы признан коэффициент TTR, широко используемый в стилометрии. Два индекса формальности текста коррелируют на уровне 0,91; предпочтение отдано индексу Хейлингера, поскольку распределение альтернативного показателя содержит аномальные выбросы, вероятно связанные с ошибками измерения.

Существенные взаимосвязи обнаружены между частотой употребления существительных, прилагательных, предлогов и длиной предложения (коэффициенты корреляции достигают 0,84). Учитывая, что длина предложения уже включена в анализ как самостоятельный признак, а частеречные характеристики частично отражаются в других переменных, принято решение исключить указанные частоты, оставив длину предложения в качестве интегральной характеристики синтаксической сложности. При этом биграммы частей речи, в частности сочетание прилагательного с существительным, несмотря на корреляцию со средней длиной слова и индексами формальности, сохраняются как носители специфической стилистической информации.

Отдельно рассматривалась частота употребления запятой: ее умеренная связь с длиной предложения и глагольными формами не является препятствием для включения в модель, поскольку данный пунктуационный маркер обладает дифференцирующей способностью при решении задач профилирования.

2.5.5 Анализ зависимостей ключевых признаков

В рамках анализа зависимостей ключевых признаков, влияющих на процесс моделирования демографических характеристик автора, были исследованы три группы факторов: длина текста, возраст и пол. Для каждой из

них оценивалась степень связи с текстовыми метриками и её практическая значимость для последующего построения моделей.

Длина текста закономерно изменяет ряд лексических и структурных показателей. На рисунке 4 представлены диаграммы рассеивания для шести признаков, демонстрирующие монотонный характер связи с длиной текста (сглаживающие прямые не имеют резких перегибов).

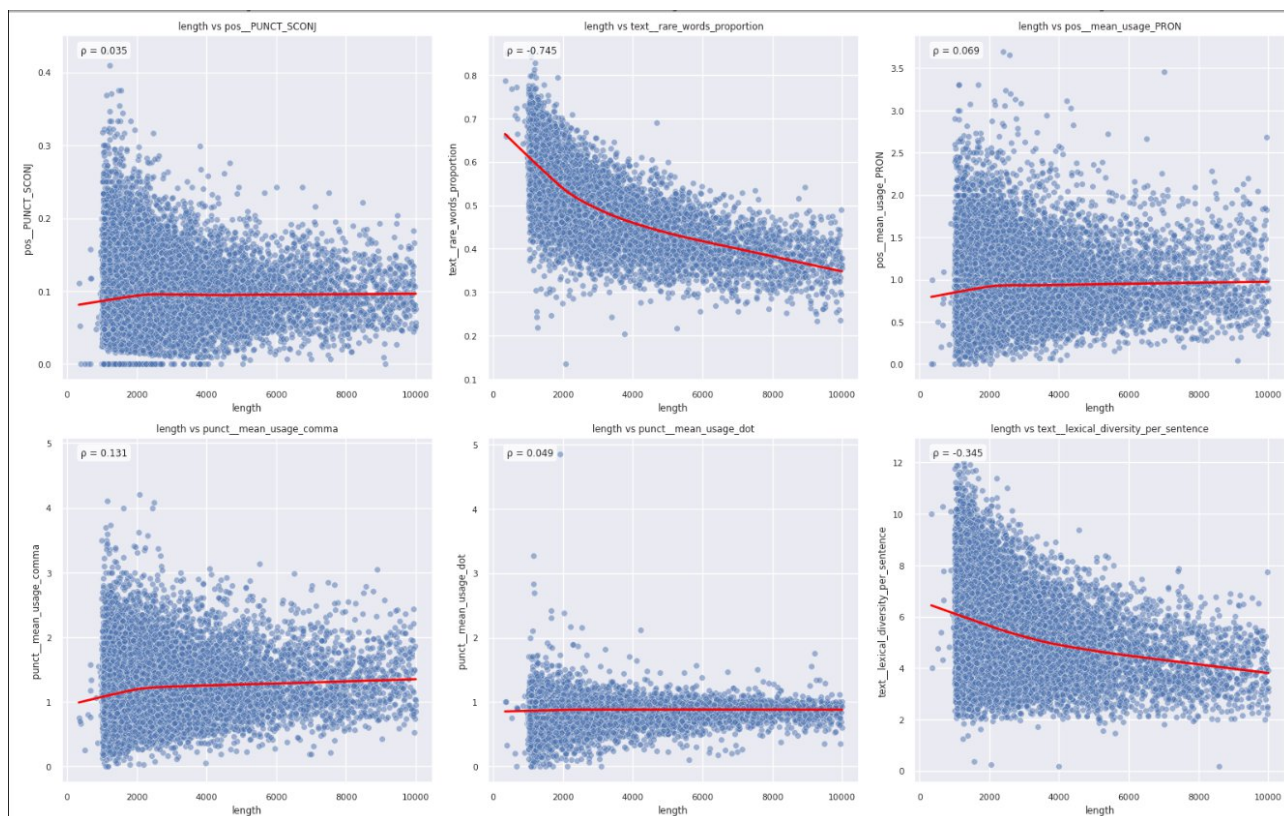


Рисунок 4 - Диаграмма рассеивания признаков с длиной текста

Корреляционный анализ Спирмена подтверждает, что с ростом длины текста снижается доля редких слов и нормализованная энтропия, одновременно расширяется набор используемых существностей и знаков препинания. При этом иные характеристики (частеречные признаки, биграммы) слабо или вовсе не связаны с длиной. Корреляция длины с полом и возрастом автора близка к нулю, поэтому включать её как отдельный предиктор в демографические модели нет необходимости.

Связь возраста автора с текстовыми характеристиками оказалась крайне слабой. На рисунке 5 представлены диаграммы рассеивания для шести признаков, демонстрирующие монотонный характер связи с возрастом автора.

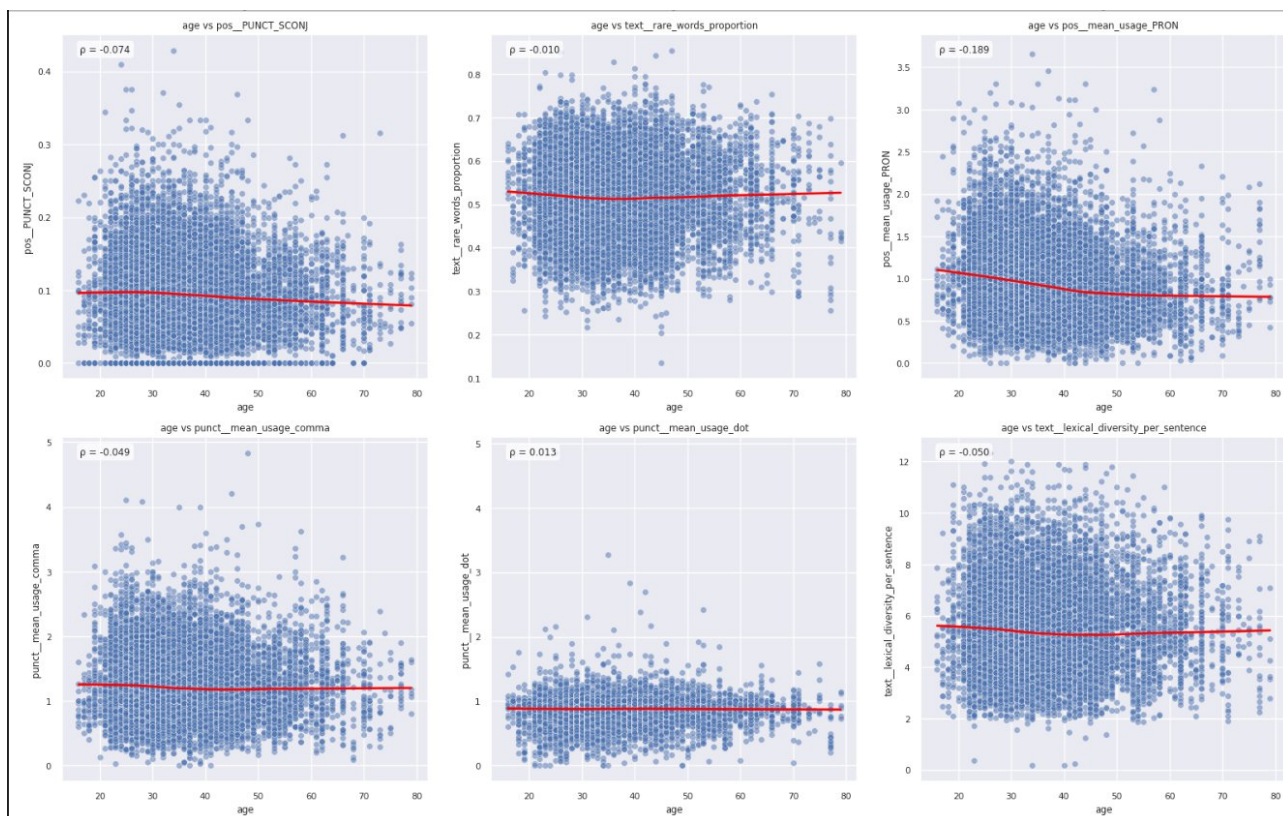


Рисунок 5 - Диаграмма рассеивания признаков с возрастом автора

Корреляционный анализ Спирмена показал крайне слабые линейные связи возраста с отдельными текстовыми характеристиками. Наиболее заметные положительные корреляции наблюдаются для индекса формальности Хейлингера (0,19), доли редких слов (0,16), средней длины слова (0,12) и доли имён собственных (0,13); отрицательные - для доли местоимений (-0,19), наречий (-0,15) и глаголов (-0,12). Длина текста и большинство пунктуационных признаков с возрастом практически не связаны. Таким образом, ни один отдельный признак не обладает достаточной предсказательной силой для точного определения возраста в годах. Однако совокупность множества слабых сигналов может дать некоторый прогностический потенциал при переходе к задаче классификации на возрастные категории.

Для признаков, связанных с полом автора, характерна иная картина. Сравнение мужской и женской выборок (примерно 6500 и 5500 наблюдений) с помощью U-критерия Манна-Уитни показало, что статистически значимые различия присутствуют по большей половине метрик ($p < 0,05$). Результаты сравнения показаны в таблице 1.

Таблица 1 - Результаты теста Манна-Уитни в женской и мужской выборке

Признак	P-value	Разница медиан
pos_pair_of_adv_per_sentence	4.966e-08	0.008071
pos_mean_usage_NUM	0.0650	0.000000
punct_mean_usage_question	0,0506	0.000000
punct_mean_usage_closeparen	7.063e-18	0.016667
punct_mean_usage_openparen	0.9471	0.000000
entities_mean_usage_ADDRESS	3.1193e-21	0.012760
punct_mean_usage_exclamation	2.4506e-28	0.014423
pos_PUNCT_PRON	2.6734e-39	0.011637
pos_mean_usage_AUX	0.7812	0.000000
punct_mean_usage_hyphen	0.5773	0.002232
entities_mean_usage_QUOTE	7.7243e-10	0.014337
entities_mean_usage_NUMBER	1.2800e-16	0.023810
pos_PROPN_PUNCT	5.3562e-45	0.018145
punct_mean_usage_colon	0.1275	0.000733
entities_mean_usage_PUNCCEM	7.5161e-14	0.010256

Величина различий невелика - доли процента по медианам. Распределения визуально почти идентичны, ящики с усами сильно перекрываются, что говорит о низкой дискриминативной способности отдельных признаков (рисунок 6).

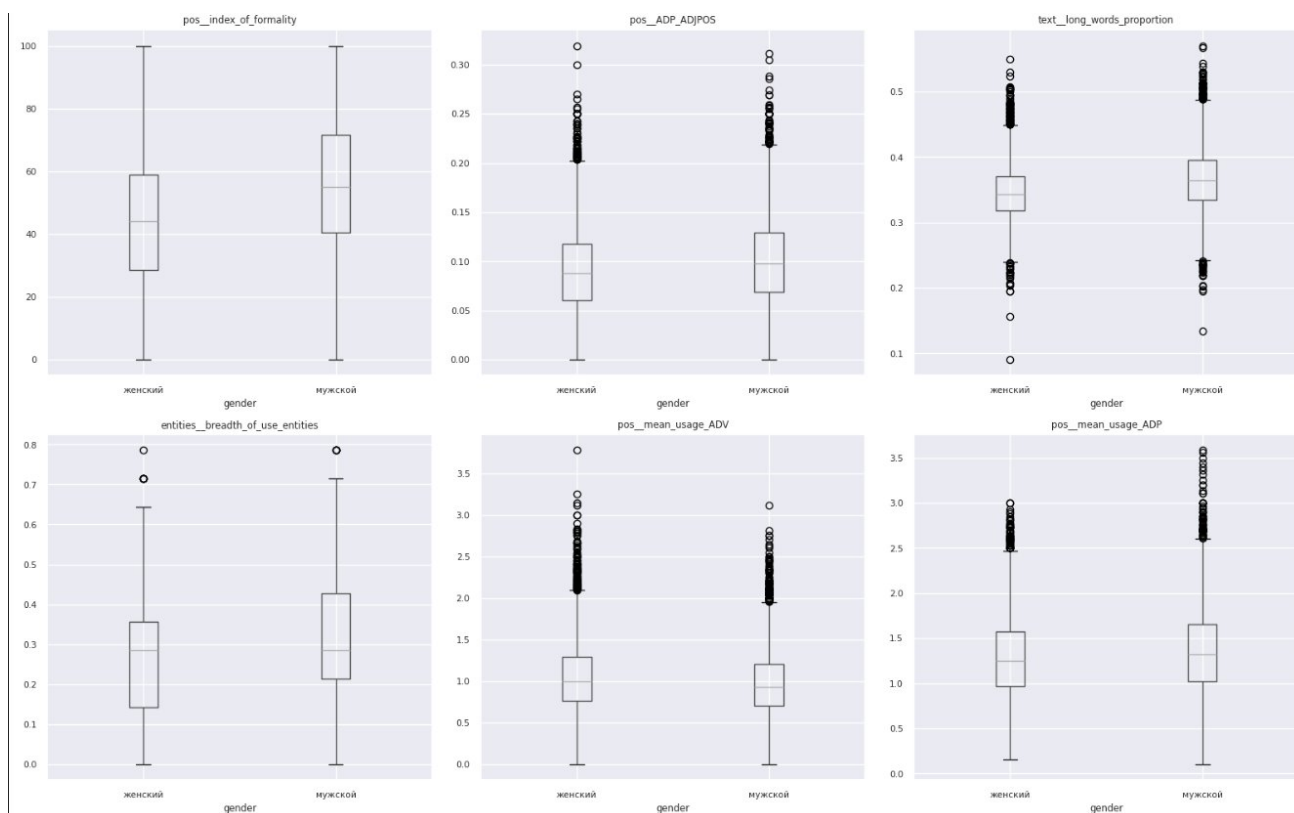


Рисунок 6 - Диаграмма размаха признаков на женской и мужской выборке

Обобщая полученные результаты, можно заключить, что исследованные текстовые метрики содержат статистически значимый, но крайне слабый сигнал о демографических характеристиках автора. Большие объёмы данных обеспечивают высокую мощность тестов, однако практическая значимость выявленных различий невелика. Полученные результаты свидетельствуют об отсутствии необходимости в стратификации данных по длине текста: модели могут обучаться на объединенной выборке без потери качества, поскольку влияние длины уже опосредованно отражено в других признаках..

2.5.6 Подготовка целевых переменных

Для решения задачи классификации целевые переменные были приведены к формату, пригодному для бинарного прогнозирования. Возрастная характеристика подвергнута бинаризации: в качестве порога возрастной группы выбрано значение 35 лет, что является типичной границей, разделяющей молодых и взрослых авторов в социолингвистических исследованиях, а также обеспечивает достаточную сбалансированность групп. Соответственно, тексты авторов в возрасте до 35 лет включительно отнесены к категории «молодой», старше 35 лет - к категории «взрослый». Признак пола преобразован в бинарный формат прямым кодированием: мужскому полу присвоено значение 1, женскому - 0.

2.6 Обучение и оптимизация моделей классификации

2.6.1 Используемые модели классификации

Для реализации решения задачи классификации был выбран набор моделей, охватывающих различные семейства алгоритмов. Исследование множества разнотипных моделей позволяет провести сравнительный анализ и выявить подход, наиболее адекватный специфике решаемой задачи. Ниже приведен список исследуемых моделей:

1. Логистическая регрессия (Logistic Regression, LR);
2. Метод опорных векторов (Support Vector Machine, SVM) с различными ядрами;

3. Наивный байесовский классификатор (Naive Bayes, GaussianNB);
4. Метод k-ближайших соседей (K-Nearest Neighbors, KNN);
5. Случайный лес (Random Forest, RF);
6. Градиентный бустинг (Gradient Boosting Machine, GBM);
7. Многослойный перцептрон (Multilayer Perceptron, MLP).

2.6.2 Методология обучения и валидации

План обучения каждой модели включает создание комплексного пайплайна, состоящего из препроцессора и классификатора. Препроцессор объединяет все этапы преобразований, необходимые для подготовки признакового пространства.

Обозначим исходную матрицу признаков как:

$$X = [X_{num} \quad X_{int} \quad X_{int_used} \quad X_{bin}] \quad (9)$$

где X_{num} - множество непрерывных признаков, X_{int} - множество признаков интенсивности использования фактора с пиковым значением в нуле, X_{int_used} - множество флаговых признаков соответствующих признаков интенсивности, X_{bin} - множество флаговых признаков

Препроцессор определяется как композиция преобразований:

$$\begin{aligned} &preprocessor(X) \\ &= [T_{num}(X_{num}) \mid T_{int}(X_{int}) \mid T_{int_used}(X_{int_used}) \mid T_{bin}(X_{bin})] \end{aligned} \quad (10)$$

где T - соответствующие колоночные трансформеры, включающие стандартизацию числовых признаков и порядковых признаков

2.6.3 Пространства гиперпараметров моделей

В таблице 2 указаны диапазоны значений гиперпараметров исследуемых моделей.

Таблица 2 - Диапазоны гиперпараметров для оптимизации моделей

Модель	Гиперпараметр	Диапазон значений	Описание
LR	«C»	[0.001, 0.01, 0.1, 0.5, 1, 5, 10, 25, 50, 100]	Сила регуляризации
	«penalty»	[‘l1’, ‘l2’]	Тип регуляризации
	«solver»	[‘liblinear’]	Алгоритм

Модель	Гиперпараметр	Диапазон значений	Описание
			оптимизации
SVM	«C»	[0.001, 0.01, 0.1, 0.5, 1, 5, 10, 25, 50, 100]	Сила регуляризации
	«kernel»	[‘linear’, ‘rbf’, ‘poly’]	Тип ядра
	«gamma»	[0.001, 0.01, 0.015, 0.1, 0.2, 0.5, 1]	Коэффициент ядра
	«degree»	[2, 3]	Степень ядра
RF	«n_estimators»	[25, 50, 100, 150, 200]	Количество деревьев
	«max_depth»	[5, 6, 7, 8, 9]	Максимальная глубина
	«max_features»	[5, 6, 7, 8, 9]	Количество признаков
	«criterion»	[‘gini’, ‘entropy’]	Критерий разделения
GBM	«n_estimators»	[25, 50, 100, 150, 200]	Количество деревьев
	«max_depth»	[5, 6, 7, 8, 9]	Максимальная глубина
	«max_features»	[5, 6, 7, 8, 9]	Количество признаков
	«criterion»	[‘friedman_mse’, ‘squared_error’]	Критерий оптимизации
NB	«var_smoothing»	[1e-8 - 1]	Сглаживание дисперсии
KNN	«n_neighbors»	[63, 87, 101, 127, 143, 169, 201, 215]	Количество соседей
	«weights»	[‘uniform’, ‘distance’, triangular_weights, gaussian_weights, quadratic_weights]	Весовая функция
	«metric»	[‘euclidean’, ‘manhattan’, ‘minkowski’]	Метрика расстояния
MLP	«hidden_layer_sizes»	[198, 128, 96, 64, 48, 32, 16, (96, 64), (64, 32), (32, 16), (16, 8)]	Нейронов в слое
	«activation»	[‘relu’]	Функция активации
	«learning_rate_init»	[1e-6, 5e-6, 1e-5, 2e-5 5e-5, 1e-4]	Скорость обучения
	«solvers»	[Adam]	Оптимизатор

2.6.4 Метрики оценки качества

Для комплексной оценки моделей использовались следующие метрики:

Accuracy - общая доля правильных предсказаний:

$$Accuracy = \frac{1}{N} \sum_{i=1}^N 1(\hat{y}_i = y_i) \quad (11)$$

F1-score - гармоническое среднее precision и recall:

$$F1_j = 2 \cdot \frac{Precision_j \cdot Recall_j}{Precision_j + Recall_j} \quad (12)$$

2.7 Выводы по главе

В данной главе было представлено теоретическое описание системы автоматического профилирования автора текста. Формализована задача классификации, где целевые переменные определяются как демографические характеристики автора - пол и возрастная группа, а вектор признаков формируется из лингвистических, статистических и синтаксических характеристик текста.

В рамках дальнейшего исследования предполагается разработать и реализовать комплексный процесс предобработки текстовых данных, включающий этапы очистки текста, извлечения сущностей, лингвистической разметки и формирования признакового пространства. На основе анализа корреляций и информационной значимости планируется сформировать оптимальный набор признаков, демонстрирующих устойчивую связь с демографическими характеристиками автора.

Для каждой из семи классов моделей машинного обучения будет выполнен подбор гиперпараметров, что позволит определить оптимальные конфигурации для различных сценариев классификации: определения пола, возраста в гендерных подвыборках и многоклассовой классификации.

Ожидаемые результаты позволят подтвердить практическую применимость предлагаемого подхода для решения бизнес-задач цифровых платформ, таких как таргетирование рекламы и персонализация контента.

ГЛАВА 3. ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ СИСТЕМЫ

В этой главе приводится архитектурный дизайн программной системы, реализующей описанные в предыдущих главах методы автоматического профилирования автора текста, а также контрольный пример и методы подбора гиперпараметров. Каждый раздел содержит ключевые диаграммы на языке UML (Unified Modeling Language).

3.1 Концептуальное проектирование системы

Концептуальное проектирование выполнено в нотации IDEF0 и отражает переход от текущей неэффективной системы таргетирования к целевой системе с автоматическим профилированием авторов.

Контекстная диаграмма «Как есть», представленная на рисунке 7, иллюстрирует общий процесс таргетирования рекламных показов в существующей системе. Основным входным сигналом выступают данные профиля пользователя, которые зачастую характеризуются неполнотой, содержат неточности или устаревшую информацию. Данная архитектура демонстрирует фундаментальную зависимость эффективности всего процесса от качества исходных демографических данных, что создает системную уязвимость.

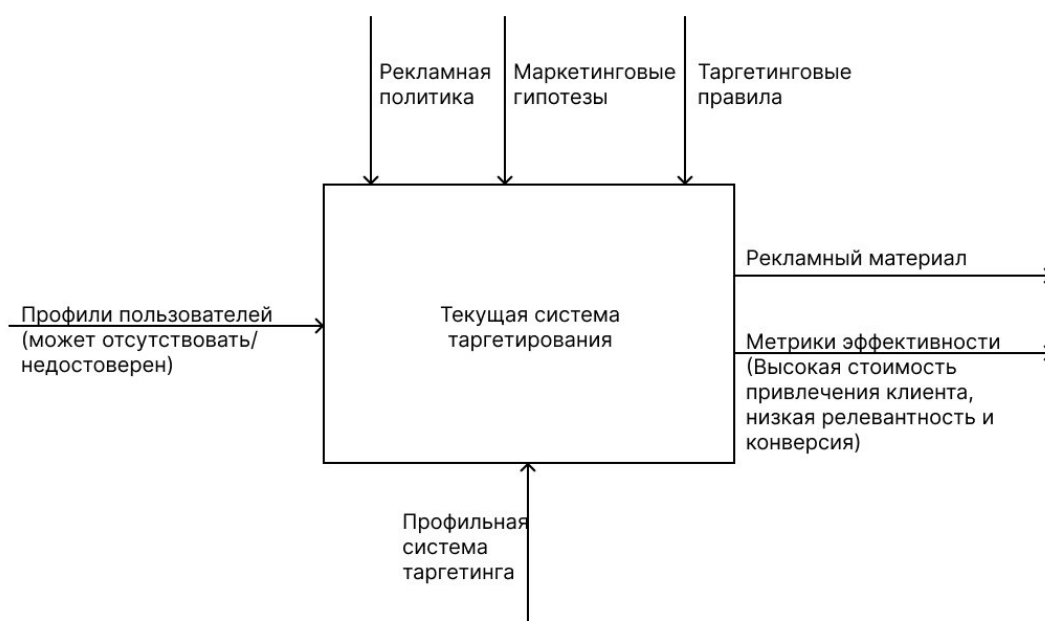


Рисунок 7 - Диаграмма «как есть»

Декомпозиция данной диаграммы, отображенная на рисунке 8, детализирует последовательность этапов: получение пользовательских данных, их обработка в системе таргетинга и финальный показ рекламных материалов. Ключевые контрольные факторы, такие как рекламная политика, маркетинговые гипотезы и бизнес-правила, выступают в роли внешних ограничений, в то время как основным механизмом остается классическая профильная логика таргетинга. На практике такой подход приводит к очевидным проблемам: большое число пользователей без корректных метаданных, низкая релевантность показов, низкая конверсия и, как следствие, высокий показатель САС.

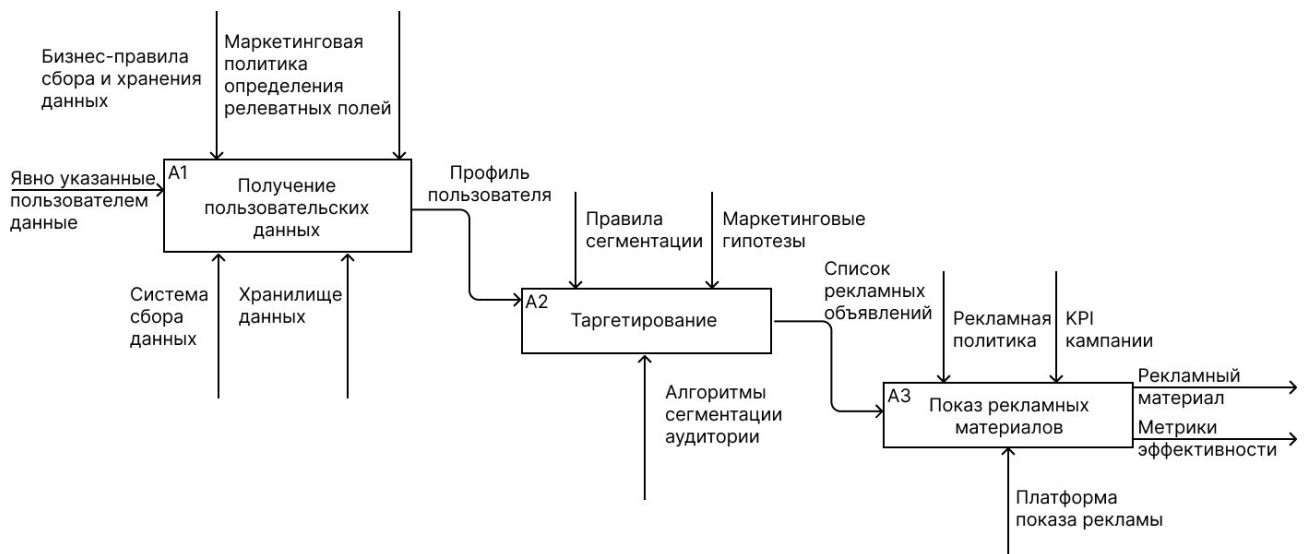


Рисунок 8 - Декомпозиция диаграммы «Как есть»

Диаграмма подчёркивает узкое место - зависимость от качества профилей и отсутствие автоматического механизма восстановления или уточнения демографической информации

В варианте «Как должно быть» предлагается интеграция модуля автоматического профилирования на основе пользовательского контента (посты, отзывы, комментарии) в существующую систему таргетирования.



Рисунок 9 - Диаграмма «Как должно быть»

Диаграмма «Как должно быть», изображенная на рисунке 9, показывает, что на вход принимаются как доступные профили, так и текстовый контент; предиктивная модель генерирует недостающие или уточнённые признаки (вероятности пола/возраста), которые передаются в систему таргетинга вместе с уже имеющимися метаданными. Контрольные механизмы остаются прежними, но появляется дополнительный механизм - модель профилирования.

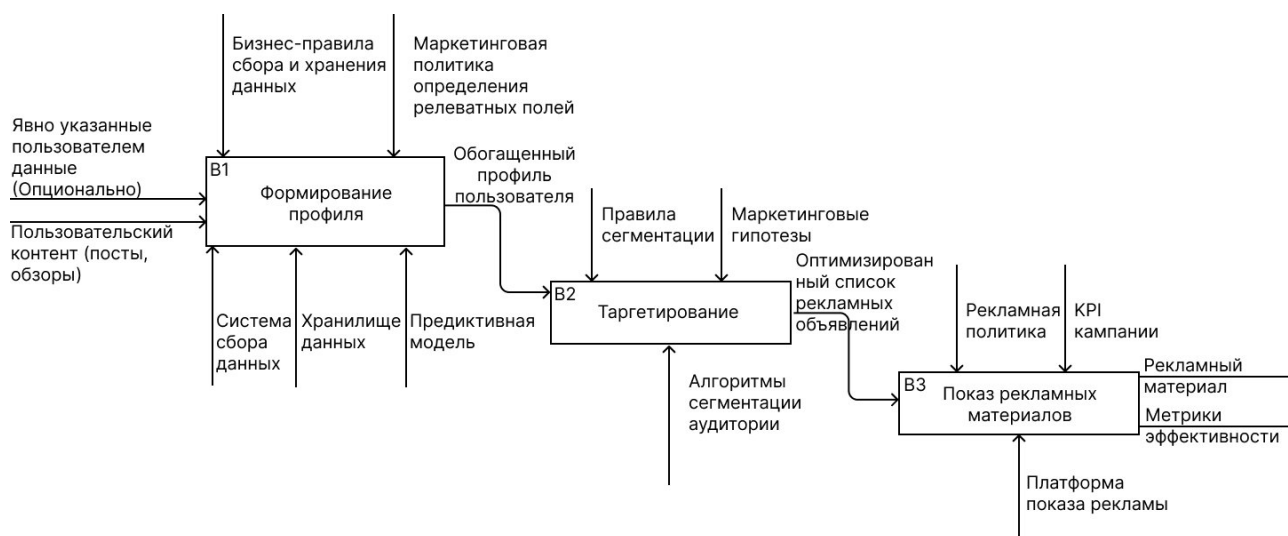


Рисунок 10 - Декомпозиция диаграммы «Как должно быть»

Декомпозиция целевого процесса, представленная на рисунке 10, детализирует трехэтапную архитектуру усовершенствованной системы. На этапе

получения и обогащения данных производится комплексная обработка как явных профильных данных, так и пользовательского контента с применением модуля автоматического профилирования. Обогащенный профиль, переданный на этап таргетирования позволяет получить более оптимизированный список рекламных объявлений. Финальный этап показа рекламы обеспечивает доставку оптимизированного контента с последующим анализом улучшенных метрик эффективности. Ожидаемое поведение системы при таком подходе: уменьшение доли «пустых» или «ошибочных» профилей в таргетах, рост релевантности показов, повышение CTR(Click-Through Rate - показатель кликабельности, отношение числа кликов к числу показов), конверсии и, как следствие, снижение САС

3.2 Диаграмма вариантов использования

Диаграмма вариантов использования иллюстрирует внешние роли (актеров) и ключевые сценарии взаимодействия с системой. Она продемонстрирована на рисунке 11.



Рисунок 11 - Диаграмма вариантов использования

На диаграмме заданы два основных актёра: «Внешняя система» (например, рекламная платформа или микросервис, обращающийся к API) и «Веб-пользователь» (пользователь, работающий через веб-интерфейс). Система автоматического профилирования предоставляет пять ключевых вариантов использования:

- Отправить текст на профилирование. Внешний сервис или веб-пользователь посылает текст в систему для получения предсказанного профиля. Система создает задачу и возвращает ее идентификатор;
- Получить результат по ID задачи. Клиент использует идентификатор задачи для получения результатов профилирования. Если обработка завершена, возвращается профиль с вероятностями, порогами и метками классов; если нет - статус обработки;
- Выбрать модель классификации. Этот вариант использования расширяет первый: пользователь или внешняя система может явно указать, какую модель применять для профилирования;
- Извлечь метрики из текста. Вариант предназначен для случаев, когда требуется получить только лингвистические характеристики текста без построения демографического профиля;
- Просмотреть информацию о моделях. Через интерфейс можно ознакомиться с метаданными доступных моделей, что помогает принимать решение при выборе и отслеживать их качество.

3.3 Диаграмма классов системы

3.3.1 Извлечения признаков

На рисунке 12 представлена диаграмма классов, описывающая структуру модуля извлечения лингвистических признаков из текста. Данный модуль является фундаментальным компонентом системы профилирования и отвечает за последовательное преобразование исходного текста в вектор признаков, используемый для дальнейшего анализа.

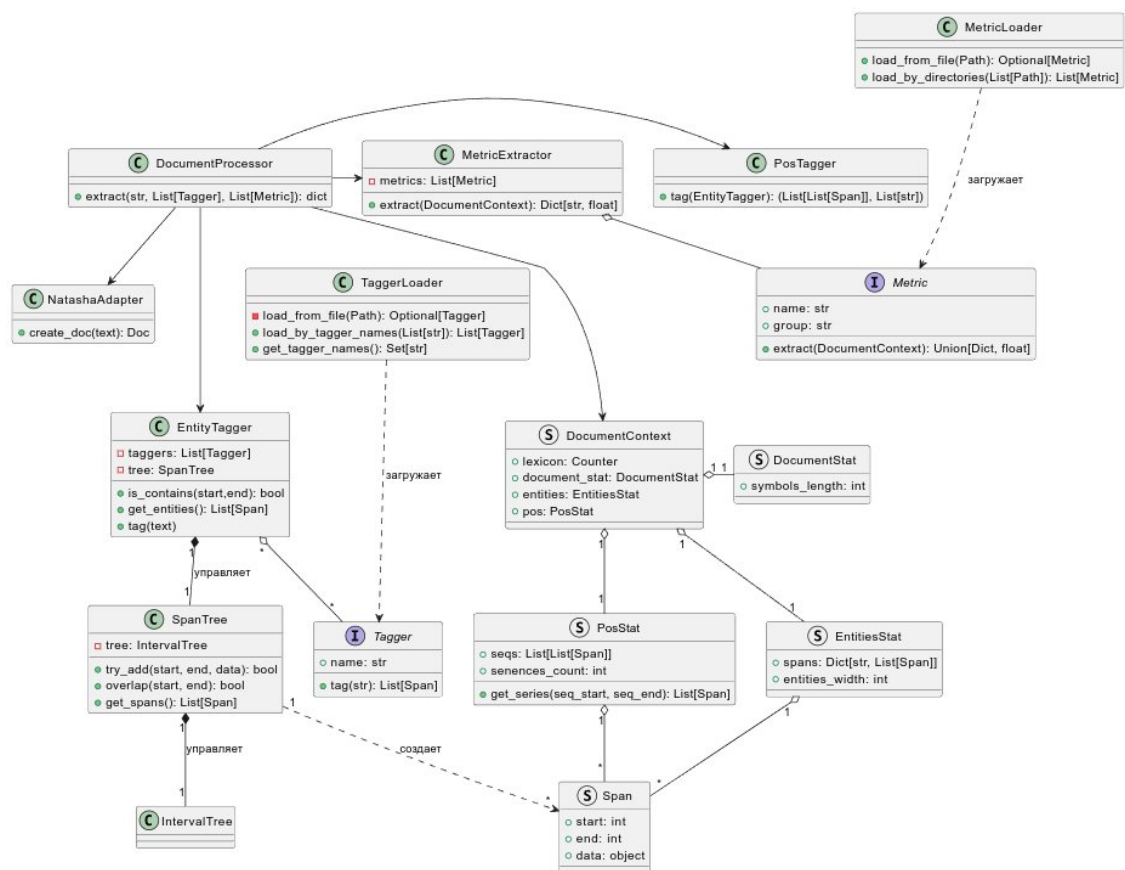


Рисунок 12 - Диаграмма классов извлечения признаков

Диаграмма классов описывает внутреннюю архитектуру модуля извлечения лингвистических признаков, центральным элементом которой выступает класс «DocumentProcessor». Он координирует полный цикл обработки текста: используя «NatashaAdapter» для лингвистического анализа, «EntityTagger» для выделения именованных сущностей и «PosTagger» для частеречной разметки, процессор создает контейнер «DocumentContext», агрегирующий результаты анализа. Для эффективного управления пересекающимися фрагментами текста применяется «SpanTree», который обеспечивает хранение и проверку наложений спанов, извлекаемых различными теггерами.

Дальнейшее вычисление признаков выполняется классом «MetricExtractor», который применяет набор метрик, реализующих единый интерфейс «Metric», к сформированному контексту документа. Модуль спроектирован с учетом расширяемости: загрузка доступных теггеров и метрик производится через классы «TaggerLoader» и «MetricLoader», что позволяет

легко добавлять новые лингвистические анализаторы и вычисляемые характеристики без изменения ядра системы. Такая организация обеспечивает модульность, переиспользуемость компонентов и гибкость при настройке признакового пространства под конкретные задачи профилирования.

3.3.2 Обработка запросов

Эта диаграмма отображает архитектуру API-слоя системы, показывая, как клиентские запросы преобразуются в демографические профили через последовательность валидации, извлечения признаков и классификации. Диаграмма изображена на рисунке 13.

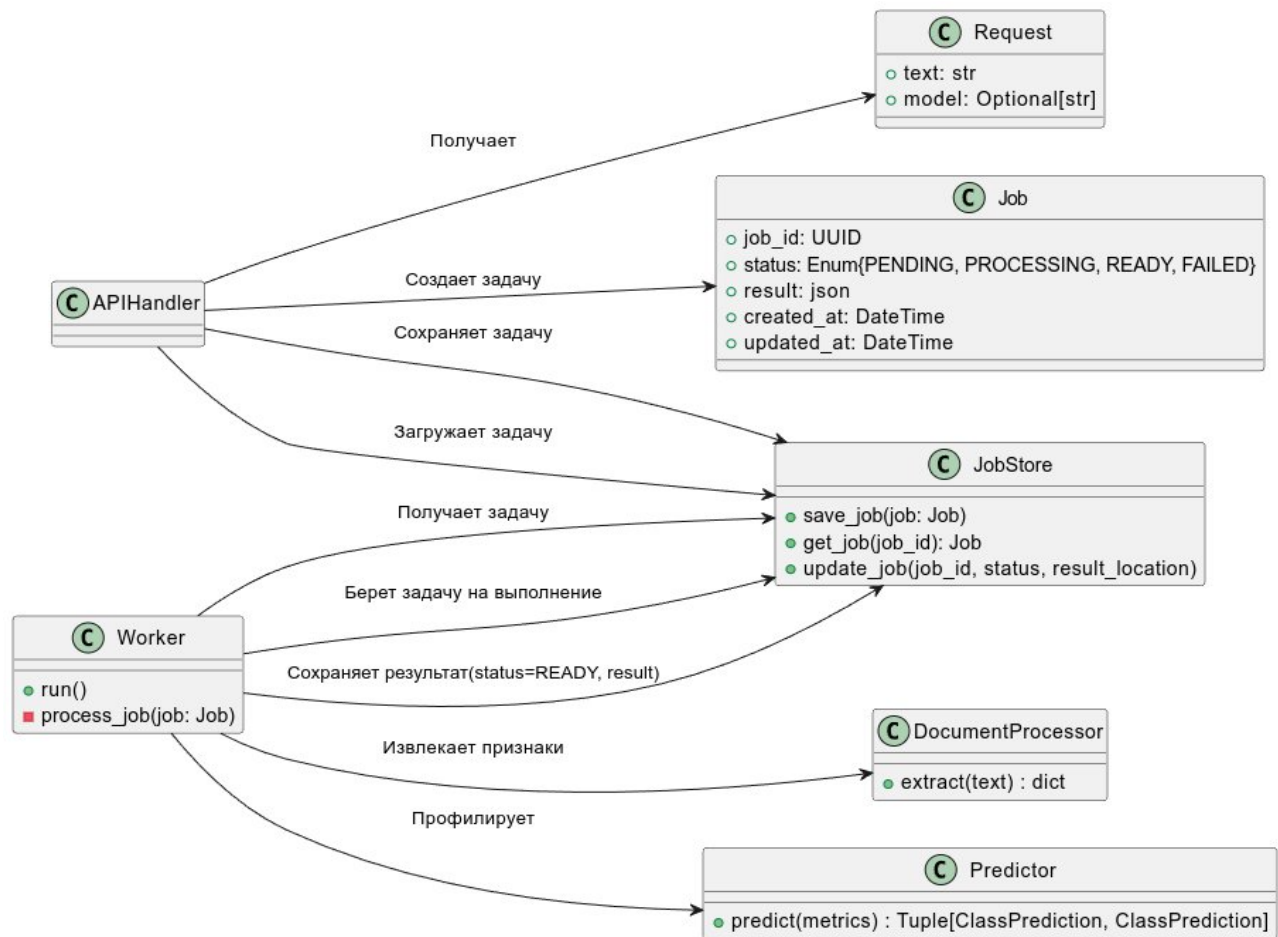


Рисунок 13 - Диаграмма классов обработки запросов

Архитектура REST API системы автоматического профилирования построена вокруг асинхронной модели обработки запросов, что обеспечивает высокую производительность и отказоустойчивость при работе с большими объемами текстовых данных.

Центральным элементом модуля является класс «APIHandler», который обрабатывает входящие HTTP-запросы и координирует весь процесс профилирования. При получении запроса создается объект класса «Request», который инкапсулирует входные данные и выполняет их валидацию, включая проверку корректности текстовых кавычек и соответствие длины текста установленным ограничениям.

Для обеспечения асинхронной обработки система использует механизм задач. «APIHandler» создает задачу «Job» с уникальным идентификатором и сохраняет ее в базу данных при помощи «JobStore». Это позволяет системе обрабатывать множественные запросы одновременно и обеспечивает устойчивость к сбоям.

Рабочий процесс «Worker» асинхронно извлекает задачи из «JobStore» и выполняет основную обработку. На этапе извлечения признаков «DocumentProcessor» преобразует сырой текст в структурированный словарь признаков. Перед передачей в модели классификации, извлеченные признаки проходят проверку на корректность и пригодность для прогнозирования.

Абстрактный класс Predictor определяет единый интерфейс для различных ML-моделей, что обеспечивает легкую замену и комбинирование алгоритмов классификации. После выполнения прогнозирования результаты вместе с оценками достоверности сохраняются в «JobStore», а «APIHandler» формирует финальный ответ, включающий как демографический профиль, так и извлеченные стилометрические признаки для последующего анализа.

3.4 Диаграмма последовательности

Диаграмма последовательности, изображенная на рисунке 14, описывает взаимодействие между компонентами системы при обработке запроса на профилирование текста. Процесс обработки запроса на автоматическое профилирование автора текста представляет собой последовательность взаимосвязанных этапов, обеспечивающих надежную и эффективную работу системы.

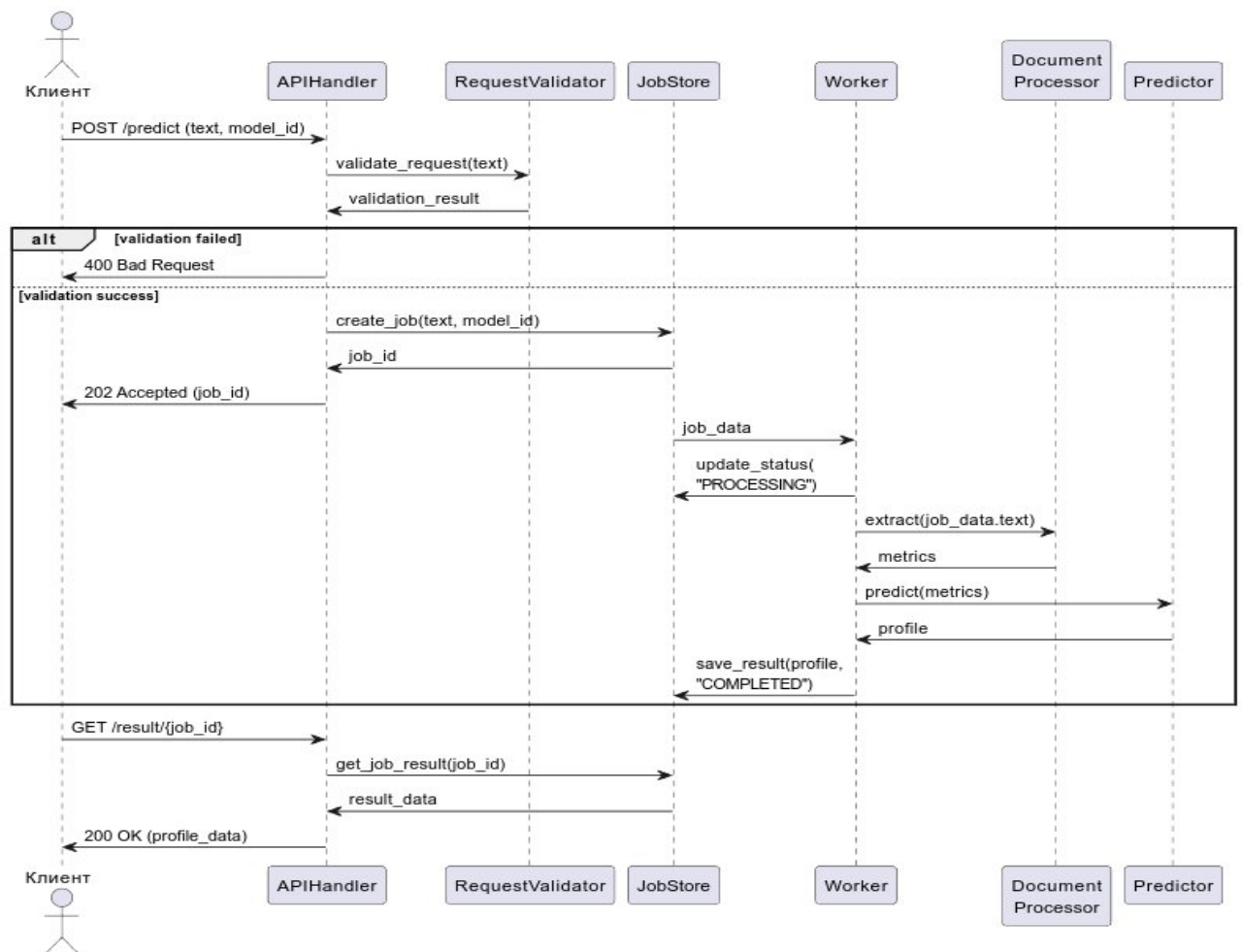


Рисунок 14 - Диаграмма последовательности обработки запроса

Этап 1: Инициализация запроса. Процесс начинается с того, что клиент (внешняя система или веб-пользователь) отправляет HTTP POST запрос с текстом для анализа и опциональным параметром «model_id», указывающий на предпочтительную модель профилирования.

Этап 2: Валидация входных данных. Получив запрос, «APIHandler» делегирует проверку корректности данных модулю «RequestValidator».

Этап 3: Обработка результатов валидации. При обнаружении ошибок валидации система немедленно возвращает клиенту HTTP статус 400 (Bad Request) с детальным описанием проблемы.

Этап 4: Создание асинхронной задачи. При успешной валидации «APIHandler» создает новую задачу в «JobStore». Задаче присваивается уникальный идентификатор, который возвращается клиенту со статусом 202 (Accepted).

Этап 5: Запуск фоновой обработки. Созданная задача автоматически становится доступной для обработки воркерами («Worker»). Воркер, получив задачу из «JobStore», немедленно обновляет ее статус на «PROCESSING», что предотвращает повторную обработку этой же задачи другими воркерами.

Этап 6: Извлечение лингвистических признаков. Ключевой этап обработки начинается с преобразования исходного текста в вектор признаков при помощи «DocumentProcessor». Результатом является структурированный вектор признаков, готовый для передачи в модель прогнозирования.

Этап 7: Классификация и прогнозирование. Вектор признаков передается в модуль «Predictor», который загружает соответствующую ML-модель и выполняет прогнозирование демографических характеристик.

Этап 8: Сохранение результатов. Полученные результаты прогнозирования сохраняются в «JobStore» вместе с обновлением статуса задачи на «COMPLETED».

Этап 9: Получение результатов клиентом. Клиент может в любой момент запросить результаты обработки, отправив GET запрос. Система извлекает данные из «JobStore» и возвращает профиль.

3.5 Диаграмма компонентов

Диаграмма компонентов системы иллюстрирует структурную организацию программных модулей и их взаимодействие. Архитектура системы построена по модульному принципу, что обеспечивает четкое разделение ответственности между компонентами и возможность их независимого развития.

Основу системы составляют четыре ключевых модуля, каждый из которых выполняет специализированные функции. Модуль «Web API» обеспечивает интерфейс взаимодействия с внешними системами, где «FastAPI» обрабатывает «HTTP»-запросы получения текстов и возврата результатов, а «APIHandler» координирует обработку запросов и управление жизненным циклом задач.

Модуль «Text Processing» отвечает за лингвистическую обработку текстов, где «Document Processor» выполняет комплексный анализ текста и извлечение

признаков, а «Natasha» предоставляет функционал для морфологического и синтаксического анализа русскоязычных текстов.

Модуль «Worker» реализует вычислительные процессы классификации, включая «JobWorker» для асинхронной обработки задач из очереди и «Predictor» для выполнения прогнозирования демографических характеристик с использованием ML-моделей.

Модуль «Data Storage» обеспечивает сохранение и управление данными через «PostgreSQL», который хранит метаданные задач, результаты профилирования и служебную информацию.

Такая архитектура обеспечивает высокую масштабируемость системы и позволяет эффективно распределять вычислительную нагрузку между компонентами. Диаграмма компонентов показана на рисунке 15.

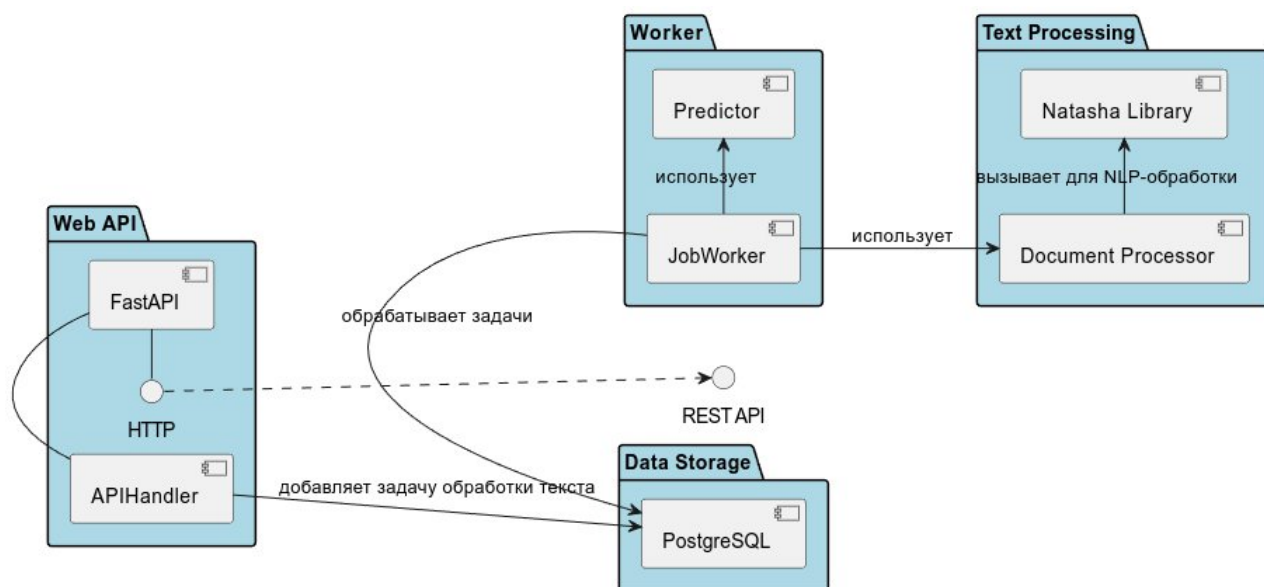


Рисунок 15 - Диаграмма компонентов

3.6 Описание архитектуры развертывания системы

Архитектура развертывания системы автоматического профилирования реализована по многоуровневому принципу, обеспечивающему масштабируемость, отказоустойчивость и эффективное распределение вычислительной нагрузки. Структура развертывания компонентов системы представлена на рисунке 16.

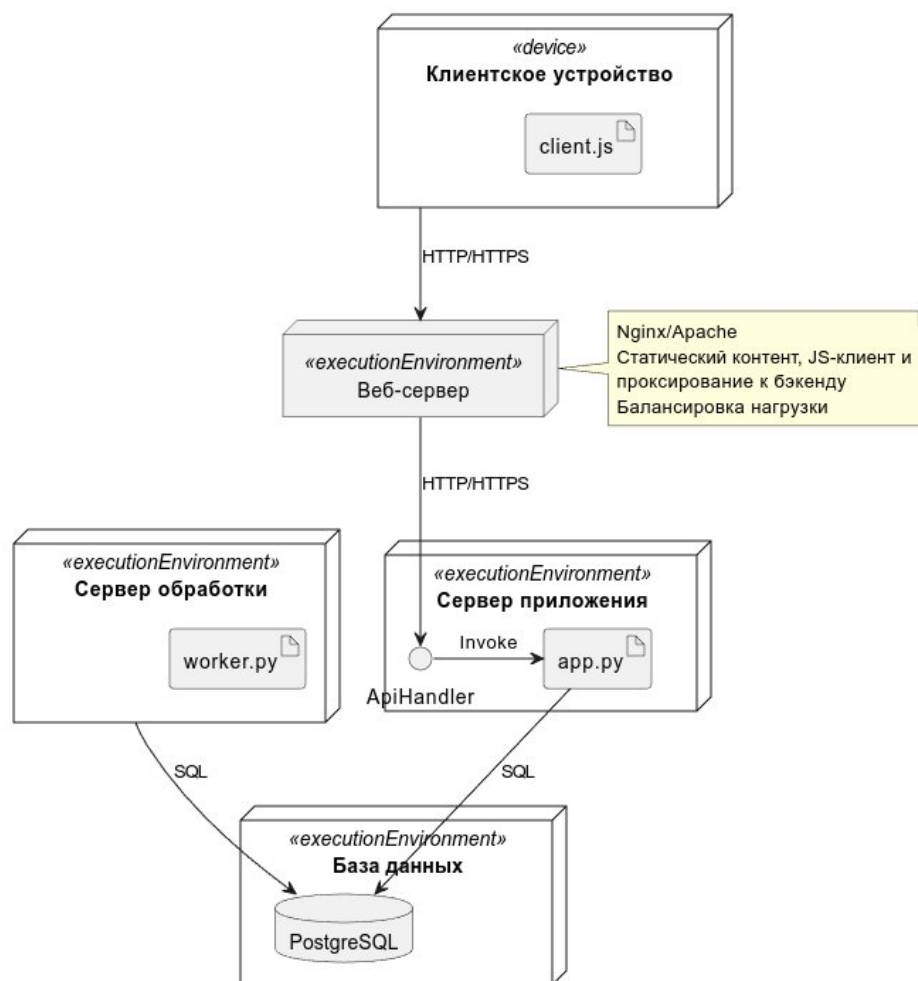


Рисунок 16 - Диаграмма развертывания

Клиентский уровень представлен веб-браузерами и внешними приложениями, которые взаимодействуют с системой через веб-сервер Nginx, выполняющий функции балансировщика нагрузки. Nginx обслуживает статический контент одностраничного приложения и осуществляет проксирование API-запросов к серверу приложений.

На уровне приложения работает FastAPI-сервер, который обрабатывает входящие запросы и управляет бизнес-логикой системы. Модуль «app.py» инкапсулирует бизнес-логику системы и обеспечивает обработку входящих запросов. Для хранения метаданных задач, результатов профилирования и служебной информации используется база данных PostgreSQL.

Вычислительно интенсивные операции классификации делегируются специализированному серверу обработки, на котором развернут Celery Worker («worker.py»). Данный компонент отвечает за выполнение прогнозирования

демографических характеристик на основе извлеченных лингвистических признаков.

3.7 Методы подбора гиперпараметров

Для настройки гиперпараметров моделей машинного обучения используется метод «GridSearchCV», реализованный в библиотеке «scikit-learn». Выбор данного подхода обусловлен его способностью выполнять полный перебор по заданной сетке значений гиперпараметров с одновременной оценкой качества с помощью кросс-валидации. Это позволяет систематически сравнивать все комбинации параметров и выбирать конфигурацию, обеспечивающую наилучшую обобщающую способность модели, а также снижает риск переобучения за счёт использования нескольких отложенных выборок.

3.8 Контрольный пример

В данном разделе представлен контрольный пример, демонстрирующий процесс взаимодействия пользователя с разработанной системой автоматического профилирования автора текста через веб-интерфейс.

3.8.1 Сценарий взаимодействия с системой

Процесс работы пользователя с системой включает следующие последовательные этапы:

1. Доступ к системе: Пользователь открывает главную страницу веб-интерфейса системы профилирования;
2. Ввод данных: Пользователь вводит анализируемый текст одним из трех способов:
 - Непосредственный ввод текста в текстовое поле;
 - Загрузка текстового файла через кнопку «Загрузить файл»;
 - Выбор демонстрационного примера из выпадающего списка;
3. Выбор модели: Пользователь выбирает модель классификации из списка доступных обученных моделей;
4. Запуск анализа: Пользователь нажимает кнопку «Прогнозировать» для инициирования процесса анализа;

5. Получение результатов: Система автоматически обновляет раздел с информацией по последнему прогнозированию. Нажатие на заголовок этого раздела приведет к перенаправлению пользователя на страницу с подробными результатами профилирования.

3.8.2 Главная страница

Рисунок 17 - Главная страница

Главная страница системы, изображенная на рисунке 17, содержит следующие ключевые элементы:

- Текстовое поле ввода с интеллектуальной валидацией, включая:
 - Подсказку с рекомендуемыми параметрами текста: «Текст для профилирования (рекомендуемая длина 300-10000 знаков)»;

- Индикатор текущего количества символов с визуальным отображением соответствия рекомендуемому диапазону;
- Модуль загрузки файлов с поддержкой формата TXT;
- Селектор моделей классификации;
- Библиотека демонстрационных примеров, включающая:
 - Тексты различной стилистической направленности;
 - Примеры с заранее известными демографическими характеристиками авторов;
 - Возможность быстрой загрузки примеров для тестирования системы.

3.8.3 Процесс обработки текста

Для демонстрации работоспособности разработанной системы автоматического профилирования был выбран текст из корпуса BlogsGenderAge, не входивший в обучающую выборку. Полный исходный текст примера приведен в Приложении А. Рассмотрим поэтапный процесс обработки текста системой.

Шаг 1 - Текст «Вопрос про студию. Слушайте, друзья...» загружен в систему. На этом этапе не производится никакой обработки, система лишь принимает данные для дальнейшего анализа.

Шаг 2 - Базовая очистка от разметки Явной HTML-разметки для удаления не обнаружено. Исходный текст остается без изменений.

Шаг 3 - Проверка структурной корректности. Система проверяет текст:

- Пустой текст: Нет, текст содержит 367 слов;
- Недопустимая длина: Нет, количество токенов (2075) находится в допустимом диапазоне;
- Нечётное количество кавычек/скобок: Проверка пройдена. Кавычки и скобки в тексте сбалансированы (цитата «На заседании думы...», упоминание магазина «Нагорный»).

Текст признан структурно корректным и передан на следующий этап без изменений.

Шаг 4 - Нормализация русских символов. Во всем тексте буква «ё» заменена на «е». («... Центр города всё-таки ...» → «... Центр города все-таки ...», «... чётко под ...» → «... четко под ...»)

Шаг 5 - Выделение сущностей и лемматизация. Система просканировала текст, используя комбинацию правил и NER-модели, и идентифицировала следующие типы именованных сущностей и структурных элементов:

- «DATE»: Присутствуют. Упоминание даты «9 декабря»;
- «ENUM»: Присутствуют. Нумерованные списки «(раз)», «(два)»;
- «MEAS»: Присутствуют. Денежные суммы, напр., «10 810 000 рублей»;
- «URL»: Присутствуют. Прямая ссылка «[http://sarov.nnov.ru/...](http://sarov.nnov.ru/)»;
- «SMILE»: Присутствуют. Текстовые смайлы «:», «:», «:(((»;
- «NUMBER»: Присутствуют. Много числовых данных;
- «QUOTE»: Обнаружена цитата. «На заседании думы ...»

После выделения всех сущностей и лемматизации внутреннее представление текста выглядит следующим образом «Сейчас вот читать - [QUOTE]. В тот число магазин [QUOTE], вот думать мочь он прикупить? [SMILE]»

Шаг 6 - Извлечение признаков. Система вычислила все заданные метрики, результаты которых представлены в таблице 3.

Таблица 3 - Извлеченные признаки по контрольному примеру

Признак	Значение	Признак	Значение
symbols_length	78.12	mean_usage VERB	1.731
sentences_count	26,00	mean_usage DET	0.154
long_sentences_proportion	0.615	mean_usage CCONJ	0.385
unpopular_words_proportion	0.307	mean_usage AUX	0.192
obscene_proportion	0.0	mean_usage ADP	0.923
rare_words_proportion	0.493	mean_usage ADJ	0.731
type_token_ratio	0.351	mean_usage PRON	0.577
normalized_shannon_entropy	1.462	mean_usage NOUN	3.385
mean_words_length	4,64	('ADJ-Pos', 'NOUN')	0.424
long_words_proportion	0.395	('PROPN', 'PUNCT')	0.03
lexicon_size	6.615	('PUNCT', 'SCONJ')	0.152
lexical_diversity_per_sentence	4.577	('ADP', 'ADJ-Pos')	0.152
mean_usage NUMBER	0.654	('ADP', 'PRON')	0.03

Признак	Значение	Признак	Значение
mean_usage_SMILE	0.154	('PUNCT', 'ADV')	0.061
mean_usage_QUOTE	0.077	('PUNCT', 'PRON')	0.091
mean_usage_URL	0.038	('ADJ-Pos', 'PUNCT')	0.03
mean_usage_DATE	0.038	('PUNCT', 'CCONJ')	0.03
mean_usage_ENUM	0.038	index_formality_tuldava	100.0
breadth_of_use_entities	0.429	index_formality_heylinger	60.0
index_of_formality	60.5	breadth_of_use_puncts	0.368
breadth_of_use_pos	0.812	mean_usage_ (	0.269
pair_of_adv_per_sentence	0.038	mean_usage_ -	0.269
mean_usage_PART	0.654	mean_usage_ :	0.038
mean_usage_SCONJ	0.269	mean_usage_ .	0.962
mean_usage_PROPN	0.115	mean_usage_)	0.269
mean_usage_ADV	0.615	mean_usage_ ,	0.731
mean_usage_NUM	0.538	mean_usage_ ?	0.115

Шаг 7 - Классификация. Полученный вектор признаков был отправлен на вход модели. Выполнив необходимые преобразования, модель сформировала прогноз в виде JSON-объекта, содержащего целевые атрибуты профиля. Данный результат был передан в модуль визуализации для отображения в веб-интерфейсе.

3.8.4 Страница результатов анализа

После завершения обработки, пользователь может перейти на страницу с выявленным профилем автора. Страница результатов показана на рисунке 18.

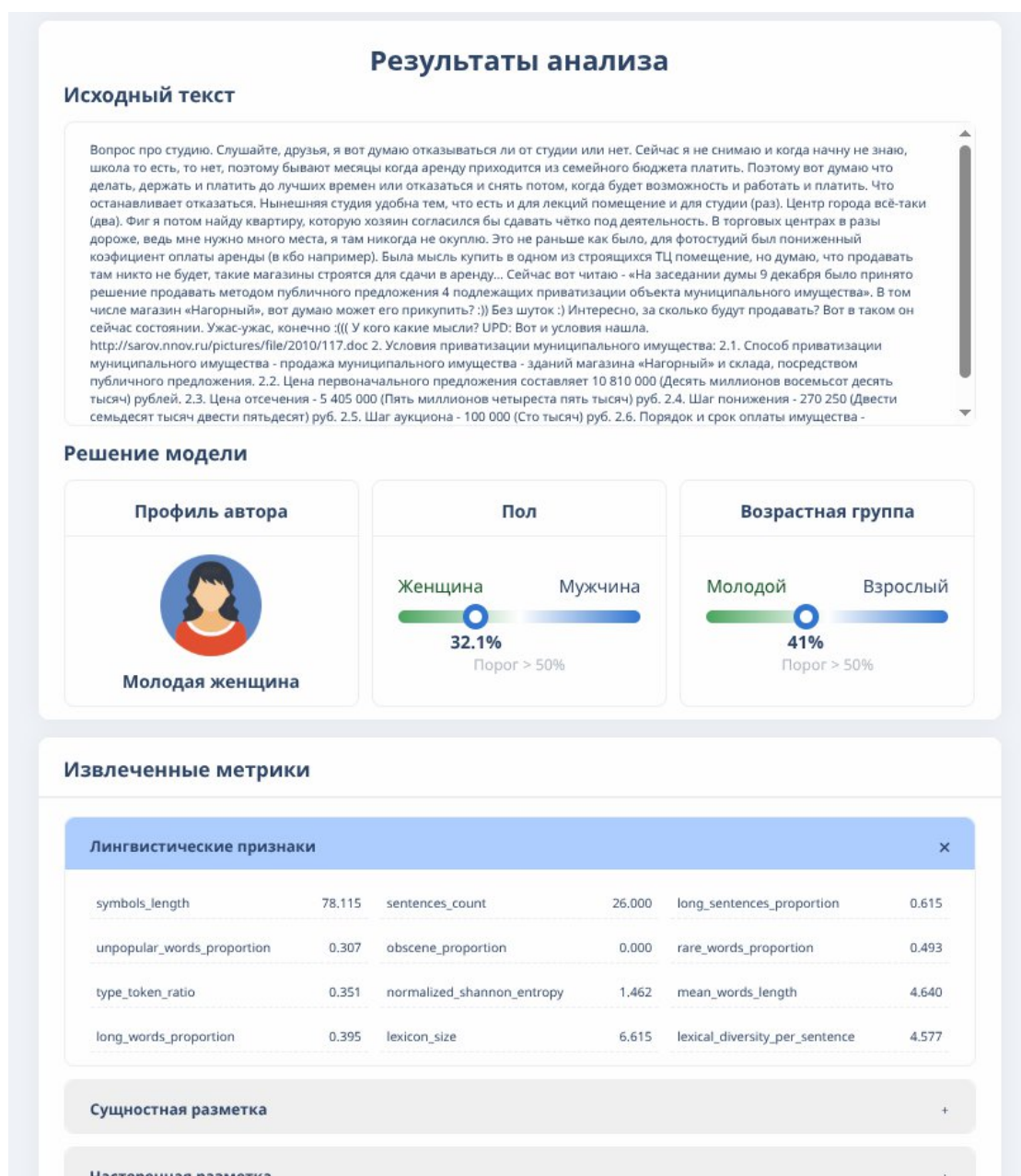


Рисунок 18 - Страница результатов

На странице результатов содержится:

- Исходный текст;
- Блок демографического профиля:
 - Вероятность принадлежности автора к полу с указанием порога классификации;
 - Вероятность отнесения автора к возрастной категории с порогом классификации;
 - Итоговые категориальные значения пола и возрастной группы

- Раскрывающиеся блоки с количественными значениями извлеченных стилометрическими признаками.

По результатам обработки и анализа исходного текста вероятность принадлежности автора к классу «мужчина» составила 0,32, к классу «взрослый» - 0,41. Оба значения ниже установленного порога (0,5), поэтому система классифицировала автора как молодую женщину.

3.8.5 Информационная панель моделей

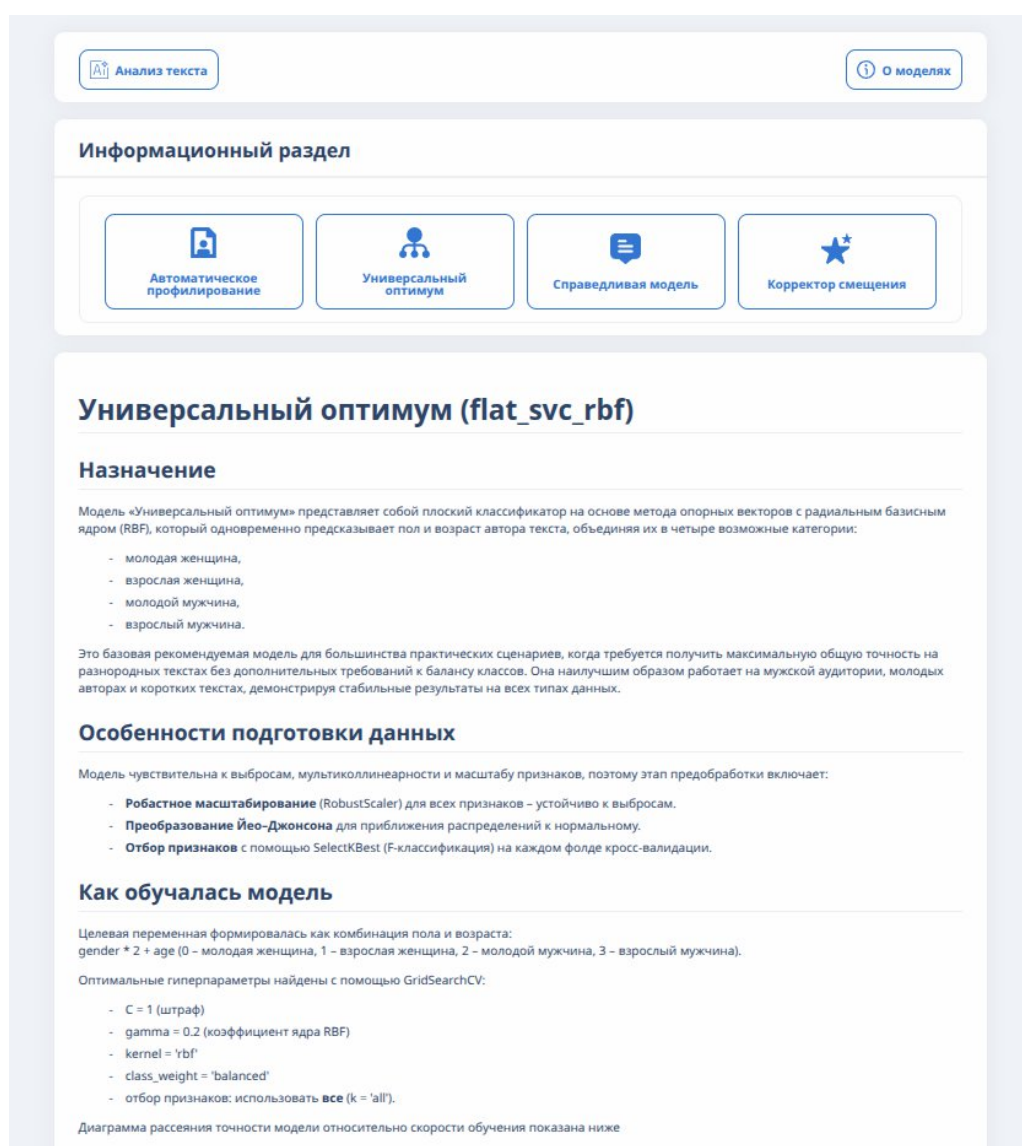


Рисунок 19 - Страница информации

На рисунке 19 представлен специализированный раздел «Модели», в котором для каждой реализованной прогностической модели приводится полная информация. В разделе указаны способы и параметры обучения, перечень применённых методов предобработки, параметры лучшей модели. Результаты

валидации представлены детализированными метриками качества (accuracy, F1-мера), матрицами ошибок для многоклассовой классификации и сравнительными характеристиками на различных подвыборках. В конце раздела содержится область применимости модели и известные ограничения.

3.9 Выводы по главе

В данной главе была представлена комплексная архитектура системы автоматического профилирования автора текста, разработанная для интеграции в бизнес-процессы цифровых платформ. На основе методологии IDEF0 выполнено концептуальное проектирование, демонстрирующее переход от текущей системы таргетирования, зависимой от профильных данных, к целевой системе с автоматическим профилированием на основе анализа пользовательского контента.

Архитектура системы реализована по модульному принципу, что обеспечивает высокую масштабируемость и поддерживаемость. Ключевые компоненты включают модуль извлечения лингвистических признаков, REST API с асинхронной обработкой запросов и механизм задач для обеспечения отказоустойчивости. Диаграммы классов и последовательностей детально описывают взаимодействие между компонентами системы на всех этапах обработки - от валидации входных данных до формирования профиля. Для настройки моделей машинного обучения описан метод подбора гиперпараметров GridSearchCV, позволяющий систематически перебирать комбинации параметров и выбирать оптимальную конфигурацию на основе кросс-валидации.

В рамках контрольного примера приведено описание обработки исходного текста, проверена работа веб-интерфейса и визуализация результатов профилирования.

ГЛАВА 4. ТЕСТИРОВАНИЕ МЕТОДОВ АВТОМАТИЧЕСКОГО ПРОФИЛИРОВАНИЯ АВТОРА ТЕКСТА

В данной главе представлены эксперименты, направленные на эмпирическую оценку предложенных подходов к автоматическому профилированию автора.

Цель экспериментов - оценить способность модельных конфигураций извлекать профиль автора из лингвистических признаков текста. Оценка проводится на отложенной тестовой выборке и включает анализ стабильности и обобщающей способности.

Ключевым фактором анализа в проведённых экспериментах является рассмотрение различных подвыборок данных. Выделяются подвыборки по полу автора (мужская и женская), по возрастной группе (молодые авторы и взрослые), а также по длине текста. Для анализа влияния объёма текста используются две категории: короткие тексты (менее 2000 символов) и длинные тексты (более 2000 символов).

4.1 Тестирование стабильности моделей

Под стабильностью модели понимается устойчивость её результатов к вариациям данных. Для оценки стабильности выборка разбивается на несколько блоков, на которых рассчитывается показатель точности. На основе этих оценок считается стандартное отклонение (std) и коэффициент вариации, определяемый как отношение стандартного отклонения к среднему значению точности. Для проверки гипотезы о равенстве дисперсий на различных подвыборках используется тест Левена.

4.1.1 Модели прогнозирования пола

На столбчатой диаграмме (рисунок 20) представлены сводные результаты оценки всех рассмотренных алгоритмов на тестовой выборке. Столбцы соответствуют средним значениям точности для каждой модели, а планки погрешности отображают разброс (стандартное отклонение) результатов по блокам. Коэффициент вариации для каждой модели указан на столбце соответственно.

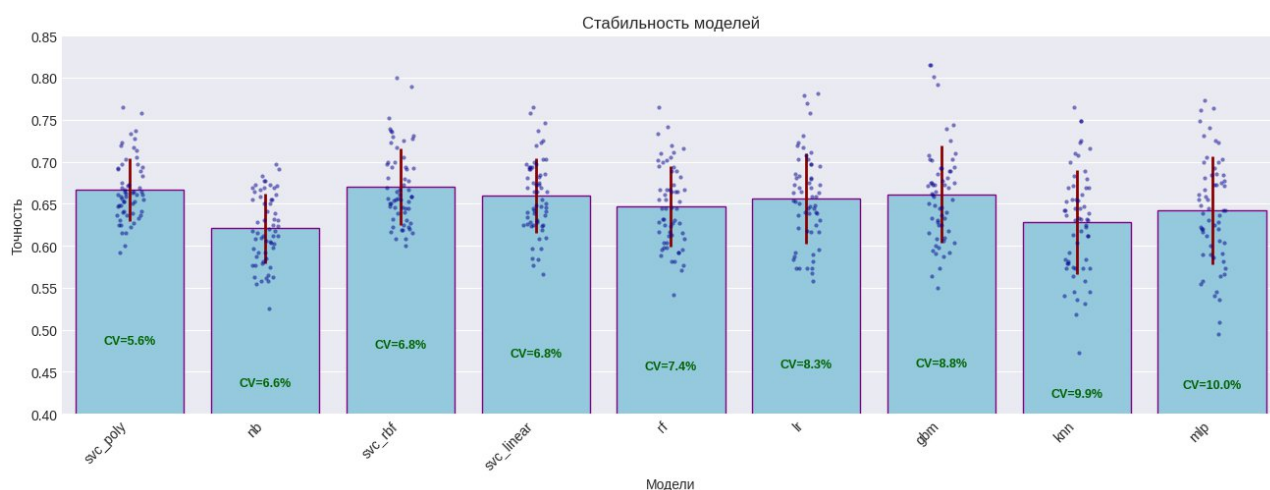


Рисунок 20 - Оценки моделей прогнозирования пола

Наименее эффективным оказался наивный байес, заметно уступающий SVM-моделям. Лучшие показатели точности у SVC с RBF и полиномиальным ядром, при этом SVC-poly лидирует по стабильности. Градиентный бустинг и линейный SVC занимают промежуточное положение, а KNN и MLP, напротив, демонстрируют наибольшую вариативность и низкую надёжность.

Таким образом, SVC-RBF обеспечивает наилучшую точность, SVC-poly - оптимальный баланс качества и устойчивости. Выбор между ними определяется приоритетами задачи.

Тест Левена, результаты которого изложены в таблице 4, не выявил статистически значимых различий дисперсий между подвыборками ($p > 0,05$ для всех моделей), причём SVC-poly подтверждает однородность максимально ($p = 0,981$), а NB ближе всего к порогу значимости ($p = 0,077$).

Таблица 4 - Тест Левена моделей прогнозирования пола

Модель	P-value	Levene-статистика
GBM	0,890	0,334
KNN	0,933	0,259
LR	0,946	0,233
MLP	0,621	0,706
NB	0,077	2,123
RF	0,292	1,266
SVC (LINEAR)	0,880	0,350
SVC (POLY)	0,981	0,144
SVC (RBF)	0,450	0,961

На рисунке 21 представлена визуализация вариации моделей на различных подвыборках. Значение коэффициента вариации (CV) изменяется от 5,9% до 9,3% в зависимости от категории: наименее вариативной оказалась выборка взрослых авторов, в то время как наибольшая нестабильность наблюдается на выборке женщин.

Диаграмма наглядно демонстрирует, что мужская подвыборка дает наивысшую среднюю точность, тогда как женская - самую низкую.

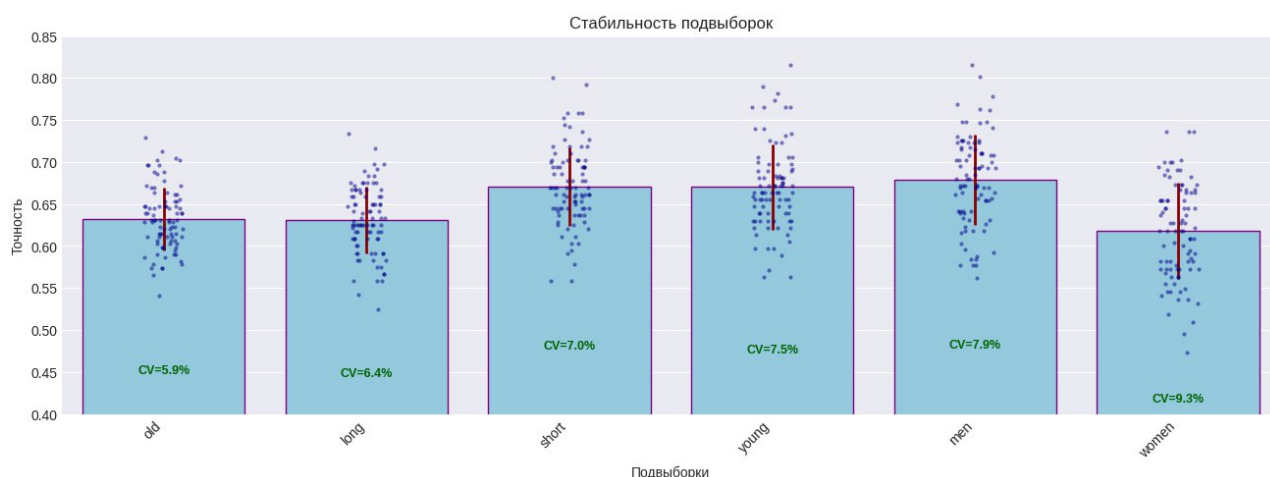


Рисунок 21 - Оценка стабильности прогнозирования пола на подвыборках

Минимальное стандартное отклонение на разных подвыборках достигается разными моделями: для длинных текстов и женщин - градиентный бустинг, для мужчин и молодых - SVC-poly, для взрослых - наивный байес, для коротких - MLP. Стандартные отклонения моделей по подвыборкам показаны на таблице 5.

Таблица 5 - Модели прогнозирования пола с минимальным отклонением по выборкам.

Подвыборка	Модель	Стандартное отклонение
Длинные	GBM	0,032
Короткие	MLP	0,030
Взрослые	NB	0,018
Молодые	SVC (POLY)	0,040
Мужчины	SVC (POLY)	0,031
Женщины	GBM	0,031

Таким образом, для задачи определения пола семейство SVC демонстрирует наилучшие результаты: RBF-ядро обеспечивает максимальную точность, а полиномиальное ядро - оптимальное сочетание точности и

стабильности. Выбор конкретной модели зависит от приоритета (качество или устойчивость), при этом на отдельных подвыборках (например, женщины или длинные тексты) наименьшую вариативность показывает градиентный бустинг.

4.1.2 Модели прогнозирования возрастной группы

На рисунке 22 представлены агрегированные результаты оценки моделей классификации возрастной группы. Приведенная диаграмма позволяет наглядно сопоставить эффективность работы алгоритмов по ключевым метрикам, демонстрируя различия в их производительности на рассматриваемой задаче.

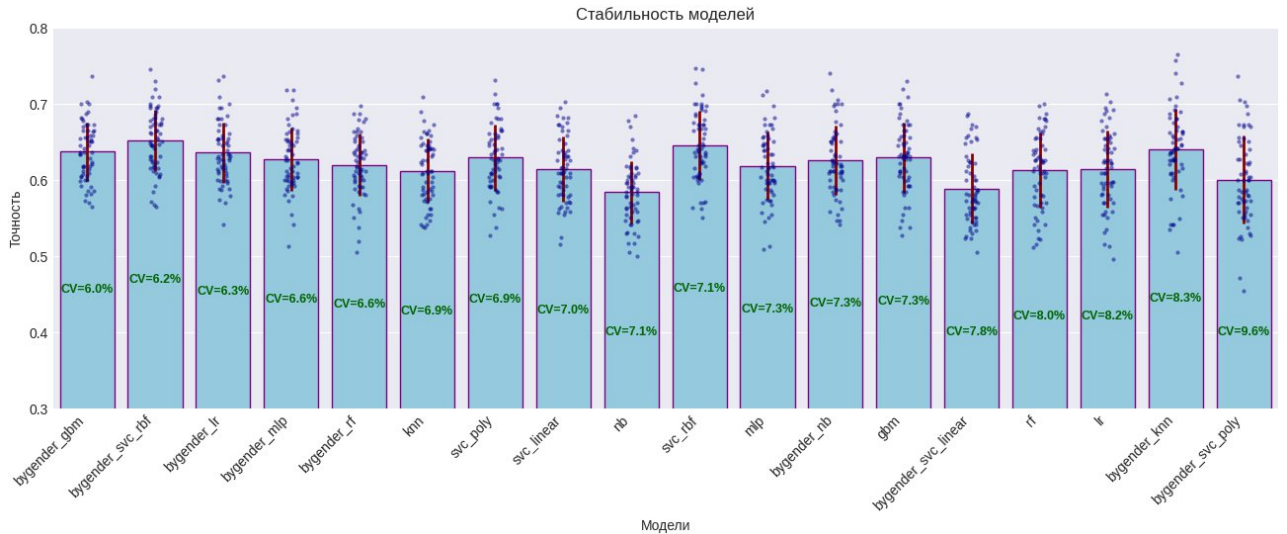


Рисунок 22 - Оценки моделей прогнозирования возрастной группы

Наилучшую точность показывает SVC с RBF-ядром, однако он уступает по стабильности гендерно-ориентированному градиентному бустингу, который обеспечивает минимальный разброс результатов при сопоставимом качестве.

Результаты теста Левена, собранные в таблице 6, не выявляют статистически значимых различий дисперсий между подвыборками при обычном пороге значимости: все р-значения превышают 0.05.

Таблица 6 - Тест Левена моделей прогнозирования возрастной группы

Модель	P-value	Levene- стат.	Модель	P-value	Levene- стат.
Гендерная GBM	0,672	0,638	GBM	0,995	0,081
Гендерная KNN	0,542	0,818	KNN	0,605	0,729
Гендерная LR	0,582	0,761	LR	0,418	1,015
Гендерная MLP	0,539	0,823	MLP	0,744	0,542
Гендерная NB	0,800	0,466	NB	0,366	1,109

Гендерная RF	0,776	0,499	RF	0,498	0,885
Гендерная SVC (LINEAR)	0,855	0,388	SVC (LINEAR)	0,550	0,807
Гендерная SVC (POLY)	0,818	0,440	SVC (POLY)	0,436	0,985
Гендерная SVC (RBF)	0,844	0,404	SVC (RBF)	0,559	0,793

Рисунок 23 наглядно демонстрирует вариативность моделей на различных подвыборках. Диаграмма показывает, что наименьшую изменчивость результатов демонстрируют модели на длинных текстах, тогда как наибольший разброс наблюдается на выборке молодых авторов.

Диаграмма также отражает различия в средних уровнях качества по подгруппам: женская подвыборка характеризуется несколько более высокой средней точностью по сравнению с мужской, однако сопровождается иным разбросом значений.

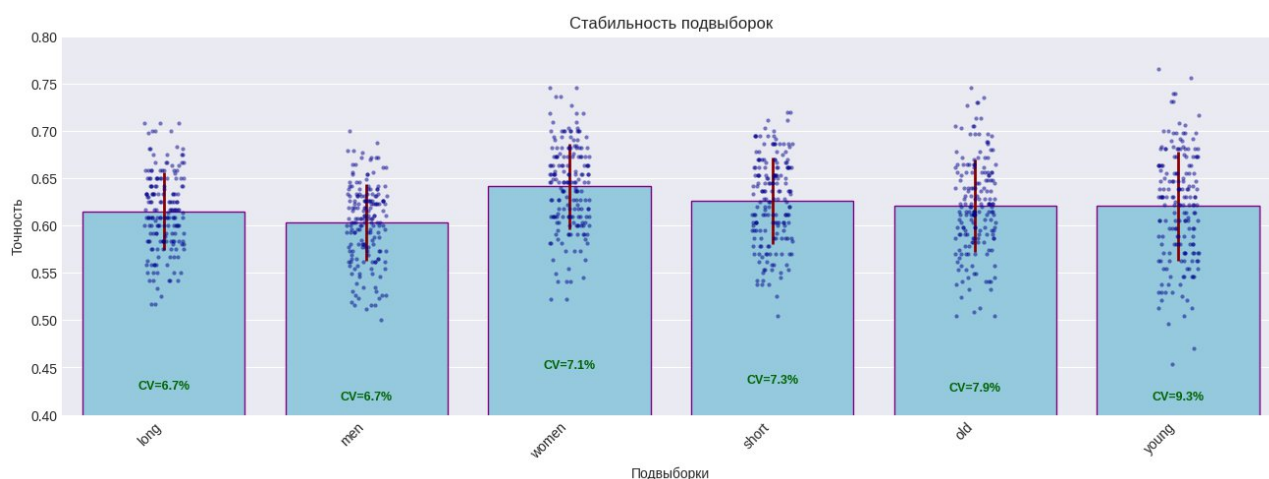


Рисунок 23 - Оценка стабильности прогнозирования возрастной группы на подвыборках

Минимальное стандартное отклонение на различных подвыборках достигается разными алгоритмами: для длинных текстов это гендерно-ориентированная логистическая регрессия, для мужской подвыборки - гендерно-специфический MLP, для взрослых авторов - наивный байес, для коротких текстов - гендерно-ориентированный SVC-RBF, для женщин - наивный байес, а для молодых - гендерно-ориентированный GBM. Модели с минимальными значениями показаны в таблице 7.

Таблица 7 - Модели прогнозирования возрастной группы с минимальным отклонением по выборкам.

Подвыборка	Модель	Стандартное отклонение
Длинные	Гендерный LR	0,026
Короткие	Гендерный SVC (RBF)	0,029
Взрослые	Гендерный MLP	0,023
Молодые	NB	0,024
Мужчины	Гендерный GBM	0,036
Женщины	NB	0,027

Таким образом, для задачи прогнозирования возрастной группы SVC-RBF и его гендерно-адаптированные варианты демонстрируют лучшие средние показатели, тогда как гендерно-ориентированный GBM обеспечивает оптимальное сочетание качества и предсказуемости. Конкретный выбор модели должен базироваться на требованиях к точности или к устойчивости на целевых подвыборках.

4.1.3 Модели прогнозирования профиля

Агрегированные результаты моделей для задачи прогнозирования составного профиля представлены на рисунке 24. На столбчатой диаграмме наглядно показаны средние значения метрик и их вариативность, что позволяет оценить стабильность работы каждой модели и сопоставить их эффективность.

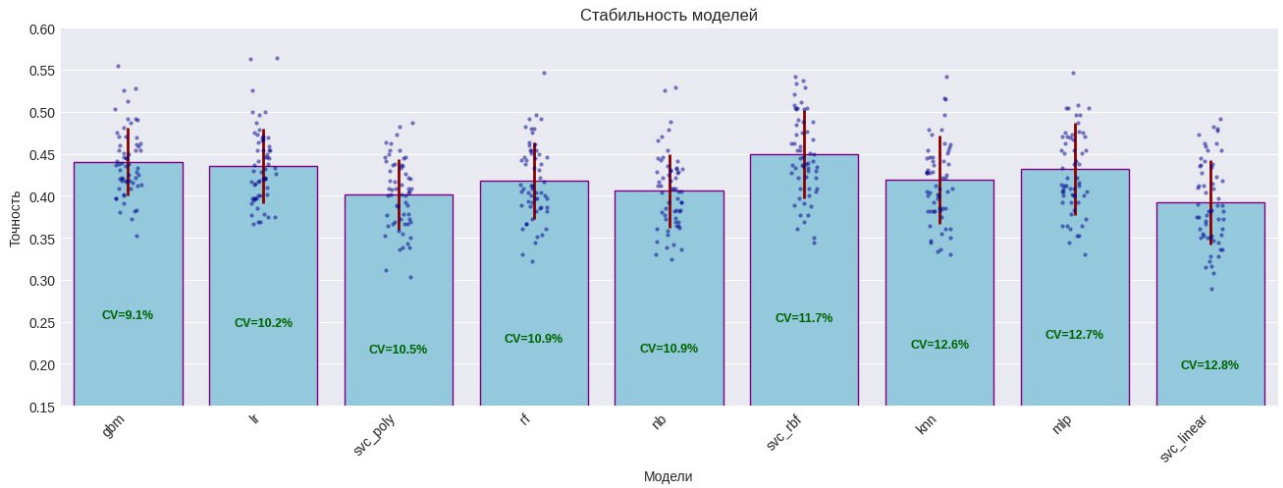


Рисунок 24 - Оценки моделей прогнозирования профиля

Наилучшую среднюю точность демонстрируют SVC с RBF-ядром и градиентный бустинг, при этом GBM обеспечивает минимальную относительную вариативность и наиболее предсказуемое поведение между

блоками. SVC-RBF лидирует по качеству, но уступает в стабильности; SVC с полиномиальным ядром выступает компромиссным вариантом. Логистическая регрессия и MLP занимают промежуточные позиции, а линейный SVC и наивный байес показывают наименьшую эффективность.

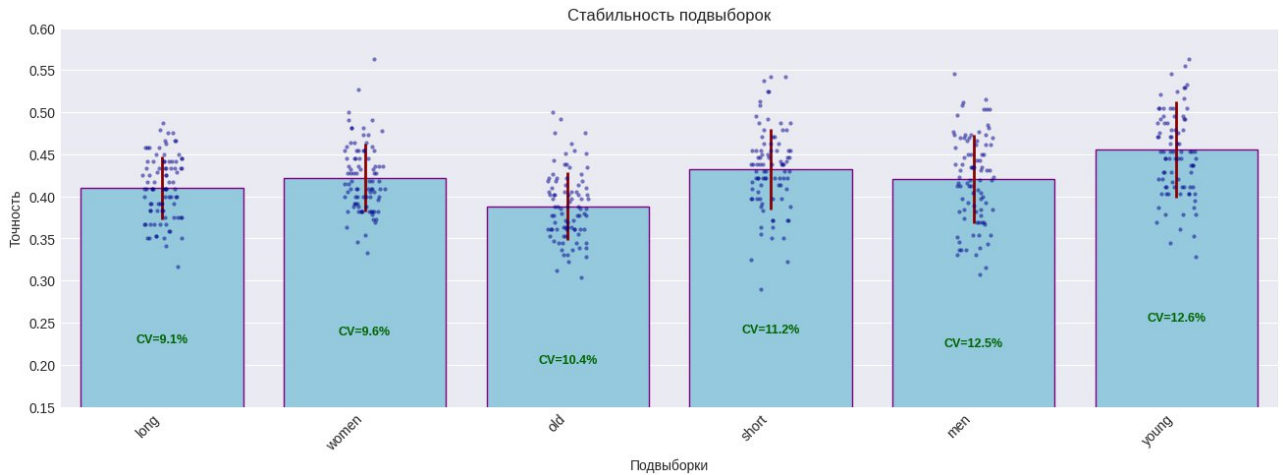


Рисунок 25 - Оценка стабильности прогнозирования профиля на подвыборках

Результаты анализа эффективности моделей на различных подвыборках представлены на рисунке 25. Длинные тексты и женская подвыборка характеризуются умеренной производительностью и низкой изменчивостью, в то время как короткие тексты и мужская подвыборка демонстрируют более высокую вариативность.

Минимальное стандартное отклонение на разных группах достигается разными алгоритмами: для длинных текстов и мужчин - GBM, для взрослых - наивный байес, для коротких и молодых - SVC-poly, для женщин - MLP. Лучшие модели с минимальным стандартным отклонением по подвыборкам изложены в таблице 8.

Таблица 8 - Модели прогнозирования профиля с минимальным отклонением по выборкам.

Подвыборка	Модель	Стандартное отклонение
Длинные	GBM	0,023
Короткие	SVC (POLY)	0,029
Взрослые	GBM	0,016
Молодые	MLP	0,036
Мужчины	SVC (POLY)	0,029
Женщины	NB	0,031

Тест Левене (таблица 9) не выявил статистически значимых различий дисперсий между подвыборками; наиболее однородна дисперсия у SVC-RBF, наивный байес приближается к порогу значимости. Таким образом, эмпирические различия в устойчивости не подтверждаются формально.

Таблица 9 - Тест Левена моделей прогнозирования профиля

Модель	P-value	Levene-статистика
GBM	0,499	0,883
KNN	0,286	1,281
LR	0,765	0,513
MLP	0,866	0,372
NB	0,083	2,074
RF	0,562	0,789
SVC (LINEAR)	0,825	0,431
SVC (POLY)	0,749	0,534
SVC (RBF)	0,961	0,200

Для построения каскадного подхода целесообразно выбирать модели, которые одновременно демонстрируют высокую точность и устойчивость на соответствующих задачах. На первом этапе, где определяется пол автора, наиболее рационально использовать SVC с RBF-ядром или SVC с полиномиальным ядром: первая модель обеспечивает наилучшую среднюю точность, тогда как вторая отличается минимальной вариативностью результатов. На втором этапе, где возраст прогнозируется внутри каждой половой группы, предпочтительны гендерно-специализированные модели, показавшие хорошее сочетание точности и стабильности, прежде всего градиентный бустинг и SVC с RBF-ядром, обученные отдельно для мужчин и женщин.

4.2 Тестирование обобщённости моделей

Оценка обобщающей способности моделей позволяет определить, насколько устойчиво алгоритмы сохраняют разделяющую силу на различных подвыборках данных. Анализ проводится как на полной тестовой выборке, так и на стратифицированных группах, выделенных по полу, возрасту и длине текста. В рамках анализа исследуются смещения предсказаний, минимальное гарантированное качество и характер ошибок классификации. Такой подход

даёт возможность выбрать не только наиболее точные, но и максимально надёжные и сбалансированные решения для задачи автоматического профилирования автора.

4.2.1 Базовая модель

Базовая стратегия, предсказывающая всегда «молодой мужчина», даёт точность 32.9% на тестовой выборке. Распределение классов иллюстрировано на рисунке 26.



Рисунок 26 - Распределение классов в тестовой выборке

Это значение служит простым ориентиром для оценки реальной пользы обученных моделей: любые улучшения над этим уровнем означают практическую добавленную ценность классификатора в условиях имеющегося дисбаланса.

4.2.2 Общая оценка моделей

Сводные результаты оценок моделей на всей тестовой выборке, показанные на рисунке 27, показывают, что по средней точности и F1 лучшие результаты демонстрирует «flat_svc_rbf» (accuracy = 0.450, F1 = 0.420), рядом идут каскадные комбинации «cascade_svc_poly_svc_rbf» и «cascade_svc_poly_gbm», а также «flat_gbm».

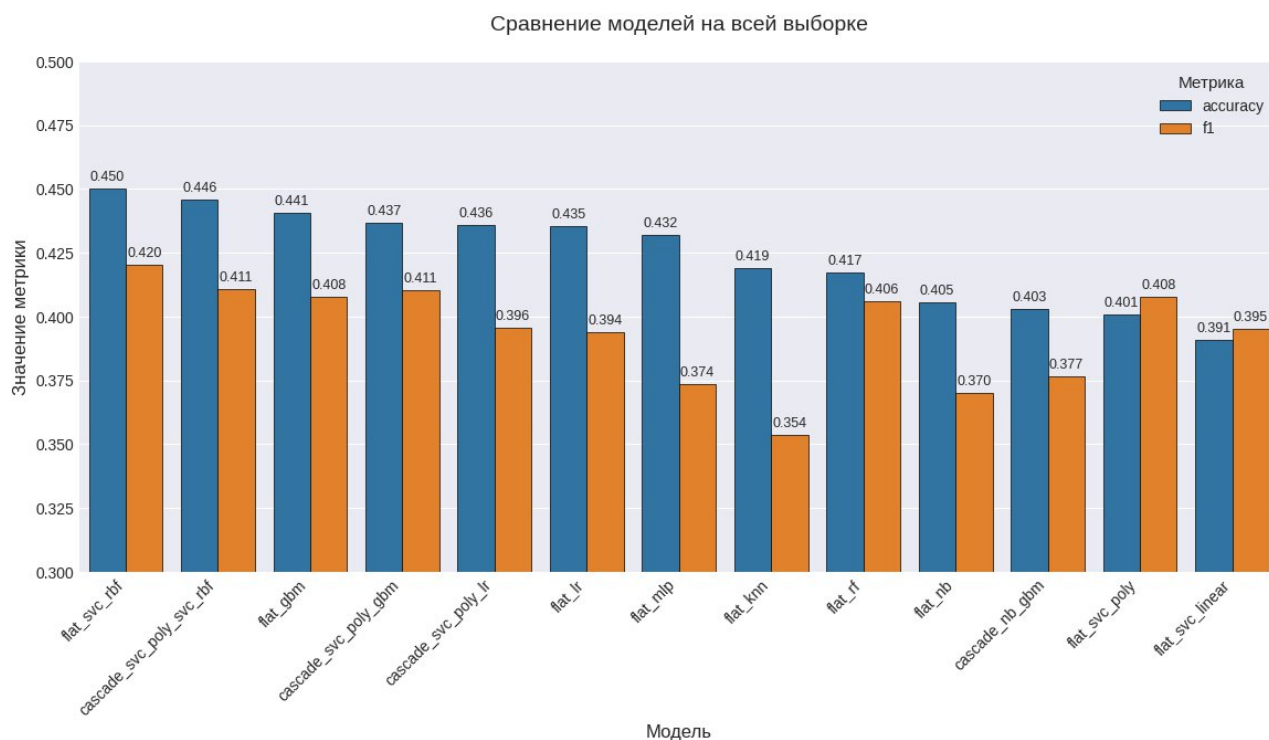


Рисунок 27 - Общая оценка моделей

Все модели увеличивают точность относительно базовой стратегии, но прирост остаётся умеренным: большинство моделей укладываются в диапазон примерно 40-45% по точности. Разрыв между лучшей моделью и остальными невелик, что указывает на ограниченный запас информативности признаков и сильное влияние классового дисбаланса.

4.2.3 Лучшие модели по подвыборкам

Анализ по подвыборкам выявляет, что «flat_svc_rbf» лидирует по общей выборке и в наиболее многочисленных группах (включая мужчин и молодых авторов), достигая на них наилучшей точности. Для коротких текстов и для молодой подвыборки его преимущество особенно заметно. Каскадные схемы проявляют себя сильнее в некоторых нишах: для женщин и для части взрослой подвыборки лучше показывают конфигурации «cascade_svc_poly_svc_rbf» и «cascade_svc_poly_gbm», а для длинных текстов предпочтительна «cascade_svc_poly_gbm». Это подтверждает, что каскадный подход даёт выигрывать в специфичных условиях, тогда как SVC-RBF остаётся наиболее универсальным по абсолютной точности. Лучшие модели показаны на рисунке 28.

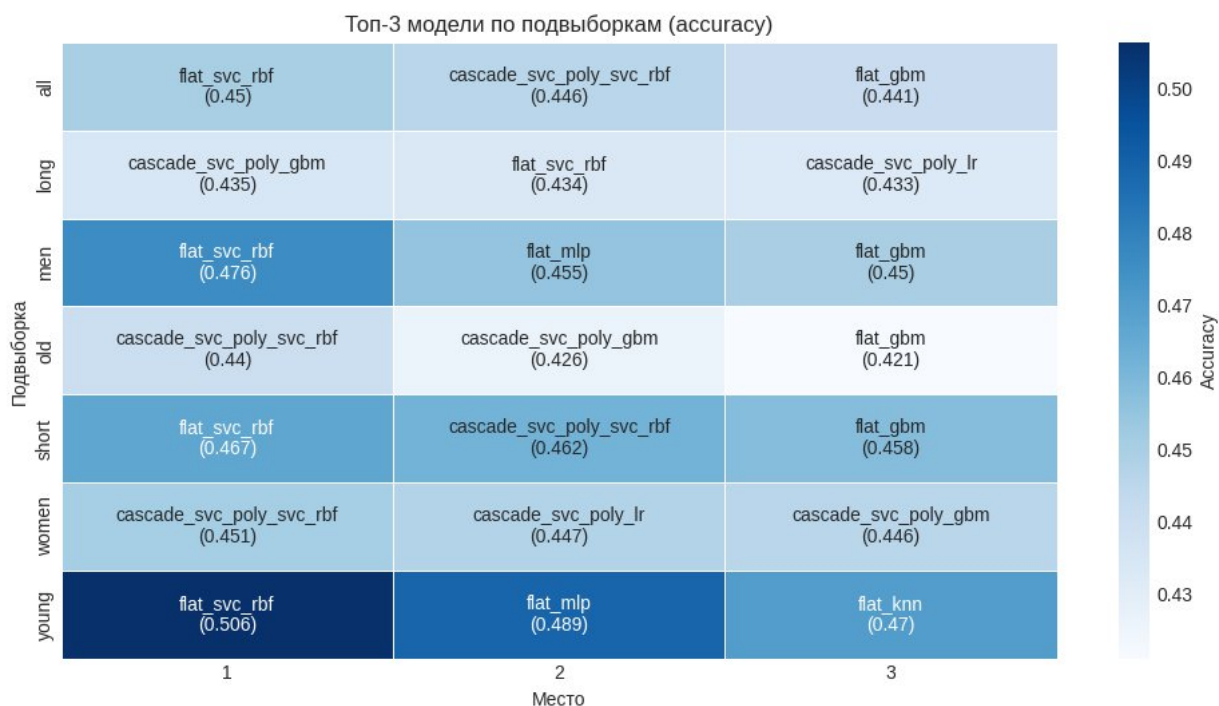


Рисунок 28 - Лучшие модели по подвыборкам

4.2.4 Оценка смещения моделей

Оценка смещений выявила, что ряд моделей систематически предпочитает доминирующие группы: «flat_svc_rbf» благоприятствует мужчинам и молодым, тогда как «flat_nb» и некоторые каскадные конфигурации более нейтральны. Наименьший возрастной сдвиг - у «cascade_svc_poly_svc_rbf», а по длине текста - у «cascade_svc_poly_gbm». Визуализация (рисунок 29) показывает, что минимизация смещения и максимизация точности часто конфликтуют, поэтому при выборе модели важно явно учитывать требования к справедливости.

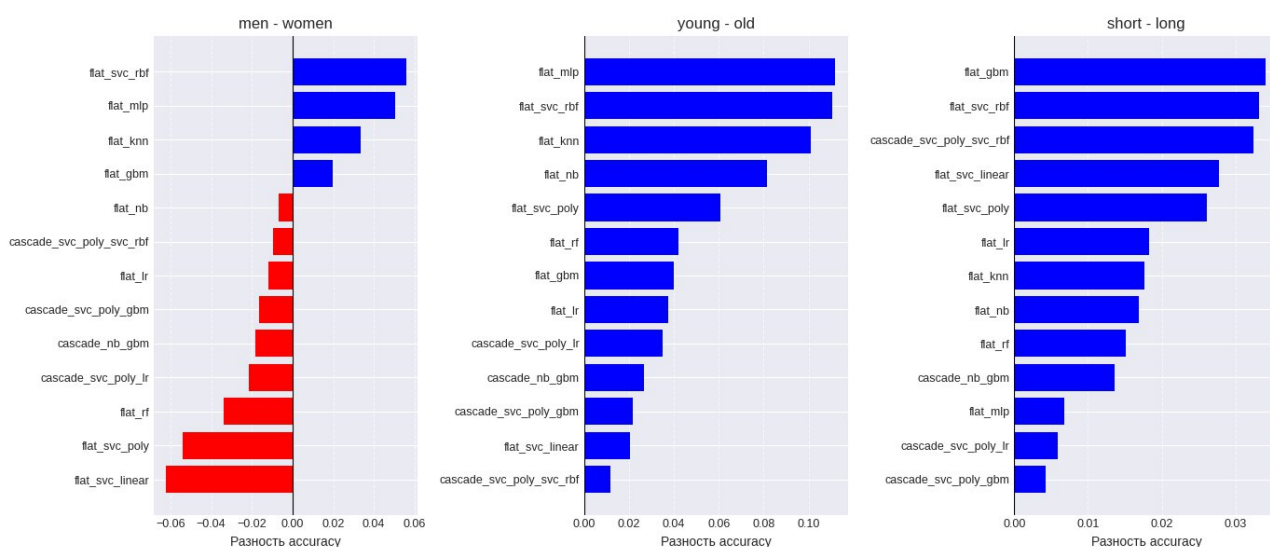


Рисунок 29 - Разница моделей на парных подвыборках

4.2.5 Модель с максимальным минимальным качеством

При применении максиминного критерия, результаты которого показаны в таблице 10, наилучший гарантийный уровень дают «cascade_svc_poly_svc_rbf», «cascade_svc_poly_gbm» и «flat_gbm». Это означает, что если важна гарантированная «нижняя граница» качества на худших подвыборках, то каскадная схема «SVC-poly» → «SVC-RBF» предпочтительнее моделей с наивысшей средней точностью, поскольку она лучше защищает от сильного падения качества в проблемных сегментах выборки.

Таблица 10 - Таблица результатов максиминного критерия

Модель	Точность
cascade_svc_poly_svc_rbf	0,430
cascade_svc_poly_gbm	0,426
flat_gbm	0,421
cascade_svc_poly_lr	0,419
flat_lr	0,417
flat_rf	0,396
flat_svc_rbf	0,396
cascade_nb_gbm	0,390
flat_mlp	0,377
flat_svc_poly	0,371
flat_knn	0,369
flat_nb	0,365
flat_svc_linear	0,362

4.2.6 Анализ матрицы ошибок

Матрицы ошибок, показанные на рисунке 30 подтверждают сильный эффект дисбаланса: большинство моделей склонно ошибочно относить образцы к доминирующему классу, в результате чего редкие классы распознаются значительно хуже. Исключением по балансу распознавания являются «flat_svc_poly» и «flat_svc_linear», которые показывают более равномерное распределение правильных предсказаний по классам. «flat_svc_poly» даёт существенно более высокую долю корректно распознанных молодых женщин по сравнению с остальными моделями, несмотря на более низкую точность.

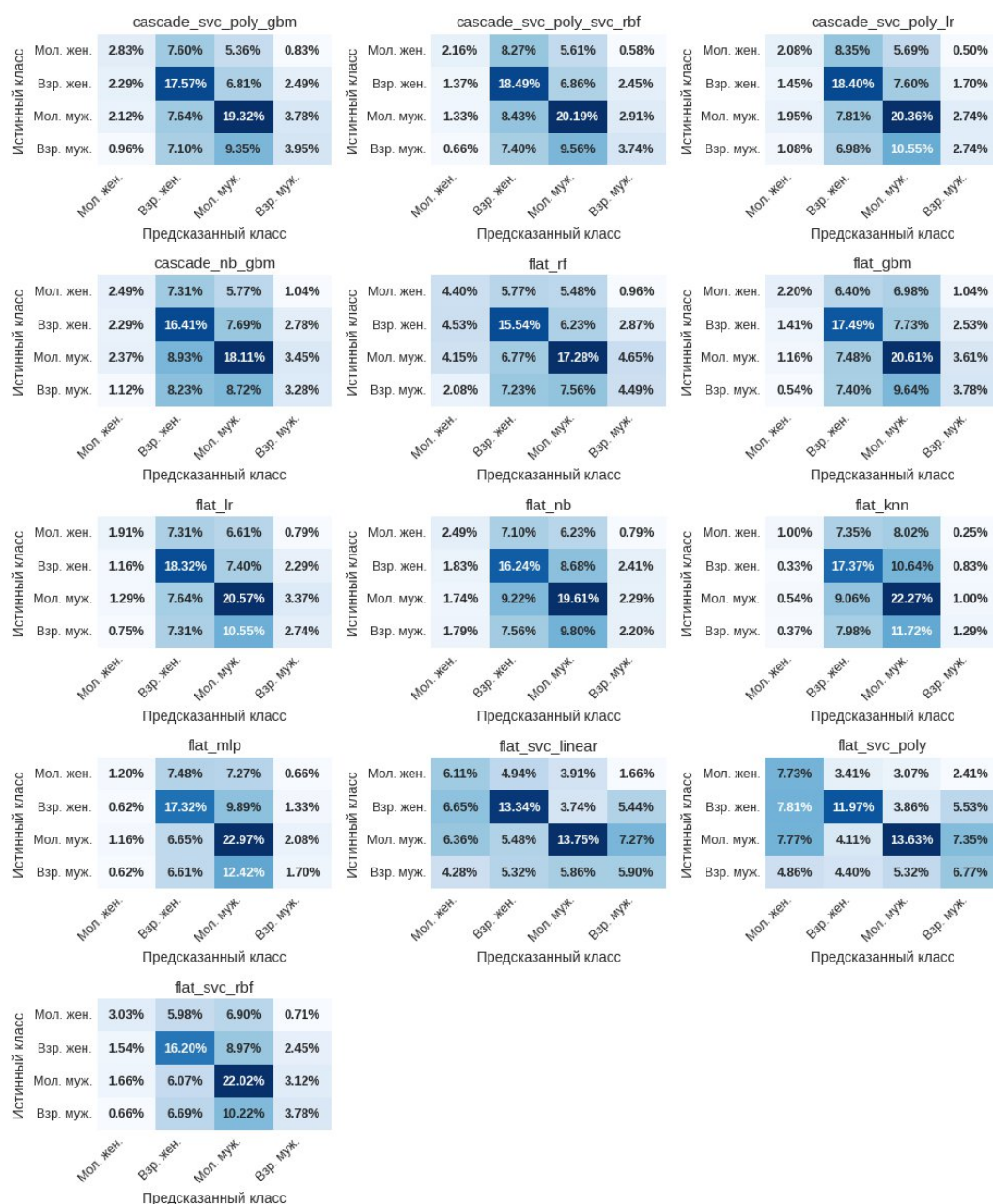


Рисунок 30 - Матрица ошибок

4.3 Результаты экспериментов

Эксперименты выявили компромисс между точностью и равномерностью распознавания классов: модель `flat_svc_rbf` показала наилучшую суммарную точность, тогда как `flat_svc_poly` и `flat_svc_linear` обеспечивают более сбалансированное распознавание редких классов.

Результаты указывают на ограниченную информативность текущего набора признаков - лучшие модели лишь умеренно превосходят базовую стратегию. Это говорит о слабом сигнале в данных, где различия между

группами авторов выражены недостаточно явно. Дальнейшее улучшение качества требует поиска новых признаков и углубленного анализа текстов.

Эксперименты подтверждают достижение цели исследования: разработанные модели способны извлекать из текста информацию, позволяющую автоматически определять пол и возрастную группу автора. При этом результаты показывают, что задача профилирования по тексту остаётся сложной и требует дальнейших исследований, направленных на усиление информативности признаков и улучшение методов представления текста.

4.4 Выводы по главе

В главе проведено экспериментальное сравнение алгоритмов машинного обучения для определения пола, возраста и полного профиля автора по лингвистическим признакам. Оценены стабильность моделей, обобщающая способность и поведение на различных подвыборках.

При определении пола лучшую точность показал SVC с RBF-ядром, наиболее стабильные результаты - SVC с полиномиальным ядром. Остальные алгоритмы несколько уступают и характеризуются высокой вариативностью.

Классификация возраста сложнее: лучшие результаты у SVC-моделей и градиентного бустинга, причём модели, разделенные по полу, значительно стабильнее. Общая точность ниже, чем при определении пола, что говорит о слабой выраженности возрастных различий в признаках.

Для совместного профилирования максимальную точность обеспечивает «flat_svc_rbf», более сбалансированное распознавание редких классов - «flat_svc_poly» и «flat_svc_linear». Каскадные схемы дают наилучшую гарантию качества на проблемных подвыборках.

Признаки содержат ограниченный, но устойчивый сигнал. Дальнейшее улучшение требует развития методов извлечения признаков. Разработаны рекомендации по выбору модели в зависимости от характеристик текста и приоритетов задачи для адаптации под конкретные бизнес-сценарии.

ЗАКЛЮЧЕНИЕ

В данной работе была разработана, реализована и всесторонне оценена система автоматического профилирования автора русскоязычного текста. Исследование представляет собой комплексное решение на стыке обработки естественного языка и машинного обучения, направленное на определение характеристик автора по его тексту.

Основные научные и практические результаты работы:

1. Разработан и научно обоснован методологический аппарат автоматического профилирования, основанный на стилометрическом подходе. В работе предложена формальная модель системы, включающая строго заданные операции предобработки, лингвистической аннотации, агрегирования признаков и их классификации, что обеспечивает воспроизводимость и прозрачность метода;
2. Создана и реализована программная система, архитектура которой предусматривает модульность, масштабируемость и возможность интеграции в бизнес-процессы цифровых платформ. Система обеспечивает сквозной процесс обработки текста - от его получения до формирования профиля;
3. Проведено комплексное экспериментальное исследование, в ходе которого был выполнен сравнительный анализ моделей профилирования. Эксперименты позволили выявить конфигурации моделей, обеспечивающие наилучший баланс между точностью, стабильностью и сбалансированностью предсказаний в зависимости от приоритетов задачи;
4. Получены практически значимые результаты, подтверждающие эффективность предложенного подхода:
 - Для определения пола автора наилучшие модели достигают точности около 67%, что существенно превышает уровень базовой стратегии;

- Для определения возрастной группы лучшие модели показывают точность порядка 64-65%, однако задача остаётся более сложной и чувствительной к характеристикам данных;
- При определении составного профиля наилучшие модели достигают точности около 45%, что заметно превосходит базовую стратегию (32.9%), но подтверждает сложность задачи и ограниченную информативность текущего набора признаков.

5. Разработаны практические рекомендации по выбору моделей в зависимости от характеристик текста и решаемой задачи, что позволяет адаптировать систему под конкретные условия эксплуатации.

Экспериментально подтверждено, что внедрение подобной системы позволяет повысить релевантность таргетированного контента и, как следствие, снизить стоимость привлечения клиента. Полученные результаты свидетельствуют о достижении поставленной цели исследования и подтверждают перспективность дальнейшего развития методов автоматического профилирования для русскоязычных текстов.

СПИСОК ЛИТЕРАТУРЫ

1. Романов А.С. Методология идентификации автора текста для решения задач информационной безопасности // Вопросы кибербезопасности 2024 № 3(61). С. 120-128
2. Соболев А.А., Федотова А.М, Куртукова А.В., Романов А.С., Шелупанов А.А. Методика определения возраста автора текста на основе метрик удобочитаемости и лексического разнообразия // Доклады ТУСУР, 2022, том 25, № 2. С. 45-52
3. Андреев И.А. Исследование методов и алгоритмов обработки текстовой информации социальных сетей в задачах формирования социального портрета пользователя - [Электронный ресурс]. Режим - доступа: <https://ulstu.ru/upload/iblock/93a/Dissertatsiya-Andreev.pdf> (дата обращения 18.10.2025)
4. Литвинова Т.А., Бюрикова, Е.Д., Загоровская О.В. Диагностирование пола, возраста и психологических характеристик автора письменного текста с использованием методов корпусной и компьютерной лингвистики: возможности и ограничения // ВОПРОСЫ КРИМИНОЛОГИИ, КРИМИНАЛИСТИКИ И СУДЕБНОЙ ЭКСПЕРТИЗЫ, 2021, № 1(49). С. 105-111
5. Кривошапова Н.В. Атрибуция анонимного текста: оригинальная методика установления авторства // ВЕСТНИК ПРИДНЕСТРОВСКОГО УНИВЕРСИТЕТА, 2022, №1(70). С. 9-15
6. Косимова Н.О. Обзор методов и этапов решения задач автоматической обработки текстов и атрибуции авторства // УСТОЙЧИВОЕ РАЗВИТИЕ НАЦИОНАЛЬНОЙ ПРОМЫШЛЕННОСТИ НА ОСНОВЕ РЕАЛИЗАЦИИ «ДВАДЦАТИЛЕТИЯ ИЗУЧЕНИЯ И РАЗВИТИЯ ЕСТЕСТВЕННЫХ, ТОЧНЫХ И МАТЕМАТИЧЕСКИХ ДИСЦИПЛИН В СФЕРЕ НАУКИ И ОБРАЗОВАНИЯ». Душанбе - 2023. Часть 1. С. 121-125

7. Dalyan T., Ayral H., Özdemir Ö. A comprehensive study of learning approaches for author gender identification // Information Technology and Control, 2022, Vol. 51 No.3. P. 429-445
8. Iqbal M. M., Raza A., Aslam M. M., Farhan M., Yaseen S. A stylometric fingerprinting method for author identification using machine learning // Technical Journal, University of Engineering and Technology (UET) Taxila, Pakistan Vol. 28 No. 1-2023 P. 28-35
9. Syamsiah N., Myint A., Lin T. Automatic identification of authors' writing style with computerbased stylometry methods: a case study on indonesian literary texts // Journal International of Lingua and Technology, 3(3) - Dec 2024. C. 659-672
10. Khonji M., Iraqi Y., Mekouar L. Authorship identification of electronic texts // IEEE Access Vol. 9, 2021. P. 101124-101146
11. Кузина А. М. Влияние лингвистических признаков на определение пола и возраста автора текста // Молодой ученый. - 2025. - № 38 (589). - С. 1-5
12. Patel N. How Natural Language Processing Affects Digital Marketing - [Электронный ресурс]. Режим доступа: <https://neilpatel.com/blog/natural-language-processing/> (дата обращения 18.10.2025)
13. Котенко И.В., Абраменко Г.Т. Использование больших языковых моделей для поиска угроз кибербезопасности на основе методов глубокого обучения: анализ современных исследований // Правовая информатика - 2024. - №3. - С. 32-41
14. Schler J., Koppel M., Argamon S., Pennebaker J.W. Effects of Age and Gender on Blogging // Computational Approaches to Analyzing Weblogs: Papers from the 2006 AAAI Spring Symposium. - Stanford, CA, USA, 2006. - P. 27-29.
15. Rao D., Yarowsky D., Shreevats A., Gupta M. Classifying latent user attributes in twitter // SMUC '10: Proceedings of the 2nd international workshop on Search and mining user-generated contents. 2010. P. 37-44.
16. Koppel. M. Automatically Categorizing Written Texts by Author Gender. // Literary and Linguistic Computing - 2002, 17(4), P. 401-412

17. Argamon S., Koppel M., Pennebaker J.W. Mining the Blogosphere: Age, gender and the varieties of self-expression [Электронный ресурс]. Режим доступа: <https://firstmonday.org/ojs/index.php/fm/article/view/2003> (дата обращения 18.10.2025)
18. Литвинова Т.А. Проблема диагностирования пола автора письменного текста: Фактор жанра. // Современные исследования социальных проблем (электронный научный журнал), №1(33), 2014.
19. Venkatesh G., Sampath Kumar Dr. M. Age groups classification on twitter dataset using flann algorithm. // JETIR June 2019, Vol.6, Issue 6. P. 651-659
20. Frank E., Bouckaert R. Naive Bayes for Text Classification with Unbalanced Classes. // Lecture Notes in Computer Science Vol. 4213, P. 503-510
21. Kucukyilmaz T., Cambazoglu B. B., Can F., Aykanat C. Chat mining: predicting user and message attributes in computer-mediated communication // Information Processing & Management Volume 44, Issue 4, 2008, Pages 1448-1466
22. Kairupan I.Y., Angdresey A., Arif H., Emor K. G. An Extreme Gradient Boosting Approach for Classification and Sentiment Analysis // The Asian Journal of Technology Management (AJTM), 2023, 16(3), P. 211-225
23. Сбоев А.Г. Нейросетевое моделирование и машинное обучение на основе экспериментальных и наблюдательных данных. [Электронный ресурс]. Режим доступа: https://ds.mephi.ru/documents/683/%D0%A1%D0%B1%D0%BE%D0%B5%D0%B2_%D0%90_%D0%93_%D0%A2%D0%B5%D0%BA%D1%81%D1%82_%D0%B4%D0%B8%D1%81%D1%81%D0%B5%D1%80%D1%82%D0%B0%D1%86%D0%B8%D0%B8.pdf (дата обращения 18.10.2025)
24. Ермакович С.П. Гендерные исследования в российском языкознании // Слово.ру: Балтийский акцент. 2015. С. 33-38
25. Mikros G.K. Systematic stylometric differences in men and women authors: a corpus-based study // Issues in Quantitative Linguistics 3, RAM-Verlag, 2013. P. 206-224

26. Литвинова Т.А., Литвинова О.А., Рыжкова Е. С., Бирюкова Е. Д., Середин П. В., Загоровская О.В. Исследование влияния пола и психологических характеристик автора на количественные параметры его текста с использованием программы Linguistic Inquiry and Word Count // Научный диалог. - 2015. - № 12 (48). - С. 101-109.
27. Балакина Л.В. Проявление гендерного фактора в художественном тексте. [Электронный ресурс]. Режим доступа: <https://dspace.kpfu.ru/xmlui/bitstream/handle/net/153489/0-755992.pdf> (дата обращения 18.10.2025)
28. Litvinova T., Seredin P., Litvinova O., Zagorovskaya O. Differences in type-token ratio and part-of-speech frequencies in male and female Russian written texts // Proceedings of the Workshop on Stylistic Variation, 2017, P. 69-73
29. Диброва Е.В. Идентификация гендерной принадлежности автора по лингвистическим особенностям продуцируемого им текста // МЕЖДУНАРОДНЫЙ НАУЧНО-ИССЛЕДОВАТЕЛЬСКИЙ ЖУРНАЛ, 2013, №11 (18) Часть 2, С. 102-103
30. Scikit-learn 1.7 documentation - [Электронный ресурс]. Режим доступа: <https://scikit-learn.org> (дата обращения 18.10.2025)
31. Литвинова Т.А. Стилеметрическая идентификация автора текста - [Электронный ресурс]. Режим доступа: https://elibrary.ru/download/elibrary_49842609_64435555.pdf (дата обращения 18.10.2025)

ПРИЛОЖЕНИЕ А. КОНТРОЛЬНЫЙ ПРИМЕР

Вопрос про студию. Слушайте, друзья, я вот думаю отказываться ли от студии или нет. Сейчас я не снимаю и когда начну не знаю, школа то есть, то нет, поэтому бывают месяцы когда аренду приходится из семейного бюджета платить. Поэтому вот думаю что делать, держать и платить до лучших времен или отказаться и снять потом, когда будет возможность и работать и платить. Что останавливает отказаться. Нынешняя студия удобна тем, что есть и для лекций помещение и для студии (раз). Центр города всё-таки (два). Фиг я потом найду квартиру, которую хозяин согласился бы сдавать чётко под деятельность. В торговых центрах в разы дороже, ведь мне нужно много места, я там никогда не окуплю. Это не раньше как было, для фотостудий был пониженный коэффициент оплаты аренды (в кбо например). Была мысль купить в одном из строящихся ТЦ помещение, но думаю, что продавать там никто не будет, такие магазины строятся для сдачи в аренду... Сейчас вот читаю - «На заседании думы 9 декабря было принято решение продавать методом публичного предложения 4 подлежащих приватизации объекта муниципального имущества». В том числе магазин «Нагорный», вот думаю может его прикупить? :)) Без шуток :) Интересно, за сколько будут продавать? Вот в таком он сейчас состоянии. Ужас-ужас, конечно :(((У кого какие мысли? UPD: Вот и условия нашла. <http://sarov.nnov.ru/pictures/file/2010/117.doc>

2. Условия приватизации муниципального имущества:

2.1. Способ приватизации муниципального имущества - продажа муниципального имущества - зданий магазина «Нагорный» и склада, посредством публичного предложения.

2.2. Цена первоначального предложения составляет 10 810 000 (Десять миллионов восемьсот десять тысяч) рублей.

2.3. Цена отсечения - 5 405 000 (Пять миллионов четыреста пять тысяч) руб.

2.4. Шаг понижения - 270 250 (Двести семьдесят тысяч двести пятьдесят) руб.

2.5. Шаг аукциона - 100 000 (Сто тысяч) руб.

2.6. Порядок и срок оплаты имущества - единовременный платеж в течение тридцати дней с момента подписания договора купли-продажи муниципального имущества.

ПРИЛОЖЕНИЕ Б. ИЗВЛЕКАЕМЫЕ ПРИЗНАКИ

Для унификации вычислений введены общие обозначения, используемые во всех формулах агрегации.

Общие обозначения:

- $C(x)$ - количество элементов последовательности x
- $S(d)$ - количество предложений в документе d
- $N(d)$ - количество словоформ в документе d
- $V(d)$ - множество словоформ в документе d
- $ENTITIES$ - множество сущностей
- $E_t(d)$ - количество сущностей типа t в документе d
- POS - множество частей речи
- $POS(d)$ - последовательность частей речи
- $POS_t(d)$ - количество частей речи типа t в документе d
- $FORM$ - множество формальных частей речи
- $INFORM$ - множество неформальных частей речи
- $PUNCT$ - множество знаков пунктуации
- $PUNCT_t(d)$ - количество частей речи типа t в документе d
- $TOP4000$ - множество популярных слов
- $OBSCENE$ - множество нецензурных слов
- $freq(v, d)$ - количество употребления словоформы v в документе d

Имя признака	Категория	Формула	Описание
symbols_length	text	$\frac{C(d)}{S(d)}$	Количество символов на предложение
long_sentences_proportion	text	$\frac{ \{s: C(s) > 8\} }{S(d)}$	Доля предложений длиннее 8 слов
lexicon_size	text	$\frac{ V(d) }{S(d)}$	Мощность словаря словоформ в среднем на предложение.
lexical_diversity_per_sentence	text	$\frac{ \{v \in V(d) : \text{freq}(v, d) = 1\} }{S(d)}$	LDPS - Лексическая насыщенность по предложениям
type_token_ratio	text	$\frac{ \{v \in V(d) : \text{freq}(v, d) = 1\} }{ V(d) }$	TTR - мера лексического разнообразия.
mean_words_length	text	$\frac{\sum_{v \in V(d)} C(v) \cdot \text{freq}(v, d)}{N(d)}$	Средняя длина словоформы в документе.
long_word_proportion	text	$\frac{\sum_{v \in V(d) : C(v) > 6} \text{freq}(v, d)}{N(d)}$	Доля слов длиннее 6 символов.

Имя признака	Категория	Формула	Описание
normalized_shannon_entropy	text	$\frac{-\sum_{v \in V} \frac{freq(v,d)}{N(d)} \log_2 \frac{freq(v,d)}{N(d)}}{\log_2(freq_{\max})}$	Нормализованная энтропия Шеннона
unpopular_words_proportion	text	$\frac{\sum_{v \in V(d), \notin \text{TOP4000}} freq(v,d)}{N(d)}$	Доля слов вне топ-4000 корпусного словаря
obsene_proportion	text	$\frac{\sum_{v \in V(d), \in \text{Obscene}} freq(v,d)}{N(d)}$	Доля нецензурных слов
mean_usage_entity	entities	$\frac{E_t(d)}{S(d)}$	Среднее количество сущностей типа t на предложение
breadth_of_use_entities	entities	$\frac{ \{t \in ENTITIES: E_t(d) > 0\} }{ ENTITIES }$	Широта использования типов сущностей
breadth_of_use_pos	pos	$\frac{ \{t \in POS: POS_t(d) > 0\} }{ POS }$	Широта использования частей речи
index_of_formality	pos	$\max \left(0, \min \left(100, \frac{\left(\sum_{t \in FORM} POS_t(d) \right) - \left(\sum_{t \in INFORM} POS_t(d) \right) + 100}{2} \right) \right)$	Индекс формальности текста
index_of_formality_tuldava	pos	$\min \left(\frac{NOUN + ADJ}{VERB + ADV + PRON} * 100, 100 \right)$	Индекс формальности

Имя признака	Категория	Формула	Описание
			Тулдава
mean_usage_pos	pos	$\frac{POS_t(d)}{S(d)}$	Среднее количество частей речи типа t на предложение
pair_of_adv_per_sentence	pos	$\frac{ \{i \in [0, POS(d)] : POS(i) = ADV \wedge POS(i+1) = ADV\} }{S(d)}$	Количество пар соседних наречий (ADV-ADV) на предложение
bigram_A_B	pos	$\frac{ \{i \in [1, POS(d)] : (p_i, p_{i+1}) = (A, B)\} }{ POS(d) - 1}$	Доля биграммы А-В среди всех биграмм
mean_usage_punct	punct	$\frac{PUNCT_t(d)}{S(d)}$	Среднее количество знаков пунктуации типа t на предложение
breadth_of_use_puncts	punct	$\frac{ \{t \in PUNCT : PUNCT_t(d) > 0\} }{ PUNCT }$	Широта использования типов знаков пунктуации

ПРИЛОЖЕНИЕ В. ИСХОДНЫЙ КОД

```
##### clean.py #####
_B='content'
_A=True
from pathlib import Path
from text_utils.cleaners import clean_html
from text_utils.converters import to_single_line
from text_utils.checkers import has_unbalanced_quotes
import pandas as pd
from urlextract import URLExtract
DATA_PATH=Path('./processed/rus_idiolect.parquet')
OUTPUT_DIR=Path('./processed')
OUTPUT_FILE=Path('rus_idiolect_clean.parquet')
OUTPUT_PATH=OUTPUT_DIR/OUTPUT_FILE
DROP_COLUMNS=['Текст: Интенция на обман','Автор:
Рукость','Автор: Родной язык','Текст: Тип текста','Корпус:
Название','Текст: Модус','Текст: Форма речи','Текст: Тип
коммуникативной ситуации']
RENAME_MAP={'Текст: Возраст автора':'age','Текст:
Текст: _B','Автор: Номер участника':'user_id','Автор: Пол':'gender'}
SOURCE_URL_PATTERNS={'livejournal','vk.com/wall','vkontakte.ru/no
te','zen.yandex.ru/media/'}
def extract_source_url(text):

F='.,,:!?)} ...';B=None;A=text;G=URLExtract();C=G.find_urls(A.get_ind
ices=_A)
    if not C:return A,B
    D,(H,I)=C[-1]
    if not any(A in D for A in SOURCE_URL_PATTERNS):return A,B
    E=A[I:].strip()
    if E and not all(A in F for A in E):return A,B
    J=A[:H].rstrip(F);return J,D
def main():
    B='text'
    if not DATA_PATH.exists():raise FileNotFoundError(f'Data file not
found: {DATA_PATH}')

A=pd.read_parquet(DATA_PATH);A.drop(columns=DROP_COLUMNS,
inplace=_A);A.rename(columns=RENAME_MAP,inplace=_A);A.dropna(
inplace=_A);A[['source_url']] =A[_B].apply(extract_source_url).tolist();
A.drop(columns=[_B],inplace=_A);A[B]=A[B].apply(clean_html);A[B]=
A[B].apply(to_single_line);C=A[B].apply(has_unbalanced_quotes);A=A.l
oc[~C].copy();A=A.loc[A[B].str.strip().astype(bool)].copy();OUTPUT_DI
R.mkdir(parents=_A,exist_ok=_A);A.to_parquet(OUTPUT_PATH,index=ils
False)
if __name__=='__main__':main()

##### merge_chunks.py #####
_A=True
from pathlib import Path
import pandas as pd
OUTPUT_DIR=Path('./processed')
OUTPUT_FILE='rus_idiolect.parquet'
OUTPUT_PATH=OUTPUT_DIR/OUTPUT_FILE
CHUNKS_DIR=Path('./raw')
CHUNK_PREFIX='chunk_'
if not CHUNKS_DIR.exists():raise FileNotFoundError(f'Директория
{CHUNKS_DIR} не найдена')
chunks=sorted([A for A in CHUNKS_DIR.iterdir()if A.suffix=='.xlsx'and
A.name.startswith(CHUNK_PREFIX)])
if not chunks:raise FileNotFoundError(f'Файлы с префиксом
'{CHUNK_PREFIX}' не найдены')
dataframes=[]
for chunk_file in chunks:
    try:df_chunk=pd.read_excel(chunk_file);dataframes.append(df_chunk)
    except Exception as e:print(f'Ошибка при чтении {chunk_file}:
{e}');continue
if not dataframes:raise ValueError('Не удалось загрузить ни один файл')
df=pd.concat(dataframes,ignore_index=_A)
df.drop_duplicates(inplace=_A)
OUTPUT_DIR.mkdir(parents=_A,exist_ok=_A)
df.to_parquet(OUTPUT_PATH,index=False)

##### app.py #####
from fastapi import FastAPI,APIRouter
from fastapi.middleware.cors import CORSMiddleware
from modules.profiling.api.v1.routes import router as profiling_v1
from core.error_handlers import setup_error_handlers
rootRouter=APIRouter(prefix='/api')
rootRouter.include_router(profiling_v1)
app=FastAPI()
setup_error_handlers(app)
app.add_middleware(CORSMiddleware,allow_origins=['*'],allow_credent
ials=True,allow_methods=['*'],allow_headers=['*'])
app.include_router(rootRouter)

##### config.py #####
from typing import Dict
import yaml
with open('config.yaml')as f:cfg=yaml.safe_load(f)
def resolve_config():return cfg
class Model:
    def __init__(A,id,name,path):A.id=id;A.name=name;A.path=path
class ProfilingSection:
    models:Dict[str,Model]
    def __init__(A,models):A.models=models
    @staticmethod
    def
from_yaml(cfg):A=config.get('profiling',{});B={A['id']:Model(**A)for
A in A.get('models',[])};return ProfilingSection(models=B)

##### error_handlers.py #####
from fastapi import FastAPI,Request,status
from fastapi.responses import JSONResponse
from fastapi.exceptions import RequestValidationError
from exceptions import AppError
from schemas import ErrorResponse
import logging
logger=logging.getLogger(__name__)
def setup_error_handlers(app):
    D='details';C='message';B='code';A=app
    @A.exception_handler(AppError)
    async def E(request,exc):A=exc;logger.warning(f'App error
{A.message}');E=ErrorResponse(error={B:A.code,C:A.message,D:A.data
validation_info},path=request.url.path);return
JSONResponse(status_code=A.status_code,content=E.model_dump())
    @A.exception_handler(RequestValidationError)
    async def F(request,exc):
        logger.info(f'Validation error: {exc.errors()}');E=[]
        for A in exc.errors():E.append({'field': '.'.join(str(A)for A in
A['loc']),C:A['msg'],B:A['type']})
        F=ErrorResponse(error={B:'request_validation_error',C:'Ошибка
валидации запроса',D:{'errors':E}},path=request.url.path);return
JSONResponse(status_code=status.HTTP_422_UNPROCESSABLE_EN
TITY,content=F.model_dump())
    @A.exception_handler(Exception)
    async def G(request,exc):logger.error(f'Unexpected error:
{str(exc)}",exc_info=True);E=ErrorResponse(error={B:'internal_server_er
ror',C:'Внутренняя ошибка сервера',D:{'debug':str(exc)if A.debug else
None}},path=request.url.path);return
JSONResponse(status_code=status.HTTP_500_INTERNAL_SERVER_E
RROR,content=E.model_dump())

##### exceptions.py #####
_A=None
from fastapi import status
class AppError(Exception):
    def
    __init__(A,message,code,status_code,errors=_A):B=message;super().__ini
t(_B);A.message=B;A.code=code;A.status_code=status_code;A.details={
'errors':errors}
def app_error(message,errors=_A):return
AppError(message=message,code='internal_error',status_code=status.HTTP
500_INTERNAL_SERVER_ERROR,errors=errors)
```

```

def business_error(message,errors=_A):return
AppError(message=message,code='business_error',status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,errors=errors)
def validation_error(message,errors=_A):return
AppError(message=message,code='validation_error',status_code=status.HTTP_412_PRECONDITION_FAILED,errors=errors)
def not_found_error(message,errors=_A):return
AppError(message=message,code='not_found',status_code=status.HTTP_404_NOT_FOUND,errors=errors)

##### schemas.py #####
from pydantic import BaseModel,Field,field_serializer
from typing import Optional,Dict,Any
from datetime import datetime
class ErrorResponse(BaseModel):

success:bool=False;error:Dict[str,Any]=Field(default_factory=dict,description='Детали ошибки');timestamp:datetime=Field(default_factory=datetime.now);path:Optional[str]=None
    @field_serializer('timestamp')
    def serialize_datetime(self,dt):return dt.strftime('%d.%m.%Y %H:%M')

##### di.py #####
from fastapi import Depends
from modules.profiling.use_cases.predict_profile import PredictProfileUseCase
from modules.profiling.infrastructure.config.profiling_models import get_model_info
from modules.profiling.infrastructure.memory_cache.text import async_get_metrics
from modules.profiling.infrastructure.ai_models.profiling import get_profile
from core.config import resolve_config
from functools import partial
def
resolve_predict_profile_use_case(config=Depends(resolve_config)):A=partial(get_model_info,config);B=get_profile;return
PredictProfileUseCase(async_get_metrics,B,A)

##### extract_text_metrics.py #####
class ExtractTextMetricsUseCase:
    def __init__(A,extract_metrics):A.extract_metrics=extract_metrics
    async def execute(A,text):B=await A.extract_metrics(text);return B

##### list_available_models.py #####
from fastapi import APIRouter,Depends
from pydantic import BaseModel
from typing import List
from modules.profiling.use_cases.get_available_models import GetAvailableModelsUseCase
from di import resolve_get_available_models_use_case
class Request(BaseModel):0
class ProfilingModel(BaseModel):id:str;name:str
class Response(BaseModel):available_models:List[ProfilingModel]
async def
get_available_models(data=Depends(),use_case=Depends(resolve_get_available_models_use_case)):A=await
use_case.execute();return {'available_models':A}
def register(router):router.get('/available-models',response_model=Response)(get_available_models)

##### predict.py #####
from fastapi import APIRouter,Depends
from pydantic import BaseModel,Field,field_validator
from typing import Dict,Optional
from core.schemas import ErrorResponse
from modules.profiling.use_cases.predict_profile import PredictProfileUseCase
from di import resolve_predict_profile_use_case
class Request(BaseModel):

text:str=Field(...,min_length=300,max_length=10000,description='Текст');##### get_available_models.py #####
model_id:Optional[str]=Field(None,description='Идентификатор модели прогнозирования')
    @field_validator('text')
    def validate_quotes(value):
        A=value;B=A.count('"');C=A.count("'")
        if C%2!=0 or B%2!=0:raise ValueError('Количество кавычек должно быть четным')
        return A
class Prediction(BaseModel):proba:float;threshold:float;label:str
class
Response(BaseModel):gender_prediction:Prediction;age_prediction:Prediction;metrics:Dict[str,Dict[str,float]];model_id:str
async def
predict(data,use_case=Depends(resolve_predict_profile_use_case)):A=await
use_case.execute(data.text,data.model_id);return {'metrics':A.metrics,'gender_prediction':A.gender,'age_prediction':A.age,'model_id':A.model_id}
def
register(router):B='description';A='model';router.post('/predict',response_model=Response,responses={404:{A:ErrorResponse,B:'Модель не найдена'},412:{A:ErrorResponse,B:'Извлеченные метрики невалидны для прогнозирования'},500:{A:ErrorResponse,B:'Внутренняя ошибка приложения'}})(predict)

##### routes.py #####
from fastapi import APIRouter
from list_available_models.list_available_models import register as reg_list_of_available_models
from extract_text_metrics.extract_text_metrics import register as reg_extract_text_metrics
from predict.predict import register as reg_predict_profile
router=APIRouter(prefix='/v1/profiling')
reg_list_of_available_models(router)
reg_extract_text_metrics(router)
reg_predict_profile(router)

##### text.py #####
from cachetools import cached,TTLCache
from hashlib import sha256
from...domain.text import extract_metrics
text_cache=TTLCache(maxsize=1000,ttl=600)
@cached(text_cache,key=lambda text:sha256(text.encode()).hexdigest())
def cached_get_metrics(text):return extract_metrics(text)
async def async_get_metrics(text):return cached_get_metrics(text)

##### profiling.py #####
from predictors.serialization import load_model
from pathlib import Path
from cachetools import cached
from core.exceptions import app_error
@cached(cache={})
def load_predictor(file_name):
    B=file_name;A=Path(B)
    if not A.exists():raise app_error(message='Модель не найдена',errors=[f'Модель по пути {B} не найдена'])
    if A.suffix!='.pkl':raise app_error(message='Недопустимый формат модели',errors=[f'Модель имеет недопустимый формат - {A.suffix}'])
    C=load_model(A);return C
async def
get_profile(metrics,model_info):A=load_predictor(model_info.path);B=A.predict([metrics]);return B[0]

##### profiling_models.py #####
from core.config import ProfilingSection
from core.exceptions import app_error,not_found_error
async def
get_models_info(config):A=ProfilingSection.from_yaml(config);return A.models.values()
async def get_model_info(config,model_id=None):
    A=model_id;C=ProfilingSection.from_yaml(config);B=C.models
    if not B:raise app_error(message='Отсутствуют модели для прогнозирования')
    if A and A not in B:raise not_found_error(message='Указана несуществующая модель',errors=[f'Модель {A} не доступна для прогнозирования'])
    return B[A]if A else list(B.values())[0]

##### predict_profile.py #####

```

```

from..domain.text import validate_metrics
from core.exceptions import validation_error
class PredictionResult:
    def
__init__(A,gender,age,metrics,model_id):A.gender=gender;A.age=age;A.
metrics=metrics;A.model_id=model_id
class PredictProfileUseCase:
    def
__init__(A,extract_metrics,get_profile,get_model_info):A.extract_metrics
=extract_metrics;A.get_profile=get_profile;A.get_model_info=get_model
_info
    async def execute(A,text,model_id=None):
        C=await A.get_model_info(model_id);B=await
A.extract_metrics(text);D=validate_metrics(B)
        if D:raise validation_error(message='Извлеченные метрики
невалидны для прогнозирования',errors=D)
        E=await A.get_profile(B,C);return
PredictionResult(metrics=B,gender=E[0],age=E[1],model_id=C.id)

##### classifier.py #####
from abc import ABC,abstractmethod
import pandas as pd,numpy as np
class Classifier(ABC):
    def __init__(A,pipeline):A.pipeline=pipeline
    @abstractmethod
    def predict(self,features):raise NotImplementedError

##### multi.py #####
from.classifier import Classifier
import numpy as np
class CascadeClassifier(Classifier):
    def __init__(A,pipeline):super().__init__(pipeline)
    def predict(A,features):B=A.pipeline.predict_proba(features);return
B[:,1::2]
class FlatClassifier(Classifier):
    def __init__(A,pipeline):super().__init__(pipeline)
    def predict(G,features):

A=G.pipeline.predict_proba(features);C=A.shape[1];B=int(np.log2(C));D
=np.zeros((A.shape[0],B))
    for E in range(B):
        for F in range(C):
            if F>>B-1-E&1:D[:,E]+=A[:,F]
        return D

##### predictor.py #####
from dataclasses import dataclass
from.classifier import Classifier
@dataclass
class ClassPrediction:proba:float;threshold:float;label:str
class Predictor:
    def
__init__(A,classifier,thresholds,labels):A.classifier=classifier;A.thresholds
=thresholds;A.labels=labels
    def predict(A,features):
        H=A.classifier.predict(features);B=[]
        for C in H:
            D=[]
            for(E,(F,G))in
enumerate(zip(A.thresholds,A.labels)):I=ClassPrediction(proba=C[E].item
(),threshold=F,label=G[0]if C[E]<F else G[1]);D.append(I)
                B.append(D)
        return B

##### feature_selection.py #####
from sklearn.base import BaseEstimator,TransformerMixin
class ModelFeatureSelector(BaseEstimator,TransformerMixin):
    def __init__(A,keys):A.keys=keys
    def fit(A,X,y=None):return A
    def transform(A,X):return X[A.keys]

##### metrics.py #####
import numpy as np
epsilon=1e-08
def triangular_weights(distances,max_distance=None):
    B=distances;A=max_distance
    if A is None:A=np.max(B)
    if A<epsilon:return np.ones_like(B)
    return np.maximum(1-B/A,0)

```

```

def gaussian_weights(distances,sigma=1.):return np.exp(-
distances**2/(2*sigma**2))
def quadratic_weights(distances):return 1/(distances**2+epsilon)

##### pipeline.py #####
import numpy as np
from sklearn.base import BaseEstimator,ClassifierMixin
class CascadedClassifier(BaseEstimator,ClassifierMixin):
    def
__init__(A,main_clf,secondary_clf,threshold=.5):A.main_clf=main_clf;A.
secondary_clf=secondary_clf;A.threshold=threshold
    def fit(A,X,y):raise NotImplementedError('Модель уже обучена, fit не
требуется.')
    def predict(B,X):
        C=B.main_clf.predict(X);D=np.empty(X.shape[0],dtype=int)
        for(E,F)in enumerate(B.secondary_clf):
            A=C==E
            if np.any(A):D[A]=F.predict(X[A])
        return np.column_stack([C,D])
    def predict_proba(A,X):

C=A.main_clf.predict_proba(X);E=(C[:,1]>=A.threshold).astype(int);D=np.
p.zeros(X.shape[0],2))
        for(F,G)in enumerate(A.secondary_clf):
            B=E==F
            if np.any(B):D[B]=G.predict_proba(X[B])
        return np.hstack([C,D])

##### serialization.py #####
import json,pickle
def write_model(path,model):
    with open(path.with_suffix('.pkl'),'wb')as A:pickle.dump(model,A)
def write_dict(path,data):
    with open(path.with_suffix('.json'),'w')as A:json.dump(data,A)
def
save_model(base_dir,model,name,model_params=None,scores=None):
    D=scores;C=model_params;B=base_dir
    if not B.exists():return FileNotFoundError('Папка с моделями не
найдена')
    A=B/name;A.mkdir();write_model(A/'model',model)
    if C:write_dict(A/'params',C)
    if D:write_dict(A/'scores',D)
def load_model(path):
    with open(path,'rb')as A:return pickle.load(A)

##### transformers.py #####
_A=None
from typing import List,Optional
import pandas as pd
from functools import reduce
class CompositionTransformer:
    def __init__(A,transformers):A.transformers=transformers
    def transform(A,X):return reduce(lambda
state,transformer:transformer.transform(state),A.transformers,X)
class ApplyListTransformer:
    def __init__(A,fn):A.fn=fn
    def transform(B,X):
        A=[]
        for C in X:A.append(B.fn(C))
        return A
class DataFrameTransformer:
    def __init__(A,columns):A.columns=columns
    def transform(A,X):return pd.DataFrame(X,columns=A.columns)
class DefaultValueTransformer:
    def __init__(A,default_val):A.default_val=default_val
    def transform(A,X):return X.fillna(A.default_val)
class RenameColumnTransformer:
    def __init__(A,mapper):A.mapper=mapper
    def transform(A,X):return X.rename(columns=A.mapper)
class BinarizationRule:
    def
__init__(A,current_name,new_name,between=_A,gt_then=_A,lt_then=_A
):A.current_name=current_name;A.new_name=new_name;A.between=bet
ween;A.gt_then=gt_then;A.lt_then=lt_then
    def apply(A,series):
        B=series
        if A.between is not _A:return B.between(A.between[0],A.between[1])
        elif A.gt_then is not _A:return B>A.gt_then
        elif A.lt_then is not _A:return B<A.lt_then

```

```

        return B
class BinarizationTransformer:
    def __init__(A, rules): A.rules = rules
    def transform(B, X):
        for A in B.rules: X[A.new_name] = A.apply(X[A.current_name]).astype(int)
        return X
class DropColumnTransformer:
    def __init__(A, columns): A.columns = columns
    def transform(A, X): return X.drop(columns=A.columns)
class ModelWrapper:
    def __init__(A, transformer, model): A._transformer = transformer; A._model = model
    def predict(A, X): B = A._transformer.transform(X); return A._model.predict(B)
    def predict_proba(A, X): B = A._transformer.transform(X); return A._model.predict_proba(B)

##### context.py #####
from typing import Dict, List
from collections import Counter
from span import Span
class DocumentStat:
    symbols_length: int
    def __init__(A, symbols_length): A.symbols_length = symbols_length
class EntitiesStat:
    spans: Dict[str, List[Span]]; entities_width: int
    def __init__(A, spans, entities_width): A.spans = spans; A.entities_width = entities_width
class PosStat:
    seqs: List[List[Span]]; sentences_count: int
    def __init__(A, seqs): A.seqs = seqs; A.sentences_count = len(seqs)
    def get_series(C, seq_start=None, seq_end=None):
        B = seq_end; A = seq_start
        for D in C.seqs:
            if A: yield A
            yield from D
            if B: yield B
class DocumentContext:

lexicon: Counter; document_stat: DocumentStat; entities: EntitiesStat; pos: PosStat
    def __init__(A, lexicon, document_stat, entities, pos): A.lexicon = lexicon; A.document_stat = document_stat; A.entities = entities; A.pos = pos

##### metric.py #####
from typing import Callable, Dict, Union, Optional, Protocol
from context import DocumentContext
class Metric(Protocol):

name: Optional[str]; group: Optional[str]; extract_func: Callable[[DocumentContext], Union[Dict, float]]
    def __init__(A, name, group, extract_func): A.name = name; A.group = group; A.extract_func = extract_func

##### span.py #####
from dataclasses import dataclass
from intervaltree import IntervalTree
from typing import List
@dataclass
class Span: start: int; end: int; data: object
class SpanTree:
    def __init__(A): A._tree = IntervalTree()
    def try_add(A, start, end, data):
        B = start; C = A.overlap(B, end)
        if C: return False
        A._tree.addi(B, end, data); return True
    def overlap(A, start, end): return A._tree.overlap(start, end)
    def get_spans(A): return [Span(A, B, C) for (A, B, C) in A._tree]

##### tagger.py #####
from typing import Callable, List
from span import Span
class Tagger:
    name: str; tag_func: Callable[[str], List[Span]]

```

```

    def __init__(A, name, tag_func): A.name = name; A.tag_func = tag_func

##### doc.py #####
from collections import Counter, defaultdict
from typing import List
from ..external.natasha import create_doc
from ..entities.EntityTagger import EntityTagger
from ..core.tagger import Tagger
from ..core.metric import Metric
from ..core.context import DocumentContext, DocumentStat, EntitiesStat, PosStat
from ..pos.pos_tagger import tag as pos_tag
from ..MetricExtractor import MetricExtractor
def extract_metrics(text, taggers, metrics):

C = taggers; A = text; B = EntityTagger(C); B.tag(A); F = B.get_entities(); D = defaultdict(list)
    for E in F: D[E.data['entity']].append(E)
    G = len(set([A.name for A in C])); H = EntitiesStat(D, G); I = create_doc(A); J, K = pos_tag(I, B); L = PosStat(J);
    M = Counter(K); N = DocumentStat(len(A)); O = DocumentContext(M, N, H, L); P = MetricExtractor(metrics); Q = P.extract(O); return Q

##### MetricExtractor.py #####
from ..core.context import DocumentContext
from ..core.metric import Metric
from collections import defaultdict
DEFAULT_GROUP = 'common'
class MetricExtractor:
    def __init__(A, metrics): A._metrics = metrics
    def extract(E, context):
        C = defaultdict(dict)
        for A in E._metrics:
            B = A.extract_func(context); F = A.group or DEFAULT_GROUP; D = C[F]
            if isinstance(B, dict): D.update(B)
            else: D[A.name] = B
        return C

##### long_sentences_proportion.py #####
from text_metrics.core.context import DocumentContext
LONG_SENTENCE_LENGTH = 8
METRIC_NAME = 'long_sentences_proportion'
METRIC_GROUP = 'text'
def extract(context):
    A = context; B = A.pos.sentences_count
    if B == 0: return 0
    C = sum(1 for A in A.pos.seqs if len(A) >= LONG_SENTENCE_LENGTH); return C/B

##### sentences_count.py #####
from text_metrics.core.context import DocumentContext
METRIC_NAME = 'sentences_count'
METRIC_GROUP = 'text'
def extract(context): return context.pos.sentences_count

##### symbols_length.py #####
from text_metrics.core.context import DocumentContext
METRIC_NAME = 'symbols_length'
METRIC_GROUP = 'text'
def extract(context): A = context; return A.document_stat.symbols_length/A.pos.sentences_count

##### regex_tag.py #####
import re
from ..core.span import Span
def regex_tagger_factory(pattern):
    A = re.compile(pattern, re.IGNORECASE)
    def B(text): B = A.finditer(text); return [Span(A.start(), A.end(), {'match': A.group(0)}) for A in B]
    return B

##### EntityTagger.py #####
from typing import List
from ..core.span import SpanTree, Span
from text_metrics.core.tagger import Tagger
class EntityTagger:
    def __init__(A, taggers): A._taggers = taggers; A._tree = None

```

```
def is_contains(A,start,end):
    if not A._tree:return False
    return A._tree.overlap(start,end)
def get_entities(A):
    if not A._tree:return[]
    return[Span(A.start,A.end,A.data)for A in A._tree.get_spans()]
def tag(A,text):
    A._tree=SpanTree()
    for C in A._taggers:
        D=C.name;E=C.tag_func(text)
        for B in E:F=A._tree.try_add(B.start,B.end,{‘entity’:D}|B.data)

##### breadth.py #####
from text_metrics.core.context import DocumentContext
from text_metrics.punct.consts import set_of_puncts
METRIC_NAME='breadth_of_use_puncts'
METRIC_GROUP='punct'
def extract(context):A=context.pos.get_series();B={A.data['text']for A in A.if
A.data['pos']==‘PUNCT’};C=B.intersection(set_of_puncts);D=len(C)/len(s
et_of_puncts);return D

##### mean_usage.py #####
from collections import Counter
from text_metrics.core.context import DocumentContext
from text_metrics.punct.consts import set_of_puncts
METRIC_GROUP='punct'
def
extract(context):A=context;C=A.pos.sentences_count;D=A.pos.get_series(
);B=Counter([A.data['text']for A in D if
A.data['pos']==‘PUNCT’]);E=B.keys()&set_of_puncts;return {f‘mean_usag
e_{A}’:B[A]/C for A in E}

##### TaggerLoader.py #####
_B=‘_init_’
_A=None
import importlib
from pathlib import Path
from text_metrics.core.tagger import Tagger
from typing import List,Optional,Set
TAGGERS_DIR_NAME='taggers'
TAG_NAME_ATTR='TAG_NAME'
TAG_FN_ATTR='tag'
class TaggerLoader:
    def __init__(A,base_path=_A):A.base_path=base_path or
Path(_file_).parent;A.taggers_dir=(A.base_path/TAGGERS_DIR_NAM
E).resolve()
    def __load_from_file(G,tagger_path):
        A=tagger_path
        if not A.exists():return
        D=A.stem
        if D==_B or not A.suffix==''.py':return
        B=importlib.util.spec_from_file_location(D,A)
        if B is _A or B.loader is _A:return

C=importlib.util.module_from_spec(B);B.loader.exec_module(C);E=getat
tr(C,TAG_NAME_ATTR,_A);F=getattr(C,TAG_FN_ATTR,_A)
    if E is not _A and F is not _A:return Tagger(name=E,tag_func=F)
    def load_by_tagger_names(A,names):
        B=[]
        for D in names:
            E=A.taggers_dir/f'{D}'.py";C=A.__load_from_file(E)
            if C:B.append(C)
        return B
    def get_tagger_names(B):
        A=set()
        if not B.taggers_dir.exists():return A
        for C in B.taggers_dir.glob('*.py'):
            if C.stem!=_B:A.add(C.stem)
        return A

##### address_natasha.py #####
from text_metrics.core.span import Span
from text_metrics.external.natasha import addr_extractor
TAG_NAME='ADDRESS'
def
tag(text):A=addr_extractor(text);return[Span(A.start,A.stop,{‘part’:A.fact})
for A in A]

##### date_natasha.py #####
from text_metrics.core.span import Span
from text_metrics.external.natasha import dates_extractor
TAG_NAME='DATE'
def
tag(text):A=dates_extractor(text);return[Span(A.start,A.stop,{‘part’:A.fact})
for A in A]

##### date_re.py #####
from text_metrics.entities.base.regex_tag import regex_tagger_factory
TAG_NAME='DATE'
pattern='\b((0?[1-9]|[12][0-9]|3[01])[./-](0?[1-9]|1[0-
2])[./-](19|20)?\d{2})(19|20)\d{2}[./-](0?[1-9]|1[0-2])[./-](0?[1-9]|[12][0-
9]|3[01])(0?[1-9]|1[0-2])[./-](0?[1-9]|[12][0-
9]|3[01])[./-](19|20)?\d{2})\\b'
tag=regex_tagger_factory(pattern)

##### email.py #####
from text_metrics.entities.base.regex_tag import regex_tagger_factory
TAG_NAME='EMAIL'
tag=regex_tagger_factory('\b[\w\.-]+@[]\w\.-]+\.\w+\b')

##### enum.py #####
from text_metrics.entities.base.regex_tag import regex_tagger_factory
TAG_NAME='ENUM'
tag=regex_tagger_factory('(?!\\S)(\\d+[a-zA-Z])+[.]?(?!\\d\\w)(?=\\s|$)')

##### foreign.py #####
from text_metrics.entities.base.regex_tag import regex_tagger_factory
TAG_NAME='FOREIGN'
tag=regex_tagger_factory('\b[a-z]+(-[a-zA-Z])?*\b')

##### hashtag.py #####
from text_metrics.entities.base.regex_tag import regex_tagger_factory
TAG_NAME='TAG'
tag=regex_tagger_factory('(?!\\w)#[a-zA-Za-zA-Za-ЯёЁ\d_]+(?:=\\W|$)')

##### measurement_natasha.py #####
from text_metrics.core.span import Span
from text_metrics.external.natasha import money_extractor
TAG_NAME='MEAS'
def
tag(text):A=money_extractor(text);return[Span(A.start,A.stop,{‘part’:A.fac
t})for A in A]

##### measurement_re.py #####
from text_metrics.entities.base.regex_tag import regex_tagger_factory
TAG_NAME='MEAS'
tag=regex_tagger_factory('(\\d+[\\.\\d,]*)\\s*(кг|кило|шт|штука|ед|г|м|см|км|л|сек|мин|ч|лет|год)\\b')

##### number.py #####
from text_metrics.entities.base.regex_tag import regex_tagger_factory
TAG_NAME='NUMBER'
tag=regex_tagger_factory('\b[\\+-]?\\d+([.],\\d+)?\\b')

##### phone.py #####
from phonenumbers import PhoneNumberMatcher
from text_metrics.core.span import Span
TAG_NAME='PHONE'
def
tag(text):A=PhoneNumberMatcher(text,region='ru');return[Span(A.start,A.
end,{‘phone’:A.raw_string})for A in A]

##### pointer.py #####
from text_metrics.entities.base.regex_tag import regex_tagger_factory
TAG_NAME='POINTER'
tag=regex_tagger_factory('(пункт|п\\.\\.раздел|гл|глава|статья|стр\\.\\.№)\\s+\\d+([\\d.-]*)')
```

```

##### punct_emotion.py #####
from text_metrics.entities.base.regex_tag import regex_tagger_factory
TAG_NAME='PUNCCEM'
tag=regex_tagger_factory('[!?!]{2,}')

##### quote.py #####
from text_metrics.entities.base.regex_tag import regex_tagger_factory
TAG_NAME='QUOTE'
tag=regex_tagger_factory('«[^\»]*»|“[^\”]*”|“[^\”]*”’)

##### smile.py #####
from text_metrics.entities.base.regex_tag import regex_tagger_factory
TAG_NAME='SMILE'
pattern='([:;]|[-^~*]?[d]p([+][d]p([+][^~*]?[:;]|[[0o]_[0o]]>[<t^] [-
^]>[<t^]x[d]p[+][[-\\w]][-:][-\\w]]\\[\\w\\])[-
_][\\w\\])\\b((x|x)+x?(x|x)+x?(x|x)+x?)\\b)'
tag=regex_tagger_factory(pattern)

##### time.py #####
from text_metrics.entities.base.regex_tag import regex_tagger_factory
TAG_NAME='TIME'
tag=regex_tagger_factory('\\b(0?[0-9]|1[0-9]|2[0-3])[[:ч.,\\s-]][[0-5]][[0-9]]\\b)')

##### url.py #####
from urlextract import URLExtract
from text_metrics.core.span import Span
extractor=URLExtract()
TAG_NAME='URL'
def tag(text):return[Span(B,C,{'link':A})for(A,(B,C))in
extractor.find_urls(text,get_indices=True)]

##### natasha.py #####
from natasha import
MorphVocab,NewsEmbedding,NewsMorphTagger,Segmenter,Doc,AddrE
xtractor,MoneyExtractor,DatesExtractor
segmenter=Segmenter()
morph_vocab=MorphVocab()
emb=NewsEmbedding()
morph_tagger=NewsMorphTagger(emb)
def create_doc(text):
    A=Doc(text);A.segment(segmenter);A.tag_morph(morph_tagger)
    for B in A.tokens:B.lemmatize(morph_vocab)
    return A
addr_extractor=AddrExtractor(morph_vocab)
money_extractor=MoneyExtractor(morph_vocab)
dates_extractor=DatesExtractor(morph_vocab)

##### load_words.py #####
import csv
def from_csv(filepath,column_name):
    A=column_name;B=set()
    with open(filepath,'r',encoding='utf-8-sig')as D:
        C=csv.DictReader(D)
        if not A in C.fieldnames:raise ValueError(f'Колонка {A}
отсутствует в csv')
        for E in C:F=E[A];B.add(F)
    return B

##### lds.py #####
from text_metrics.core.context import DocumentContext
METRIC_NAME='lexical_diversity_per_sentence'
METRIC_GROUP='text'
def extract(context):A=context;B=sum(A for A in A.lexicon.values())if
A==1);return B/A.pos.sentences_count

##### lexicon_size.py #####
from text_metrics.core.context import DocumentContext
METRIC_NAME='lexicon_size'
METRIC_GROUP='text'
def extract(context):A=context;return
len(A.lexicon)/A.pos.sentences_count

##### long_words_proportion.py #####
from text_metrics.core.context import DocumentContext
LONG_WORD_LENGTH=6
METRIC_NAME='long_words_proportion'
METRIC_GROUP='text'
def extract(context):
    A=context.lexicon

```

```

        if not A:return 0
        B=sum(B for(A,B)in A.items())if
len(A)>=LONG_WORD_LENGTH);C=A.total();return B/C

##### mean_words_length.py #####
from text_metrics.core.context import DocumentContext
METRIC_NAME='mean_words_length'
METRIC_GROUP='text'
def extract(context):
    A=context.lexicon
    if not A:return 0
    B=sum(len(A)*B for(A,B)in A.items());C=A.total();return B/C

##### normalized_shannon_entropy.py #####
from text_metrics.core.context import DocumentContext
import math
METRIC_NAME='normalized_shannon_entropy'
METRIC_GROUP='text'
def extract(context):
    A=context.lexicon
    if not A:return .0
    C=A.most_common(1)[0][1]
    if C==1:return 1.
    E=A.total();D=.0
    for F in A.values():
        B=F/E
        if B>0:D=B*math.log2(B)
    return D/math.log2(C)

##### obscene_proportion.py #####
from text_metrics.core.context import DocumentContext
from text_metrics.lexicon.load_words import from_csv
from pathlib import Path
SOURCE_CSV=Path(__file__).parent.parent/'data'/obscene.csv'
COLUMN_NAME='Lemma'
if not SOURCE_CSV.exists():raise FileNotFoundError(f'Файл
{SOURCE_CSV} с перечнем самых частых слов не найден')
obscene_words=from_csv(SOURCE_CSV,COLUMN_NAME)
METRIC_NAME='obscene_proportion'
METRIC_GROUP='text'
def extract(context):
    A=context.lexicon
    if not A:return 0
    B=A.keys();C=B&obscene_words;D=sum([A[B]for B in C]);return
D/A.total()

##### rare_proportion.py #####
from text_metrics.core.context import DocumentContext
METRIC_NAME='rare_words_proportion'
METRIC_GROUP='text'
def extract(context):
    A=context.lexicon
    if not A:return 0
    B=sum(A for A in A.values())if A<=2);C=A.total();return B/C

##### ttr.py #####
from text_metrics.core.context import DocumentContext
METRIC_NAME='type_token_ratio'
METRIC_GROUP='text'
def extract(context):
    A=context.lexicon
    if not A:return 0
    B=sum(A for A in A.values())if A==1);C=A.total();return B/C

##### unpopular_proportion.py #####
from text_metrics.core.context import DocumentContext
from text_metrics.lexicon.load_words import from_csv
from pathlib import Path
SOURCE_CSV=Path(__file__).parent.parent/'data'/freq.csv'
COLUMN_NAME='Lemma'
if not SOURCE_CSV.exists():raise FileNotFoundError(f'Файл
{SOURCE_CSV} с перечнем самых частых слов не найден')
popular_words=from_csv(SOURCE_CSV,COLUMN_NAME)
METRIC_NAME='unpopular_words_proportion'
METRIC_GROUP='text'
def extract(context):
    A=context.lexicon
    if not A:return 0

```

```
B=A.keys();C=B-popular_words;D=sum([A[B]for B in C]);return
D/A.total()
```

```
##### loader.py #####
```

```
from typing import List
from pathlib import Path
import importlib.util
from ..core.metric import Metric
METRIC_NAME_ATTR='METRIC_NAME'
METRIC_GROUP_ATTR='METRIC_GROUP'
EXTRACT_FUNC_ATTR='extract'
def load_from_file(file_path):
    D=None;B=file_path
    if B.name=='__init__.py'or not B.suffix=='py':return
    E=B.stem;C=importlib.util.spec_from_file_location(E,B)
    if C is D or C.loader is D:return
    A=importlib.util.module_from_spec(C);C.loader.exec_module(A)
    if not hasattr(A,EXTRACT_FUNC_ATTR):return
```

```
F=getattr(A,EXTRACT_FUNC_ATTR);G=getattr(A,METRIC_NAME_ATTR,E);H=getattr(A,METRIC_GROUP_ATTR,D);return
Metric(name=G,group=H,extract_func=F)
def load_by_directories(directories):
```

```
    B=[]
    for A in directories:
        if not A.exists():raise FileNotFoundError(f"Директория {A} не
найдена")
        for D in A.glob("*.py"):
            C=load_from_file(D)
            if C:B.append(C)
    return B
```

```
##### consts.py #####
```

```
set_of_puncts=set(',-."()![];/~*_><&')
```

```
##### index_of_formality_heylinger.py #####
```

```
from collections import Counter
from text_metrics.core.context import DocumentContext
METRIC_NAME='index_of_formality_heylinger'
METRIC_GROUP='pos'
def extract(context):
    D=context.pos.get_series();E=Counter([A.data['pos']for A in D])
    def A(pos):return E[pos]
```

```
B=A('NOUN')+A('ADJ')+A('ADP')+A('DET');F=A('PRON')+A('VERB')+
A('ADV')+A('INTJ')+A('NUM');C=B+F
if C==0:return 50
return B/C*100
```

```
##### index_of_formality.py #####
```

```
from collections import Counter
from text_metrics.core.context import DocumentContext
METRIC_NAME='index_of_formality'
METRIC_GROUP='pos'
def extract(context):
    B=context.pos.get_series();C=Counter([A.data['pos']for A in B])
    def A(pos):return C[pos]
```

```
D=A('NOUN')+A('ADJ')+A('ADP');E=A('PRON')+A('VERB')+A('VERB
PART')+A('VERBCONV')+A('PART')+A('ADV')+A('CCONJ')+A('SCO
NJ')+A('INTJ');F=(D-E+100)/2;return max(0,min(F,100))
```

```
##### index_of_formality_tuldava.py #####
```

```
from collections import Counter
from text_metrics.core.context import DocumentContext
METRIC_NAME='index_of_formality_tuldava'
METRIC_GROUP='pos'
def extract(context):
    D=context.pos.get_series();E=Counter([A.data['pos']for A in D])
    def A(pos):return E[pos]
    B=A('NOUN')+A('ADJ');C=A('VERB')+A('ADV')+A('PRON')
    if C==0:return 100 if B>0 else 50
    F=B/C*100;return min(F,100)
```

```
##### pair_of_adv.py #####
```

```
_A='pos'
from text_metrics.core.context import DocumentContext
METRIC_NAME='pair_of_adv_per_sentence'
METRIC_GROUP=_A
```

```
def extract(context):
```

```
    F='ADV';C=context;D=C.pos.sentences_count
    if D<1:return 0
    E=0
    for B in C.pos.seqs:
        for A in range(1,len(B)):
            G=B[A-1];H=B[A]
            if G.data[_A]==F and H.data[_A]==F:E+=1;A+=2
            else:A+=1
    return E/D
```

```
##### transitions.py #####
```

```
_B='ADJ-Pos'
_A='PUNCT'
from text_metrics.core.context import DocumentContext
from text_metrics.core.span import Span
from nltk import ConditionalFreqDist,bigrams
import numpy as np
transitions=[(_B,'NOUN'),('PROPN',_A),(_A,'SCONJ'),('ADP',_B),('ADP',
_A,'PRON'),(_A,'ADV'),(_A,'PRON'),(_B,_A),(_A,'CCONJ')]
METRIC_GROUP='pos'
def extract(context):
    B='code';E=context.pos.get_series(seq_start=Span(-1,-
1,{B:'START'}),seq_end=Span(-1,-
1,{B:'END'}));C=ConditionalFreqDist()
    for(F,G)in bigrams([A.data[B]for A in E]):C[F][G]+=1
    A=np.array([C[A][B]for(A,B)in transitions]);H=A.sum()
    if H!=0:D=A/A.sum()
    else:D=np.zeros_like(A)
    return dict(zip(transitions,D.tolist()))
```

```
##### pos_tagger.py #####
```

```
from ..core.span import Span
def __get_feat(token,feature):return token.feats.get(feature,'None')
def __get_code(token):
    A=token;B=A.pos;C=B
    if B=='ADJ':C=f"ADJ-{'__get_feat(A,"Degree")}"
    elif B=='PART':C=f"PART-{'__get_feat(A,"Polarity")}"
    elif
    B=='VERB':D=__get_feat(A,'VerbForm');E=__get_feat(A,'Voice');F=__g
et_feat(A,'Tense');C=f"VERB- {D}-{E}-{F}"
    return C
def __get_span(token):A=token;B=A.pos;C=__get_code(A);return
Span(A.start,A.stop,{ 'pos':B,'code':C,'text':A.text})
def tag(doc,entity_tagger):
    B=[];C=[]
    for E in doc.sents:
        D=[]
        for A in E.tokens:
            if not
entity_tagger.is_contains(A.start,A.stop):D.append(__get_span(A));C.appe
nd(A.lemma)
            B.append(D)
    return B,C
```

```
##### processor.py #####
```

```
from pathlib import Path
from entities.TaggerLoader import TaggerLoader
from metrics.loader import load_by_directories
from documents.doc import extract_metrics
tagger_names=['email','url','hashtag','phone','address_re','address_natasha','
date_re','date_natasha','timemeasurement_natasha','measurement_re','enum
','number','pointer','quote','smile','punct_emotion','foreign']
taggers=TaggerLoader().load_by_tagger_names(tagger_names)
metric_dirs=['./documents/metrics','./entities/metrics','./lexicon/metrics','./p
os/metrics','./punct/metrics']
base_dir=Path(__file__).parent.resolve()
metric_paths=[(base_dir/Path(A)).resolve()for A in metric_dirs]
metrics=load_by_directories(metric_paths)
def extract(text):A=text;A=A.replace('ё','е');return
extract_metrics(A,taggers,metrics)
def flat_metrics(metrics):
    A={}
    for(B,C)in metrics.items():
        for(D,E)in C.items():F=f"{B}__{D}";A[F]=E
    return A
```